Nucleus

A Deterministic Algorithm/Schedule Co-Compiler with First-Class IO Semantics and a Petri-Net Soundness Gate

Technical Report

Mark Ruvald Pedersen

June 2026

Abstract

Portable parallel software couples three concerns that ought to be separable: the *algorithm* (what to compute), its *decomposition and IO* (how to lay it out across workers in space and time, and with what IO semantics), and the *target platform*. Moving an algorithm from CPU threads to a message-passing cluster to an embedded microcontroller today means rewriting it, because decomposition and IO semantics are written by hand, per platform, and tangled into the source. Compute scheduling has first-class language support; IO scheduling does not.

This document presents Nucleus, a source-to-source compiler — it emits Rust that a downstream host toolchain builds — founded on a strict separation between an algorithm sublanguage and a schedule sublanguage. Decomposition, IO semantics (synchronous or asynchronous, buffered or unbuffered, event-driven or polled, shared-memory or message-passing or DMA), and target are first-class schedule directives that the algorithm never sees: one algorithm composes with many schedules, and each schedule composes with many backends. A single Petri-net intermediate representation unifies transfer synthesis, synchronisation, and a compile-time soundness gate, so that — for the restricted class of nets the language produces — boundedness and deadlock-freedom are reported as compile errors rather than runtime surprises. The genericity claim is made mechanically falsifiable by a cross-backend bit-identical differential test: the same algorithm, under every schedule and backend, must produce byte-identical output.

The implementation drives twenty-nine worked examples through ten backends across three tiers — seven shared-memory and message-passing CPU backends, two MPI backends, and one embedded backend with multi-microcontroller co-simulation. The seven CPU backends agree byte-for-byte across the full schedule-by-backend matrix; the embedded backend reproduces reference output byte-exactly under co-simulation. The work is honest about its boundary: the algorithm class is affine, static, single-assignment, and integer-valued, and the schedule is checked, not optimised.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 The portability problem

A parallel algorithm is rarely written once. The same computation — a stencil blur, a blocked matrix multiply, a small neural-network forward pass — is expected to run on a developer's laptop using operating-system threads, to scale out across a message-passing cluster, and, increasingly, to run on an embedded microcontroller inside a product. In each setting the *arithmetic* is identical. What changes is everything around it: how the data is partitioned across workers, how those workers exchange intermediate results, and what input/output (IO) semantics govern that exchange — whether a transfer blocks or returns immediately, whether it is buffered, whether the receiver is woken by an event or discovers the data by polling, and whether the bytes move through shared memory, a socket, a direct-memory-access (DMA) engine, or a cluster interconnect.

Today, moving an algorithm between these settings means rewriting it. The decomposition and the IO semantics are written by hand, once per platform, and tangled directly into the source. A threads version, an MPI version, and a microcontroller version of the "same" algorithm are three separate programs that happen to compute the same function, maintained in parallel, drifting apart with every change. The cost of this duplication is not the arithmetic — the kernel bodies are small and stable — but the decomposition and IO glue, which is large, platform-specific, and the part most likely to harbour a deadlock, a buffer overrun, or a subtle ordering bug.

This document argues that the duplication is avoidable, and that avoiding it requires treating decomposition and IO not as code to be written but as *schedule directives* to be chosen — separately from, and without modifying, the algorithm they apply to.

1.2 Motivation

Three observations motivate the work, in decreasing order of how much weight they carry.

IO has been a second-class citizen. The separation of an algorithm from its *schedule* — the choices of tiling, fusion, vectorisation, and parallelism that determine performance without changing the result — is a mature idea for *compute*. Halide [20] made it the organising principle of an image-processing language and compiler; TVM [5], Tiramisu [2], and Exo [11] carried it into deep learning and hardware accelerators. In all of these systems the schedule controls how computation is laid out, but the *IO* that connects parallel workers — the transport, the synchronisation, the buffering discipline — is either implicit in a fixed runtime or written by hand outside the language. There is no schedule directive for "use asynchronous double-buffered transfers here, blocking unbuffered ones there." This document makes IO semantics a first-class schedule directive, swappable per target without touching the algorithm.

Hardware changes faster than algorithms do. An algorithm written for x86 with threads should not have to be rewritten for an MPI cluster, an ARM system-on-chip, or a custom accelerator — yet today it usually is. The parallel runtimes that target these platforms — MPI [16] for clusters, OpenMP [6] for shared memory, and async embedded frameworks such as Embassy [22] for microcontrollers — each impose their own programming model and their own hand-written IO. They are excellent at what they do, but they do not compose: a program written against one is not a program against another. The algorithm/schedule split delivers the most value precisely where the hardware churns under a stable algorithm.

Decomposition is the bottleneck. Algorithm code is comparatively cheap to write and easy to get right. Deciding how to decompose it across heterogeneous workers, with explicit and correct IO, is the expensive, error-prone part. The right response is to make *that* the cheap thing to iterate on: to let an engineer change the decomposition, the IO discipline, or the target platform by editing a small schedule file, while the algorithm stays fixed and the compiler does the rewriting.

1.3 Thesis statement and contributions

Thesis statement. *A parallel algorithm can be written once, in a form that contains no decomposition and no IO, and then be mapped onto shared-memory threads, a message-passing cluster, and an embedded microcontroller purely by changing a separate schedule — with the compiler synthesising all data transfers*

and synchronisation, guaranteeing at compile time — for the restricted class of nets the language produces — that no declared communication buffer overflows and that the program cannot deadlock, and with the claim that the algorithm is genuinely unchanged made mechanically falsifiable by requiring byte-identical output across every schedule and every CPU backend.

The system that realises this statement is called NUCLEUS. Its contributions are:

1. **A strict algorithm/schedule separation in which decomposition and IO are first-class schedule concerns.** The algorithm is expressed in one sublanguage that has no notion of workers, buffers, transports, or synchronisation; the schedule is expressed in a second sublanguage in which the number of workers, the data partitioning, the loop transformations, and the IO semantics are all explicit directives. One algorithm composes with many schedules; each schedule composes with many *backends* — code-generation paths, each targeting one combination of runtime and transport, such as shared-memory threads, message passing, or embedded DMA (fig. 4.1).

2. **Compiler-synthesised, statically scheduled communication unified by a single Petri-net intermediate representation (IR).** From the declared data-access patterns the compiler infers which data each worker must send to each other worker, including the *halo regions* (the boundary slices of a neighbour's data a stencil needs). This is transfer *synthesis* — deriving the communication for a decomposition the programmer has already fixed — rather than automatic parallelisation, which would discover parallel structure from scratch. The whole program is lowered to one Petri net (a directed bipartite graph of places and transitions, defined in section 2.4) whose transitions are kernel firings, transfers, and synchronisations.

3. **A compile-time soundness gate.** The gate is not a separate analysis bolted on; it is an intrinsic reading of the same IR. Because the firing order — the order in which the program's operations execute — is statically determined for the restricted net class, boundedness (no declared buffer ever overflows) and deadlock-freedom (no required transfer ever waits on a value that is never produced) are decided at compile time and reported as compile errors, not discovered as runtime hangs or crashes.

4. **A falsifiable genericity claim.** The assertion that the algorithm is truly independent of schedule and backend is tested, not merely asserted: the same algorithm, compiled under every schedule and every CPU backend, must produce byte-identical output against a hand-written reference. A single differing byte falsifies the separation and points at which boundary leaked.

5. **A realised target ladder.** The model is carried from theory to running code across three tiers: seven CPU backends (the cheap falsification rig),

two MPI backends, and one embedded backend demonstrated by multi-microcontroller co-simulation — ten backends in total, driving twenty-nine worked examples.

1.4 Scope and non-goals

The portability claim is deliberately bounded, and the boundary is stated up front rather than buried. NUCLEUS is portable *for algorithms whose decomposition fits a restricted model*, not for all algorithms. Concretely, the algorithm class is:

- **Affine and static.** Loop bounds and array indices are affine functions of the iteration variables — of the form $a \cdot i + b$ for compile-time constants a and b — and the decomposition and firing order are fixed at compile time. There is no dynamic scheduling, no work stealing, and no load balancing.
- **Single-assignment.** Each array location is written exactly once. The producer of every value is therefore unambiguous, which lets the compiler derive inter-worker transfers by comparing read and write index sets without having to reason about whether two names refer to the same storage (aliasing). This is what makes transfer synthesis sound rather than best-effort.
- **Integer-valued for the bit-identity guarantee.** Byte-identical cross-backend output requires deterministic arithmetic. Examples are integer-valued, or use floating-point only where reductions do not reorder; an example that would reorder a floating-point sum is restated or excluded.

The following are explicit non-goals. NUCLEUS does *not* perform automatic parallelisation: the programmer annotates the decomposition, and the compiler maps and schedules it (it does, however, infer transfers and halos from declared access patterns, which is a narrower and decidable problem). Data-dependent array *indexing* is supported, in both single-worker and distributed form: a data-dependent *read* (a gather, $x[c[k]]$ where c is itself data) and a data-dependent *write* (a scatter, such as a histogram indexed by data) both compile and run byte-identically across the CPU backends. The data-dependent dimension is handled conservatively — the gathered array is broadcast in whole rather than sliced, and a scattered target is replicated per worker and recombined — which preserves correctness at some cost in communication volume. What remains *outside* the model is data-dependent control *flow* (convergence-driven loop termination), a data-dependent loop trip count, and recursion; consequently full sparse solvers, which need convergence-driven termination, are out of scope. A scatter whose write target is itself partitioned along an affine axis is conservatively rejected at compile time rather than

mis-compiled. NUCLEUS performs *no* polyhedral analysis (the mathematical framework that represents loop nests as polyhedra to automate dependence analysis and loop transformation) [24]: single-assignment plus explicit views suffice for the targeted class, and a case that genuinely needs the polyhedral model is out of scope. Finally, the schedule's runtime assertions are *checked, not prescriptive*: the compiler reports whether a hand-written schedule meets a stated latency or buffer bound, but has no cost model and does not search for a schedule that does. Chapter 11 returns to these limits as honest threats to the contribution.

1.5 Structure of this document

Chapter 2 introduces the background concepts the rest of the work builds on — parallel decomposition and separation of concerns, IO semantics as a programming concern, static versus dynamic scheduling, Petri nets, and the determinism that differential testing depends on — neutrally and without assuming the contribution. Chapter 3 surveys the prior art, positions NUCLEUS against the compute schedulers and parallel runtimes it borrows from and composes with, and draws the line to the author's own 2013 prototype [18], whose genericity claim could not be falsified because it had a single target. Chapter 4 states the portability gap precisely and maps the design space. Chapter 5 presents the two sublanguages; chapter 6 the compilation pipeline (fig. 6.1) and the Petri-net IR; and chapter 7 the ten backends and the target ladder. Chapter 8 then traces a single program — the 3×3 stencil — through every compiler stage, from the two source files to generated code on three backends, making the abstract pipeline concrete before the validation chapters. Chapters 9 and 10 present the validation methodology — the bit-identical differential rig, the soundness gate, and the negative falsifiers that prove the checks bite — and the results obtained. Chapters 11 to 13 discuss advantages and shortcomings, sketch future work, and conclude. Appendices catalogue the worked examples, the grammar, the capability matrix, and the reproducibility setup.

Chapter 2

Background

This chapter introduces the concepts the rest of the document depends on. It is deliberately neutral: it presents established ideas — separation of concerns in parallel programming, the dimensions of IO semantics, static versus dynamic scheduling, Petri nets, and differential testing — without yet assuming the contribution. A reader already fluent in any of these sections may skip it.

2.1 Parallel decomposition and separation of concerns

A sequential program states *what* to compute. A parallel program must also state *how* the work is divided across processing elements and *how* those elements coordinate. These are different concerns, and conflating them is the source of much of the difficulty in parallel programming: a change to the data layout for one machine forces edits all through code that expresses the mathematics, and the mathematics becomes hard to read through the layout.

The decomposition itself has two axes. *Spatial* decomposition divides data or iterations across workers that run concurrently — domain decomposition of a grid, blocking of a matrix, partitioning of an array. *Temporal* decomposition orders work within a worker and overlaps it across workers — loop tiling, software pipelining (overlapping successive loop iterations so workers stay busy), and double buffering (alternating between two buffers so a producer can fill one while the consumer drains the other). Both axes are scheduling decisions: they change performance and resource use, but, applied correctly, they do not change the computed result.

The idea that the result-defining part of a program should be written once and the performance-defining part chosen separately has a clear lineage in high-performance compute. Halide [20] crystallised it as the explicit split between an *algorithm* (pure functions over an index domain) and a *schedule* (the tiling, fusion, vectorisation, and parallelism applied to it), and showed that a single algorithm could be retargeted across schedules by orders of magnitude

in performance without touching its definition. The polyhedral compilers [2, 4] pursue the same separation through a more powerful mathematical model. What unites them is the discipline that the algorithm never mentions the schedule. This document adopts that discipline and extends it from scheduling computation *within a single machine* (intra-node) to decomposition and IO *across many machines or workers* (inter-node). A middle ground between fully manual and fully automatic decomposition is to have the programmer annotate the decomposition explicitly — in a separate schedule sublanguage — while leaving the compiler to infer the resulting transfers from those annotations; that is the approach taken here and developed in chapter 5.

2.2 IO semantics as a programming concern

When two workers exchange data, the exchange is governed by choices that are usually made implicitly by whatever runtime is in use, but which materially affect correctness and performance. These choices form a small set of orthogonal dimensions:

Synchronous vs. asynchronous. Does the producer block until the transfer completes, or does it initiate the transfer and continue, completing it later? Asynchrony enables latency hiding but requires somewhere to track the in-flight operation.

Buffered vs. unbuffered. Is there storage that decouples producer and consumer rates, allowing the producer to run ahead by a bounded number of items, or must each transfer rendezvous directly?

Event-driven vs. polled. Is the consumer woken when data arrives (by an interrupt, a condition variable, or an event-loop readiness notification), or does it repeatedly test for arrival?

Transport. Do the bytes move through shared memory, a loopback or network socket, a Unix domain socket, a cluster interconnect via a message-passing library, or a DMA engine on an embedded device?

These dimensions are largely independent of one another and of the algorithm. A shared-memory threads runtime such as OpenMP [6] fixes one combination; a message-passing library such as MPI [16] offers several (blocking and non-blocking sends, buffered and synchronous modes) but within its own model; an embedded async framework such as Embassy [22] fixes yet another around interrupts and DMA. Crucially, none of them lets these choices be *stated* as part of a schedule and *swapped* without rewriting the communication code. That gap — IO semantics as an unspoken property of the chosen runtime rather than a first-class, selectable directive — is the gap this work targets.

2.3 Static versus dynamic scheduling

A *schedule* assigns operations to workers and orders them in time. Scheduling may happen dynamically, at run time — as in work-stealing thread pools or dataflow runtimes that fire operations when their inputs become available — or statically, at compile time, fixing the order before the program runs.

Dynamic scheduling adapts to data-dependent and machine-dependent variation, at the cost of runtime overhead, nondeterminism, and analyses that can only be approximate. Static scheduling gives up that adaptivity but gains three properties this work depends on. First, *determinism*: a fixed firing order means a fixed result and a fixed communication pattern, which is what makes byte-identical cross-backend comparison meaningful. Second, *analysability*: with the order known at compile time, properties such as buffer sufficiency and freedom from deadlock become decidable by replaying that one order, rather than searching a reachable state space. Third, *suitability for embedded real-time targets*, where dynamic scheduling is often unacceptable regardless. Static scheduling of dataflow has a long pedigree — synchronous dataflow [13] schedules a restricted graph entirely at compile time precisely to remove runtime overhead — and this work sits firmly in that static tradition. The scope restrictions stated in section 1.4 — affine bounds, single-assignment arrays, and a statically fixed decomposition — are exactly what place this work in the static-scheduling camp and make the analyses of the following sections tractable.

2.4 Petri nets

A Petri net [17, 19] is a bipartite directed graph with two kinds of node: *places* and *transitions*. Places hold *tokens*; a marking is an assignment of a token count to every place. A transition is *enabled* when each of its input places holds enough tokens, and *firing* it removes tokens from its input places and deposits tokens on its output places, producing a new marking. The net's behaviour is the set of markings reachable from an initial marking by firing enabled transitions.

A handful of standard properties characterise such a net. A net is *bounded* if no place can ever exceed a stated token capacity, and *k-bounded* if that capacity is k. It is *live* if no transition is permanently disabled. A *deadlock* is a reachable marking in which no transition is enabled — the system cannot make progress. For general Petri nets these properties are decidable but expensive; for restricted subclasses they become cheap.

Petri nets are a natural model for the problem at hand because their vocabulary maps directly onto statically scheduled communication. A transition models a kernel firing, a data transfer, or a synchronisation; a place models a data slot, a communication channel, or a barrier; a token models the presence

of a value or a unit of buffer credit; a place's capacity models a declared buffer size; and tokens placed on a channel in the initial marking model the head-start that double-buffering or software-pipelining provides. Under this mapping, "does this schedule's communication stay within its declared buffers?" is exactly boundedness, and "can this schedule's communication hang?" is exactly the reachability of a deadlocked marking. Chapter 6 shows how restricting the net to a statically determined firing order collapses these checks to a single deterministic replay.

2.5 Determinism and differential testing

Differential testing [15] establishes confidence in a system by comparing the outputs of multiple implementations that are supposed to agree: where they differ, at least one is wrong, and the disagreement localises the fault. It is especially powerful for compilers, where an oracle for "the correct output" is otherwise hard to obtain; the Csmith [25] work famously used differential testing across C compilers to find hundreds of bugs by generating programs whose output every correct compiler must agree on.

Differential testing requires a deterministic notion of agreement. Two implementations can only be compared byte-for-byte if both are deterministic for the same input. This is why the algorithm class in this work is restricted to deterministic arithmetic (section 1.4): integer computation, or floating-point computation whose reductions do not reorder, guarantees that a single reference output exists for every example. Given that determinism, differential testing becomes a means not only of finding bugs but of *falsifying a design claim*. If one algorithm is compiled under many schedules and many backends and all outputs are byte-identical to a single hand-written reference, then the claim that the algorithm is independent of schedule and backend has survived a concrete attempt to break it; a single differing byte refutes it. Chapter 9 develops this rig in full.

Chapter 3

Prior Art and Literature Study

With decomposition, IO semantics, static scheduling, and Petri nets established (chapter 2), this chapter can position NUCLEUS precisely against the traditions it borrows from or declines to follow: the compute schedulers from which it takes its central discipline, the parallel runtimes it composes with rather than replaces, the polyhedral line it deliberately sets aside, and the dataflow and Petri-net scheduling on which it builds its IR. It then draws the line to the author's own 2013 prototype, whose ambition this work shares but whose central claim it could not test, and closes by stating precisely the gap NUCLEUS fills.

3.1 Compute schedulers

The closest intellectual ancestors of this work are the languages that made the algorithm/schedule separation explicit for compute. Halide [20] separates an image-processing algorithm — pure functions over an index domain — from a schedule that dictates tiling, fusion, vectorisation, parallelisation, and the trade-off between recomputation and storage. The same algorithm retargets across very different schedules with no change to its definition, and the schedule is where all performance engineering happens. TVM [5] carried the idea into deep-learning compilation, adding automated schedule search over a large space of tensor-program transformations. Tiramisu [2] backed the separation with a polyhedral representation and a multi-layer IR that separates the algorithm, loop transformations, data layout, and communication. Exo [11] pushed scheduling control outward to user-level libraries, so that custom hardware instructions and memories can be described outside the compiler.

These systems are the proof that the algorithm/schedule split is both expressive and practical, and NUCLEUS adopts their discipline wholesale. The difference is one of *what the schedule controls.* In all of them the schedule governs

how *computation* is laid out on a target whose communication and IO are fixed — by a shared-memory backend, a tensor runtime, or a generated kernel that runs in one address space. None of them makes the *decomposition across distributed workers* and the *IO semantics* of the transfers between them schedule-level directives. Tiramisu represents communication in its IR, but as a layer the compiler manages for its targets, not as a palette of synchronous/asynchronous, buffered/unbuffered, event/poll, shared-memory/socket/DMA choices a user selects per platform. That extension — the schedule as the place where transport and synchronisation are chosen, swappable without touching the algorithm — is what this document contributes on top of the compute-schedule tradition.

3.2 Parallel runtimes and models

Where the compute schedulers are the conceptual ancestors, the parallel runtimes are the *substrate*: Nucleus does not replace them but emits code against them, treating each as a backend. MPI [16] is the dominant model for distributed-memory clusters, offering an explicit and rich vocabulary of point-to-point and collective communication with blocking and non-blocking, buffered and synchronous variants. OpenMP [6] is the standard for shared-memory parallelism, expressing parallel loops and regions through compiler directives. For embedded systems, async frameworks such as Embassy [22] structure interrupt- and DMA-driven IO around Rust's asynchronous execution model.

Each is excellent within its domain, and each fixes a programming model. The consequence is that the IO and decomposition written against one do not transfer to another: an MPI program is not an OpenMP program is not an Embassy program, even when they compute the same function. The engineer who must span all three writes the algorithm three times, or, more precisely, writes it once and writes its decomposition and IO three times around it. Nucleus positions itself one level above: the algorithm and the decomposition are written once in a platform-neutral form, and the compiler emits the MPI, the threaded, or the embedded realisation — each runtime appearing as a backend that lowers the same intermediate communication contract into its own primitives (a transfer becomes a memory copy, a socket write, an `MPI_Isend`, or a DMA descriptor enqueue depending on the backend).

3.3 Polyhedral compilation

The polyhedral model represents loop nests as integer points in parametric polyhedra and expresses transformations as affine maps over them, enabling powerful automatic loop transformation and parallelisation. Tools in this line — the isl integer set library [24], the Pluto automatic paralleliser [4], and the

affine scheduling theory behind them [9] — can extract parallelism and locality from sequential loop nests far beyond what a syntactic approach reaches.

This work deliberately declines to adopt the polyhedral model. The algorithm class targeted here is affine and single-assignment, and the decomposition is *annotated* by the programmer rather than discovered by the compiler; given those restrictions, explicit array views plus single-assignment are sufficient to infer the inter-worker transfers, and the heavy machinery of integer set manipulation is not needed. The trade-off is honest and explicit: the polyhedral approach is strictly more powerful and could handle programs Nucleus rejects, at the cost of substantial implementation complexity and a less predictable compilation outcome. Where a real case genuinely needs the polyhedral model, it falls outside this work's scope; chapter 12 revisits this as a possible extension.

3.4 Petri-net and dataflow scheduling

The intermediate representation in this work is a Petri net, chosen because its formal vocabulary maps directly onto statically scheduled communication (section 2.4). Petri nets [17, 19] provide a well-studied theory of boundedness, liveness, and deadlock, and a large body of analysis techniques. The model used here is a small, restricted subclass — bounded by construction (every place has a finite declared capacity, so its reachable state space is finite), with a statically determined firing order and no coloured, stochastic, or hierarchical extensions — which is what allows the analyses to run as a cheap per-build gate rather than a general model checker.

The static-dataflow scheduling tradition is the other relevant line. Synchronous dataflow [13] schedules graphs in which each actor produces and consumes a fixed number of data tokens per firing, so the whole schedule reduces to a static sequence of firings computed once at compile time — a far simpler analysis than a general Petri net; the StreamIt language and compiler [23] operationalised this synchronous-dataflow discipline into a streaming language whose steady-state schedule the compiler computes once. Kahn process networks [12] give a deterministic semantics to networks of sequential processes communicating over *unbounded* channels. Nucleus's restricted nets sit in the same statically schedulable, deterministic region of the design space, and can be read as a bounded specialisation of these models: the firing order is fixed at compile time, the unbounded channels are replaced by declared buffer capacities that the soundness gate enforces, and the result is independent of timing. The contribution relative to this tradition is not a new scheduling theory but the use of one unified Petri-net IR as the single point where transfer synthesis, synchronisation, and the soundness gate all meet, and from which every backend's per-worker code is projected.

3.5 Static-schedule streaming, multi-backend array languages, and data-partitioning systems

Two further lines sit especially close to this work's design point and deserve an explicit distinction, lest the contribution be mistaken for one of them. The first is the static-schedule streaming tradition that StreamIt [23] exemplifies: a stream graph of actors with fixed per-firing token rates, whose steady-state schedule the compiler computes once, at compile time — exactly the property, a statically determined firing order, that this work's central trade also buys. The distinction lies in what is scheduled, and across what. StreamIt fixes a stream-graph structure over fixed token rates and targets primarily tiled and cluster architectures for throughput; NUCLEUS instead separates an affine array *algorithm* from a *schedule* in which decomposition and IO semantics — synchronous or asynchronous, buffered or unbuffered, event or poll, over shared memory, sockets, or DMA — are user-selectable directives, retargeted across shared-memory, message-passing, and bare-metal embedded backends and gated by a Petri-net soundness check the streaming tradition has no direct analogue of.

The second is the one-source, multi-backend array-language tradition, of which Futhark [10] is the canonical modern instance: a purely functional array program compiled, unchanged, to very different backends such as multicore CPU and GPU. NUCLEUS shares the "write the array computation once, compile it to many targets" ambition but reaches it by the opposite route. Futhark's parallelism is *discovered and mapped by the compiler* from a functional source with no user-facing schedule and no notion of IO semantics or distributed and embedded transports; NUCLEUS's portability is over an *explicitly scheduled, programmer-annotated* decomposition with first-class IO directives and a compile-time soundness gate, reaching the message-passing and embedded tiers Futhark does not target. The contrast also sharpens this work's deliberate non-goal: it does not discover parallelism (section 4.5).

Closest on the distributed-decomposition axis is Legion [3], which makes data partitioning and dependence-driven communication first-class, separately-described concerns on distributed-memory machines. Here the distinction is the scheduling regime. Legion schedules *dynamically* at run time, through a deferred-execution model that discovers task dependences as the program runs, whereas NUCLEUS fixes the firing order *statically* to obtain its compile-time soundness gate and its byte-identical determinism — trading Legion's dynamic adaptivity for decidable, ahead-of-time guarantees suited to hard-real-time and embedded targets. None of these three does what the next section identifies as the open gap: make IO semantics a schedule-level directive across distributed *and* embedded targets under a single gated Petri-net IR.

3.6 From the 2013 prototype to this work

The ambition of this document is not new to its author. A 2013 prototype [18] introduced an earlier version of the Nuc language and sketched the same core idea: annotate a computation so that a compiler can produce decomposed, scheduled code — firmware for one image-signal-processor platform. That prototype, however, had a single target — it generated code for that one platform through one backend — and was written as a C++ system with kernels represented as text fragments substituted into templates, with data transfers inferred but not statically scheduled and with aliasing left unresolved.

The single most important limitation was epistemic rather than technical: *with one target, the genericity claim could not be falsified.* A system that emits code for exactly one platform cannot demonstrate that its algorithm representation is genuinely independent of the platform, because there is nothing to vary. The claim "the algorithm does not depend on the target" was a design aspiration with no test that could refute it.

This work is the realisation of what the prototype only sketched, with the specific design decisions chosen to make the central claim testable:

- **Many backends, so genericity is falsifiable.** A single dead target becomes a ladder of ten backends across three tiers; the cross-backend differential test (chapter 9) is the falsifier the prototype lacked.
- **Real kernels, not text substitution.** Kernels are typed functions whose signatures carry array-dimension (shape) information and which are compiled by the host toolchain, replacing the fragile string-fragment templating of the prototype.
- **Single-assignment arrays.** Aliasing is removed by construction, which is what makes transfer synthesis sound rather than best-effort.
- **Statically scheduled, gated communication.** Transfers are scheduled at compile time and the resulting net is checked for boundedness and deadlock-freedom as a per-build gate, replacing the prototype's ad-hoc, ungated analyses.
- **IO semantics as first-class schedule directives**, where the prototype tangled decomposition and target annotations into the algorithm source itself.

The purpose of the present work is therefore not to redo the prototype but to build the thing it only gestured at, on commodity hardware, with the genericity claim mechanically testable.

3.7 Positioning

The gap NUCLEUS fills can now be stated precisely. The compute schedulers (section 3.1) made the algorithm/schedule split first-class but kept it inside a single address space with fixed IO. The parallel runtimes (section 3.2) made distributed and embedded IO expressible but only by hand, once per platform, with no algorithm-level portability. The polyhedral line (section 3.3) made loop transformation powerful but at a complexity and unpredictability this work does not need for its target class. The dataflow and Petri-net tradition (section 3.4) supplied the static, deterministic, analysable scheduling substrate but was not connected to a schedule language in which decomposition and IO are user-selectable directives.

NUCLEUS occupies the intersection: it takes the algorithm/schedule discipline of the compute schedulers, extends the schedule to govern *decomposition and IO semantics* across distributed and embedded targets, unifies the result in a single restricted Petri-net IR that carries a compile-time soundness gate, and makes the whole claim falsifiable by a cross-backend bit-identical differential test. No prior system is known to combine these — the closest, set out in section 3.5, each diverges on one of them — and that combination is the contribution the remaining chapters develop and defend.

Chapter 4

Problem Statement and Solution Space

The previous chapters established the portability problem informally and positioned it against the literature. This chapter states the gap precisely, maps the space of possible solutions, and justifies the central design commitments — the algorithm/schedule separation and the single Petri-net intermediate representation — before any implementation detail. The defence of those commitments is deferred to the chapters that realise them; here the argument is only that they are the right commitments to make.

4.1 The portability gap, precisely

Consider a parallel computation C and a set of target platforms $T_1, \ldots, T_n$ that differ in their concurrency model and their communication substrate: shared-memory threads, distributed processes over sockets, ranks of a message-passing cluster, cores of an embedded system-on-chip. To run C on T_i a programmer must supply three things: the *arithmetic* of C; a *decomposition* that assigns the work and data of C to the concurrent units of T_i; and the *IO* that moves intermediate results between those units with definite semantics (synchronous or asynchronous, buffered or not, event-driven or polled, over the substrate T_i provides).

The gap is this: only the first of the three is naturally portable. The arithmetic of C is the same on every T_i, but in current practice the decomposition and the IO are written by hand, separately for each target, and interleaved with the arithmetic in the source. The artefact that runs on T_i is therefore not C but a target-specific program P_i in which C is no longer cleanly recoverable. There are n such programs, they must be maintained together, and the parts that differ between them — the decomposition and the IO — are exactly the parts most prone to deadlock, buffer mismanagement, and ordering errors.

A precise statement of the goal follows directly. The goal is a representation in which C is written *once*, with no decomposition and no IO in it, together with a separate, per-target description of the decomposition and IO, such that a compiler can mechanically produce each P_i. Success is not subjective: if the single C truly contains no target-specific information, then every P_i generated from it must compute the same function, and that can be tested by running them on common input and comparing outputs.

4.2 The design space

Where can the decomposition and IO live? There are four broad options, and the choice among them determines everything that follows.

In the algorithm. The traditional answer: write the decomposition and IO directly into the source, once per target. This is maximally expressive — anything the target supports can be expressed — but it is exactly the status quo whose cost the previous section described. The arithmetic is no longer portable because it is no longer separable.

In a general-purpose runtime. Delegate decomposition and IO to a library or runtime — a thread pool, a message-passing library, an async executor. This factors the mechanism out of the source, but it does not make the *choice* of mechanism portable: a program written against one runtime is not a program against another, and the runtime fixes the IO semantics rather than exposing them as choices. Portability across runtimes is not achieved.

Inferred automatically by the compiler. Have the compiler discover the decomposition from the sequential source, as automatic parallelisers and polyhedral compilers do. This maximises convenience but at the cost of predictability and of a powerful, complex analysis; and for irregular cases the discovery fails or produces something the programmer cannot control. The portion that *is* decidable cheaply — given an explicit decomposition, inferring the data transfers it implies — is worth keeping; the open-ended discovery is not.

In a separate schedule the compiler consumes. Have the programmer state the decomposition and IO explicitly, but in a *separate* description that names the parts of the algorithm without being part of it, and have the compiler combine the two. This keeps the arithmetic portable (it never mentions a target), keeps the decomposition explicit and predictable (the programmer writes it, the compiler does not guess it), and lets the IO semantics be stated as selectable directives rather than baked into a runtime. The compiler still does

the genuinely mechanical work — synthesising the transfers the decomposition implies and lowering them to each target.

This last option is the one this work takes. It is the design point that the compute-scheduling languages reached for performance, applied instead to decomposition and IO across distributed and embedded targets.

4.3 The central commitment: algorithm/schedule separation

The architecture commits to a strict separation between two artefacts: an *algorithm*, which states what to compute and contains no worker, no buffer, no transport, and no synchronisation; and a *schedule*, which states how to decompose the algorithm across workers and with what IO semantics, and which names the algorithm's kernels and loops without redefining them.

The separation is meaningful only if it composes in both directions (fig. 4.1). One algorithm must compose with many schedules: the same arithmetic, decomposed first across threads, then across cluster ranks, then across embedded cores, by editing only the schedule. And one schedule must compose with many backends: the same decomposition, lowered to whichever runtime and transport the chosen backend provides. The boundary is real exactly when either file can be changed independently and the other still works — a property that is not asserted but tested (chapter 9).

The payoff of getting the boundary right is that the expensive, error-prone work — choosing and tuning the decomposition and IO — becomes a small, isolated, swappable description, while the arithmetic, written once, is never touched again. The risk is that the boundary leaks: that some target-specific assumption creeps into the algorithm, or some algorithmic detail leaks into the schedule. The differential test is designed precisely to expose such leaks.

4.4 Why a single Petri-net IR

Combining an algorithm with a schedule produces a concurrent program with inter-worker communication, and three questions about that communication must be answered before any code is emitted. Does it stay within its declared buffers (boundedness)? Can it hang (deadlock)? Is its outcome independent of the order in which independent events happen to occur (determinism)? Answering these with separate, ad-hoc analyses — one for buffer sizing, one for deadlock, each with its own model of the program — invites the analyses to disagree with each other and with the code that is finally generated.

The commitment here is to answer all three from *one* model: a Petri net (section 2.4) whose transitions are the program's kernel firings, data transfers, and synchronisations, and whose places are its data slots, communication

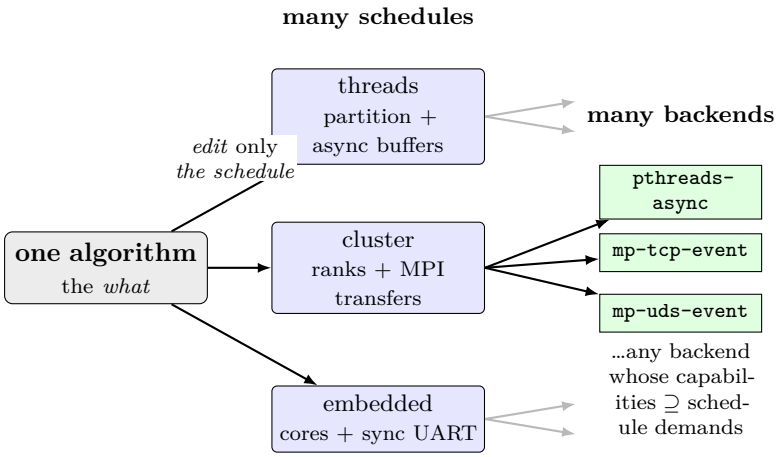


Figure 4.1: The separation is meaningful only if it composes in both directions. *One algorithm composes with many schedules*: the same arithmetic is decomposed first across threads, then across cluster ranks, then across embedded cores, by editing only the schedule. *Each schedule composes with many backends*: the same decomposition is lowered to whichever runtime and transport a backend provides, and a schedule compiles against any backend whose capabilities are a superset of what it demands (only one schedule's fan-out is drawn in full; the grey stub arrows indicate the other schedules fan out the same way). The boundary is real exactly when either file can be changed independently and the other still works — a property that is tested, not asserted.

channels, and barriers. Buffer capacity becomes a place capacity; the head-start of a pipelined or double-buffered transfer becomes an initial marking; boundedness becomes the question of whether any place exceeds its capacity, and deadlock becomes the reachability of a marking in which no transition can fire. One representation carries transfer synthesis, synchronisation, and the soundness questions together, and the code emitted for each worker is a projection of that same net — its per-worker slice of events, extracted as defined in section 6.8 — so the analysis and the artefact cannot drift apart.

Using the full theory of Petri nets would be both unnecessary and expensive. The nets this work builds are deliberately a small, restricted subclass — bounded by construction and with a statically determined firing order — which is what allows the soundness questions to be decided cheaply, as the next section commits and section 6.7 realises.

4.5 Design commitments and deliberate non-goals

The genericity claim — "the algorithm is independent of schedule and backend" — is only worth making if it can be falsified. Several design commitments exist specifically to keep it falsifiable, and several capabilities are deliberately given up to keep the model tractable.

Determinism, hence a restricted algorithm class. Byte-identical comparison across backends is only meaningful if each backend's output is itself deterministic. The algorithm class is therefore restricted to deterministic computation: affine loop bounds and indices, single-assignment arrays (no aliasing, so the producer of every value is unambiguous), and integer-valued arithmetic — or floating-point only where reductions do not reorder. This is a real restriction, stated plainly: it is what makes the falsification test possible.

Static scheduling, hence a decidable soundness gate. In a general concurrent system the order in which independent operations happen is decided at run time, by whichever inputs become available first; here, that order is instead fixed at compile time, derived from the dataflow and the source order. This gives up dynamic adaptivity but buys a soundness gate that is a single deterministic replay rather than a state-space search, and it is the only regime that suits hard real-time embedded targets in any case.

Explicit decomposition, hence predictability. The programmer annotates the decomposition; the compiler infers the transfers it implies but does not discover parallelism. This narrows what can be expressed but keeps the compilation outcome predictable and the analysis tractable.

Deliberate non-goals. The model does not target general-purpose GPU or accelerator programming; it does not perform polyhedral analysis; it does not support data-dependent control flow (such as convergence-driven loop termination), data-dependent loop trip counts, or recursion; and its schedule assertions are checked, not optimised against (there is no cost model). Some data-dependent *indexing* is supported — both gather reads and scatter writes, handled conservatively — but full sparse solvers, which additionally need data-dependent termination, remain out of reach. Chapter 11 returns to each of these as an honest limitation rather than an oversight.

Together these commitments define a model that is narrower than a general parallel programming system but whose central claim can be put to a concrete, mechanical test. Realising the commitments demands, first, a surface language in which the algorithm genuinely contains none of the schedule's decisions — the subject of the next chapter — and then a compiler that combines the two (chapter 6) and the backends that target the result (chapter 7).

Chapter 5

The Nuc Language

This chapter presents Nuc, the surface language in which an algorithm and a schedule are written. It is two small sublanguages that share one name table: an algorithm sublanguage that says what to compute, and a schedule sublanguage that says how to lay it out across workers and with what IO semantics. The chapter introduces each by example, works one algorithm through several schedules to show the boundary is real, and states precisely what the language deliberately cannot express. All examples are drawn from the implemented system.

5.1 Two sublanguages, one name table

A program is exactly two files: an algorithm file and a schedule file. The algorithm file (`.algo.nuc`) declares data, kernels, and dataflow. The schedule file (`.sched.nuc`) names the algorithm's kernels and loops and binds them to workers, loop transformations, and transfer semantics. There is no third file, no module system, and no imports; kernel *bodies* live in an adjacent ordinary Rust file compiled by the host toolchain.

The two sublanguages share one namespace. A schedule refers to a kernel or a loop variable by the same name the algorithm gave it; a schedule that names a kernel the algorithm does not define, or that fails to place a kernel the algorithm does define, is a compile error. This shared name table is the entire coupling between the files: the algorithm is meaningless to change for scheduling reasons, and the schedule cannot express anything about what is computed. An algorithm without a schedule does not compile (there is no default schedule, because an implicit default would hide exactly the decisions the language exists to make explicit); a schedule without an algorithm is meaningless.

5.2 The algorithm sublanguage

The algorithm sublanguage has just enough surface to express dataflow over shaped arrays. It has no notion of workers, blocking, buffers, or transfers. Listing 5.1 shows a complete algorithm: a 3×3 box-blur stencil.

Listing 5.1: A complete algorithm file: a 3×3 stencil. It declares shape, kernels, and dataflow — and nothing about decomposition.

```
const H : usize = 16;
const W : usize = 16;

data img_in : i32[H][W];
data img_out : i32[H][W];

kernel blur3 : (i32,i32,i32,i32,i32,i32,i32,i32,i32) -> i32 pure;
kernel load_image : () -> i32[H][W] effectful;
kernel save_image : (i32[H][W]) -> () effectful;

img_in <-- load_image();

for y : 1 .. H-1 {
for x : 1 .. W-1 {
    img_out[y][x] <-- blur3(
        img_in[y-1][x-1], img_in[y-1][x], img_in[y-1][x+1],
        img_in[y ][x-1], img_in[y ][x], img_in[y ][x+1],
        img_in[y+1][x-1], img_in[y+1][x], img_in[y+1][x+1]);
}}

save_image(img_out);
```

Four properties of this sublanguage are load-bearing.

Shaped, single-assignment data. Arrays carry an explicit shape and element type (`i32[H][W]`); scalars are degenerate arrays. There are no pointers and no aliases: a slice such as `img_in[y-1..y+2]` is a read-only view, never a writable alias. Each array location is written exactly once — the loop in listing 5.1 writes each interior `img_out[y][x]` once and no more. Single assignment is what makes the producer of every value unambiguous, which in turn is what lets the compiler infer inter-worker transfers soundly (section 6.5).

Kernels are real, shape-typed Rust functions. A `kernel` declaration is a *contract* — a shape-typed signature and a purity annotation — whose body is an ordinary Rust function in a sibling file. The compiler generates the code that calls the kernel; it never substitutes text into the body. The blur kernel above is a pure function of nine integers returning their sum divided by nine; the IO kernels read and write the image buffer. Aggregate parameters lower

to a row-major `Vec<T>` on the Rust side, and a contract-checking pass verifies that each Rust function matches the shape and purity its declaration promises. Treating kernel bodies as opaque Rust buys the host language's type checker, borrow checker, unit testing, and IDE tooling for free, and it eliminates the text-substitution hazards of a templating approach.

Mandatory purity annotation. Every kernel is annotated `pure` or `effectful`. A pure kernel has no side effects and may be reordered, deduplicated, or eliminated; an effectful kernel has its ordering preserved within its block and is never duplicated. There is no third, unannotated case — the annotation is required — which closes a class of silent reordering bugs.

Combine operators for distributed reduction. A kernel that produces a partial result destined to be folded into a shared accumulator may carry a *combine* annotation on its declaration — written `combine=sum` (and likewise `or`, `xor`, `min`, `max`, `and`) after the purity keyword. The annotation names the associative, commutative operator by which worker-local partials are recombined into a global result, together with its algebraic identity element, so that when a schedule distributes the reduction the compiler knows how to initialise each accumulator and merge the partials order-independently. The annotation lives in the algorithm because it is a property of the arithmetic — whether a result *can* be combined associatively is not a scheduling choice — while the decision to actually distribute the reduction remains in the schedule. This is the mechanism the cross-backend determinism result of section 10.2 rests on, and the reason a floating-point `sum` combine is rejected: IEEE-754 addition is not associative.

Affine, const-bounded iteration. Loops have plain bounds and an iteration variable, and the bounds fold to compile-time constants. The iteration variables (`y`, `x` above) are the handles a schedule will later refer to. Iteration variables and data share one namespace, disambiguated by scope. The kernel's nine-neighbour access pattern on `img_in` — offsets of $-1, 0, +1$ in each dimension — is exactly what the compiler reads to infer halo regions when a schedule distributes the work; the access pattern is in the algorithm, but the decision to distribute is not.

What is deliberately absent is as important as what is present: no worker names, no `block=` or `pipeline=`, no `transfer=` or `buffer=`, no conditionals across worker boundaries, no recursion, no closures, no generics. Every one of those belongs in the schedule, or nowhere.

5.3 The schedule sublanguage

A schedule binds an algorithm's kernels and loops to a worker topology and chooses IO semantics. Schedules are short — a typical one fits on a screen. Listing 5.2 shows a distributed schedule for the stencil of listing 5.1.

Listing 5.2: A distributed schedule for the stencil. Decomposition, loop transforms, and IO semantics are all here; none of it appears in the algorithm.

```
schedule for "../prog.algo.nuc" {
    workers = { host, w0, w1, w2, w3 };

    place load_image on host;
    place save_image on host;
    place blur3 on { w0, w1, w2, w3 };

    loop y : partition=rows;
    loop x : block=64, reuse;

    transfer img_in : async, buffer=2, notify=event;
    transfer img_out : sync;
}
```

The schedule has four kinds of directive.

Workers (space). The `workers` set names the concurrent units; their cardinality is fixed at compile time. The simple form shown here is a flat pool, suitable for commodity CPU and clusters. A richer *typed* form, used for heterogeneous targets, additionally declares worker *classes* — groups of workers with distinct capabilities such as SIMD width and available memories — together with named memory regions and directives that bind data to them. On a homogeneous backend the typed form collapses to the simple form, so the same surface extends from a thread pool to a heterogeneous embedded system-on-chip without a second sublanguage; section 7.4 exercises it on a multi-microcontroller target.

Placement. Each kernel is placed on exactly one worker or, with `on {…}`, distributed across several. Distributing `blur3` across the four compute workers tells the compiler to partition that loop's iteration space across them. Every kernel must have exactly one placement; an unplaced kernel is a compile error.

Loop transformations (time). A `loop` directive attaches transformations to a loop variable by name. `block=N` tiles the iteration into chunks of N; a `partition` directive chooses how a distributed loop is cut across workers — into one contiguous band per worker (`workers`), contiguous row bands (`rows`), or a two-dimensional grid (`blocks2d`); `reuse` recycles loop-carried

slices; `pipeline=D` software-pipelines with depth D. Combinations that make no sense — a two-dimensional partition on a one-dimensional loop, a pipeline depth that exceeds the tile count — are rejected at compile time. Notably there is no `vectorize` directive: SIMD is delegated to the host Rust compiler, because prescribing a vector width in the schedule would force every backend to honour it and so conflict with the byte-identical-across-backends guarantee.

Transfer / IO semantics. A `transfer` directive sets the IO semantics of a data symbol that crosses workers: `sync` or `async`, `buffer=N` for in-flight depth, and `notify=event` or `notify=poll` for the wakeup discipline. These are exactly the dimensions of section 2.2, raised to first-class directives. A transfer that crosses workers *must* carry a directive — omitting it is a compile error naming the data symbol — so that one schedule cannot silently mean different things on different backends. The compiler resolves each directive against the chosen backend's declaration of which transfer mechanisms it supports (its capability matrix, whose structure section 7.1 describes); asking for `async` on a backend that does not support it is a hard error, not a silent downgrade.

A schedule may additionally attach `check` assertions — for example `check loop frame : latency_max = 10ms` — which inject measurement code that verifies a property at run time. These are checkable, not prescriptive: the compiler reports whether the hand-written schedule meets the bound, but has no cost model and does not adjust the schedule to meet it.

5.4 A worked example: one algorithm, many schedules

The single algorithm of listing 5.1 composes with a family of schedules that decompose it in increasingly aggressive ways, with no change to the algorithm file. The smoke-test schedule places all three kernels on one `host` worker and applies no transforms:

Listing 5.3: The naive single-worker schedule for the same algorithm.

```
schedule for "../prog.algo.nuc" {
    workers = { host };
    place load_image on host;
    place save_image on host;
    place blur3 on host;
}
```

A second schedule keeps the single worker but tiles the outer loop with `loop y : block=4`, exercising the strip-mining transform (including a trailing partial tile, since the loop length is not a multiple of four). A third is the distributed schedule of listing 5.2, which partitions the outer loop into row bands across four workers; here the compiler must infer that each worker's `blur3` needs a

one-row halo from its neighbours' bands, and synthesise the transfers that carry those halo strips — none of which is visible in the algorithm. A fourth partitions the same loop into a two-dimensional grid, which additionally requires genuine worker-to-worker halo exchange on the east–west seams.

The crucial property is that all of these schedules drive the *same* algorithm file and, on the same input, must produce the *same* output. The naive schedule is the reference behaviour; every other schedule is required to match it byte-for-byte. This is the algorithm/schedule boundary made concrete: the decomposition changed completely — from one worker to four, from no transfers to halo exchange — while the arithmetic did not change at all. The mechanism that turns this from an aspiration into a test is the subject of chapter 9.

5.5 What the language cannot express, and why

The language's restrictions are deliberate, and each one buys a property the model depends on. They are stated here rather than discovered later.

Data-dependent control flow is bounded, and realized only on the single-worker path. There is no conditional in the algorithm sublanguage, and *unbounded* data-dependent loop termination is excluded: a loop that runs "until converged" with no iteration limit cannot be written, because its trip count is not a compile-time constant and the firing order would no longer be statically determined. What the grammar *does* admit is one bounded early-exit surface — an optional `until` clause that pairs a runtime halt predicate with a mandatory compile-time iteration cap, so the worst-case trip count stays statically known (appendix B). This bounded form is realized on the single-worker path: two examples (`21-jacobi-converge` and `29-jacobi-cap-hit`) compile and run a convergence loop value-correctly — one exiting early, one running the full cap so the worst-case bound is itself exercised — and both do so byte-identically across all seven tier-1 backends, since a host-only schedule delegates to the shared single-worker emitter on every backend. The construct is thus seven-backend, not single-backend. Static analysability is preserved because the soundness gate replays the full capped unroll and any early exit is a sub-trace of it; byte-identity is kept within reach because the halt predicate is an exact-integer, backend-agnostic function of the data, so every backend emitting the loop stops at the same generation. On the *multi*-worker (distributed) path the break is still unsupported — it requires a collective all-reduce of the convergence scalar so all workers exit together, which is future work (section 12.3) — so the multi-worker break-condition path is fail-loud-rejected rather than mis-compiled. A floating-point halt predicate is at risk under the float-determinism limit below; unbounded data-dependent control flow, and the convergence-driven solvers that need it, remain out of scope.

Limited data-dependent indexing. A data-dependent *read* (a gather, where an array is indexed by another array's contents) and a data-dependent *write* (a scatter, such as a histogram) *are* expressible, and the compiler handles the data-dependent dimension conservatively — broadcasting a gathered array in whole, or replicating and recombining a scattered target. What cannot be expressed is a scatter whose target is itself partitioned along an affine axis, which is rejected rather than mis-compiled. Combined with the absence of convergence-driven termination, this means full sparse iterative solvers fall outside the language.

No recursion, no higher-order kernels. Kernels are first-order and non-recursive. Recursive or higher-order structure has no static firing order and so cannot be scheduled in this model.

No carried scans in the surface. An in-array prefix scan whose recurrence carries across iterations cannot be written directly, because it would require either a conditional to guard the boundary or a second assignment to the same symbol. Such patterns are instead expressed as a kernel-level idiom — the carry logic lives inside an ordinary Rust kernel over a rectangular reduction-accumulator — which keeps the algorithm sublanguage free of conditionals and single-valued per symbol, exactly the properties the intermediate representation and the differential test rely on.

These limits share one root cause — the model trades expressiveness for a statically determined, deterministic firing order — and they are not merely documented but enforced: each prohibited construct is rejected by a specific compiler pass with a typed diagnostic rather than mis-compiled, as the next chapter shows when it constructs and checks that firing order. Chapter 11 weighs the trade honestly.

Chapter 6

Architecture and Methodology

This chapter walks the compiler from its two source files to per-worker code, defends the two boundaries the architecture rests on, and explains the soundness gate and why it is sound for the restricted class of programs the language admits. The compiler is implemented in Rust [14]. The description here is of the shipping system, not an idealised sketch.

6.1 The compilation pipeline

Compilation is a single, deterministic chain of passes (fig. 6.1). The algorithm and schedule files are parsed independently. The algorithm is desugared and type-checked into a typed algorithm intermediate representation (IR) that contains kernels and dataflow but no decomposition. The schedule is then *linked* against that IR, binding kernels to workers and attaching loop transformations and IO semantics. From the linked form the compiler builds an application control-flow graph (ACFG) — a tree-structured intermediate form whose nodes are operations, sequences, and loops, defined in section 6.4 — applies the schedule's transformations to it, infers the halo regions and reuse windows implied by the access patterns, and injects the synchronisations and data transfers the decomposition requires. The fully elaborated ACFG is lowered to a Petri net, which the soundness gate checks; the same ACFG is then projected to a totally-ordered event list per worker; and finally a chosen backend renders each worker's event list into code. Each named stage is defined in its own section below; this paragraph is only the map.

The pass order is fixed and total: build the ACFG; apply blocking; apply the chosen partition (per-worker ranges, row bands, or a two-dimensional grid); infer halos; infer reuse; inject synchronisations; inject transfers. Each pass either succeeds or returns a typed error that becomes a compile diagnostic; none of them consults a backend, because everything above the projection step is backend-independent by construction.

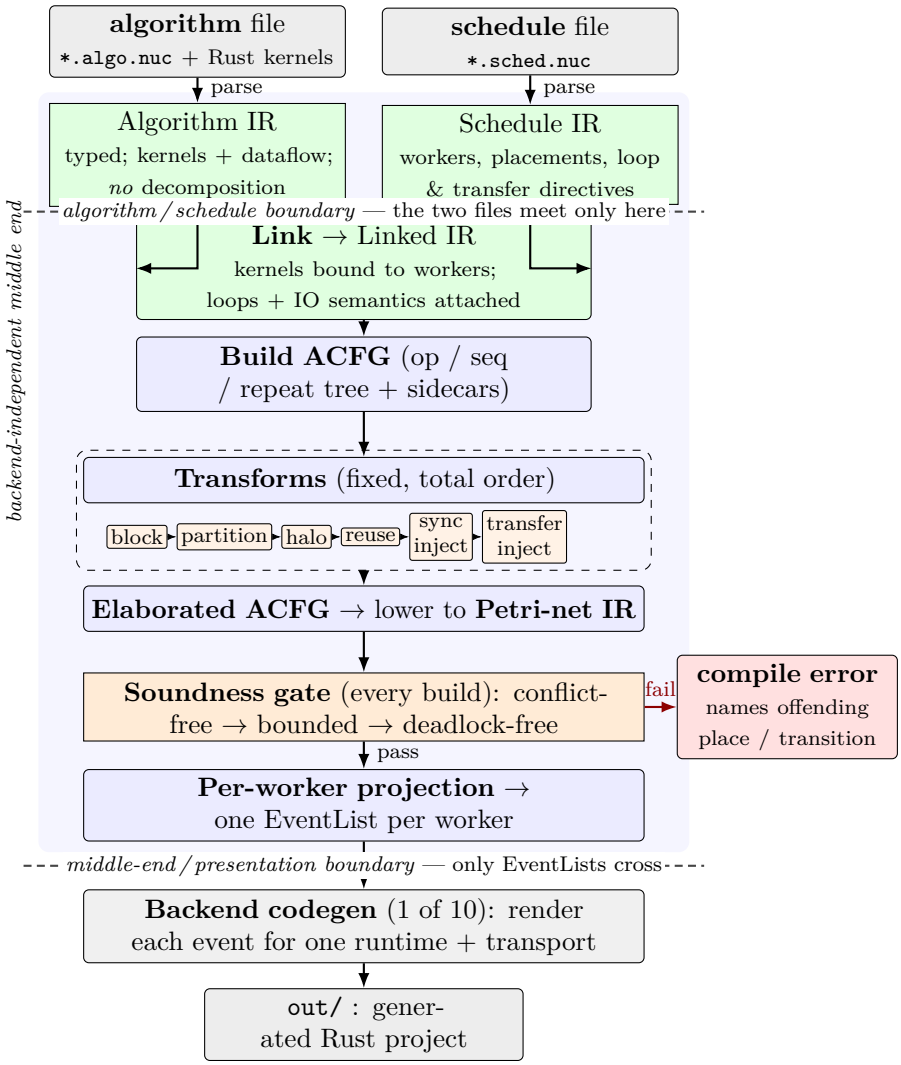


Figure 6.1: The compilation pipeline: a single deterministic chain of passes. The algorithm and schedule are parsed independently into typed intermediate representations (IRs) that meet only at *link* (the algorithm/schedule boundary); from the linked form the compiler builds an application control-flow graph (ACFG), applies the schedule's transforms in a fixed total order (block, partition, halo, reuse, sync-inject, transfer-inject), lowers the elaborated ACFG to a Petri net, and runs the soundness gate on *every* build — a failure is a compile error naming the offending place or transition, never a runtime hang. The net is then projected to one totally ordered EventList per worker; only those lists cross the middle-end/presentation boundary into a chosen backend. Everything above that line (shaded) is backend-independent by construction.

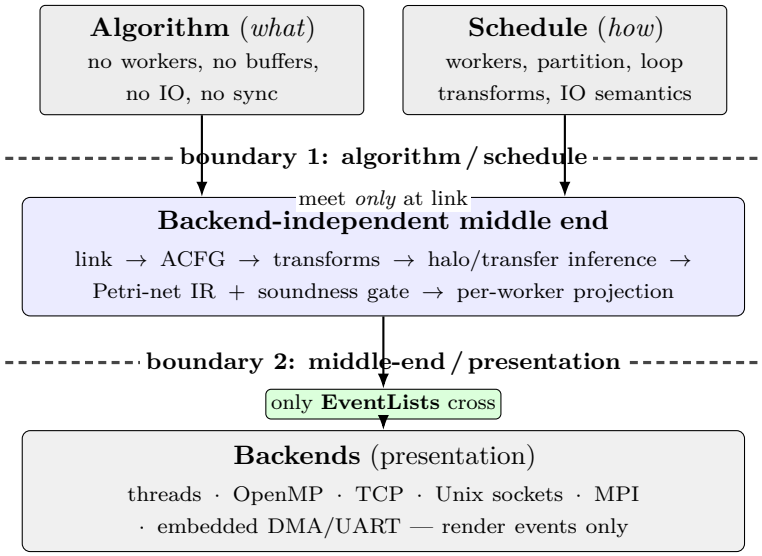


Figure 6.2: The two boundaries the architecture rests on. *Boundary 1* (algorithm/schedule): the algorithm IR carries no worker, buffer, or IO information; those enter only at the link step, so the two files meet exactly once. *Boundary 2* (middle-end/presentation): every pass from parsing through per-worker projection is backend-independent, and a backend consumes only the per-worker EventLists — never the algorithm or schedule. Each boundary is real precisely when either side can be rewritten independently; a leak across either one surfaces as a backend that disagrees with the others in the cross-backend differential test.

6.2 The two boundaries

Two boundaries must hold for the architecture to be honest, and both are visible in the pipeline (fig. 6.2).

The first is the *algorithm/schedule* boundary. The algorithm IR contains no worker bindings, no buffer sizes, and no IO semantics; all of those enter only at the link step, from the schedule. The test of this boundary is independence: if either file can be rewritten without the other, the boundary is real. Renaming a worker or changing a transfer's buffering touches only the schedule; rewriting a kernel body touches only the algorithm and its sibling Rust file.

The second is the *middle-end/presentation* boundary. Everything from parsing through per-worker projection is backend-independent; the backend enters only at the final codegen step, and it consumes only the per-worker event lists, never the algorithm or schedule directly. Anything backend-specific that appeared above this line would be a defect. The cross-backend differential test (chapter 9) is what keeps both boundaries honest: a leak across either one shows up as a backend that disagrees with the others.

6.3 Algorithm IR and linking

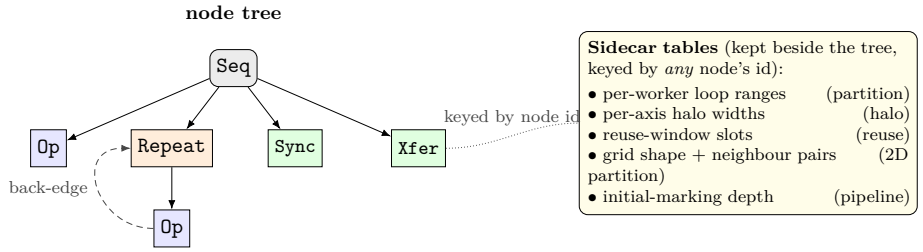
Parsing the algorithm yields a typed IR over shaped arrays, kernels with shape-typed signatures and purity, and a dataflow body of single-assignment writes and affine loops. A separate contract pass checks each kernel's Rust body against its declared signature and purity before scheduling begins, so a kernel whose implementation disagrees with its contract is rejected early rather than miscompiled.

Linking combines this IR with the parsed schedule. It resolves every name the schedule mentions against the algorithm's name table — a schedule that places a non-existent kernel, or that leaves a kernel unplaced, is rejected here — and attaches to each kernel its worker placement and to each loop its transformations and the IO semantics of the data that crosses workers. The output is a *linked* form: the algorithm's dataflow, now annotated with where each operation runs and how its data moves, but not yet elaborated into concrete transfers.

6.4 The ACFG and its transforms

The linked form is built into an ACFG (fig. 6.3): a tree whose nodes are operations (a kernel firing with its dataflow edges), sequences, and repeats (loops with a back-edge), later joined by synchronisation and transfer nodes. The ACFG carries a set of *sidecar* tables — auxiliary maps kept beside the node tree rather than inside it, keyed by node identity — which downstream passes populate incrementally: per-worker loop ranges from partitioning, per-axis halo widths from halo inference, reuse-window slots from reuse inference, a grid shape and neighbour pairs for two-dimensional partitions, and the initial-marking depth for each pipelined transfer. Keeping these as sidecars rather than rewriting the node structure lets each pass add precisely the information it derives without disturbing what earlier passes established.

The schedule's loop transformations are applied to the ACFG in order. Blocking strip-mines a loop into a tile loop wrapping an intra-tile loop, splitting a non-divisible range into full tiles plus a trailing partial tile so the decomposition is faithful to the original iteration count. Partitioning rewrites a distributed loop's per-worker ranges: a row-band partition cuts the outer loop into contiguous bands (with a documented spill-over policy when the rows do not divide evenly), and a two-dimensional partition cuts a grid. Reuse and pipelining record their information in the sidecars for the transfer and projection stages to consume. Vectorisation is deliberately not a transform here — it is left to the host compiler — for the reason given in section 5.3.



node kinds: `Op` (kernel firing + read/write dataflow edges), `Seq`, `Repeat` (loop with back-edge); the green `Sync` (barrier) and `Xfer` (push/wait) are *injected* peer nodes, added by the sync/transfer passes.

Figure 6.3: The application control-flow graph (ACFG): a tree whose nodes are operations (a kernel firing with its read/write dataflow edges), sequences, and repeats (loops with a back-edge), later joined by the injected synchronisation and transfer nodes (green, added as peer nodes of the enclosing sequence by the sync and transfer passes). Rather than rewriting the tree, each middle-end pass populates *sidecar* tables — auxiliary maps kept beside the tree and keyed by node identity — so a pass adds precisely the information it derives (per-worker ranges, halo widths, reuse slots, grid/neighbour data, pipeline initial-marking depth) without disturbing what earlier passes established. The concrete instance for the stencil is shown in fig. 8.1.

6.5 Transfer and halo inference

With the decomposition fixed, the compiler synthesises the communication it implies. This is the step that distinguishes the system from a hand-written parallel program: the programmer never writes a send or a receive.

Transfer injection walks the ACFG tracking, for each data symbol, the worker that last produced it. Wherever a consumer runs on a different worker than the producer, it emits a matched pair of transfer nodes — a push on the producer and a wait on the consumer — carrying a fresh, globally unique sequence tag. The sequence tag is the compile-time identity that lets the receiving side know exactly which send each receive pairs with, without any runtime matching machinery. For a distributed producer feeding a distributed consumer, the pass fans the pairs out across the worker cross-product.

Halo inference derives, from each kernel's declared access offsets, how far beyond its own partition a worker must read. The stencil's offsets of $-1, 0, +1$ in each dimension yield a halo width of one; transfer injection then extends each worker's transferred tile by that halo so the boundary values are present. This is transfer *synthesis* from an explicit decomposition, not parallelism discovery: the compiler computes the consequences of a decomposition the programmer chose, a far narrower and decidable problem.

The inference is careful at its edges. When a partitioned index is affine and involves only the partitioned axes, in order (a *contiguous prefix* of those

axes), the compiler infers a precise tiled region for each worker — the slice that worker will actually read; when it does not — a sparse, multi-variable, or data-dependent index — the pass conservatively degrades to the whole array, which is always a correct superset of what the worker might read. This inferred region governs both how the receiving worker gathers its portion and, where the region is a contiguous span, what the wire carries: such edges transmit only the span (the sender slices, the receiver pastes from a zero-based payload), while non-contiguous, data-dependent, and accumulator edges still transmit the whole array as the sound envelope. The receiver's local buffer remains full-shaped in either case — the narrowing cuts communication volume, not per-worker footprint, an asymmetry weighed in section 11.3. Data-dependent indices (gather and scatter) take the whole-array path on the data-dependent dimension; a scatter whose target would have to be split along an affine partition axis is rejected as a typed error rather than silently mis-handled.

6.6 The Petri-net IR and the event contract

The elaborated ACFG is lowered to a Petri net (fig. 6.5). The net is a small place/transition net — the implementation is well under a thousand lines, not a general-purpose Petri-net toolkit — in which a transition is a kernel firing, a transfer, or a synchronisation, and a place is a data slot, a communication channel, or a barrier. Each place carries a capacity (the declared buffer size) and an initial marking (the head-start a pipelined or double-buffered transfer provides). The net exists for analysis and inspection; it is not what the backend consumes. The same data edge takes a different net shape under different IO directives: fig. 6.4 places a synchronous single-buffer transfer beside an asynchronous buffered one, showing how the schedule's choice sets the channel capacity and initial marking while the algorithm stays fixed.

What the backend consumes is the *event contract*: a per-worker list of events, each the projection of one transition onto the worker that owns it. It is a contract in a precise sense — it is the sole interface between the backend-independent middle-end and the backends, and it cuts both ways: the middle-end guarantees the list is well-formed and has passed the soundness gate, and each backend undertakes to render every event faithfully. The contract has a small fixed vocabulary. A `Fire` runs a kernel over an iteration tile; a `Push` sends a data tile to a destination worker under a sequence tag, and a matching `Wait` receives it; a `Sync` is a control-only barrier over a set of participants, each carrying the same stable sync tag so that disjoint per-worker lists agree on which barrier is which without any worker inspecting another's list; and a `Loop` preserves an iteration's structure rather than unrolling it. A sync tag plays a different role from a sequence tag: a sequence tag uniquely identifies one push/wait pair on a data channel, whereas a sync tag is a single identifier shared by every participant of one barrier. The contract also reserves

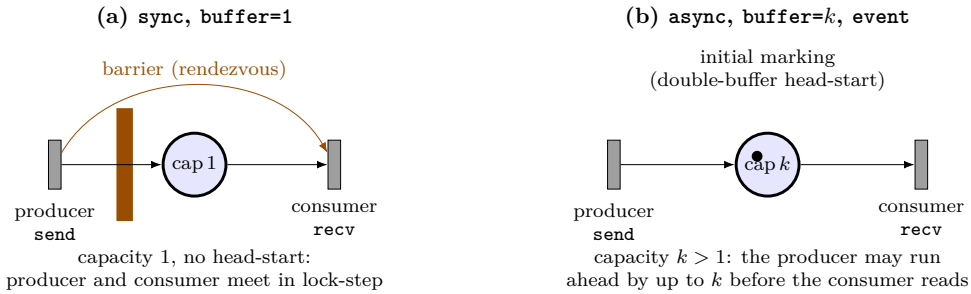


Figure 6.4: The same producer/consumer data edge under two schedule-chosen IO directives — the algorithm is identical in both; only the schedule differs. **(a)** A `synchronous`, single-buffer transfer lowers to a capacity-one channel with no initial marking, mediated by a barrier (orange): producer and consumer proceed in lock-step rendezvous. **(b)** An `asynchronous`, event-notified transfer with `buffer=`k lowers to a capacity-k channel carrying an initial marking, so the producer may run ahead of the consumer by up to k tiles. The buffer depth and initial marking are exactly the schedule directives the algorithm never sees; the boundedness check (fig. 6.6) holds the resulting marking to whichever capacity the directive set.

`Alloc` and `Free` events for explicit memory-region placement; in the current system the backends manage local storage themselves and these two are not yet emitted, an honest gap noted here and revisited in section 12.4.

Iteration tiles are described uniformly as rectangular slices in iteration space, and memory regions are opaque tags the backend interprets, so the same contract serves a shared-memory thread and a message-passing rank alike.

6.7 The soundness gate

Before any code is emitted, the net is checked, and the check runs on *every* build, not only under a diagnostic flag (fig. 6.1). A failure is a compile error, with a message naming the offending place or the stalled transition — never a runtime hang discovered later.

The gate has three parts, run in order. The first concerns *conflict-freedom*: that no place offers its token to two consumer transitions that could both be enabled, so every token has at most one possible consumer and the firing order determines a single execution trace rather than a branching set of them. This property is the precondition that makes the next two checks sound — with one trace to consider, they need only replay it — and it holds *by construction* for the nets this compiler emits: the lowering threads a single-token control place through each worker, serialising that worker's transitions, so a free-choice conflict cannot arise. The in-gate conflict-freedom check is therefore a

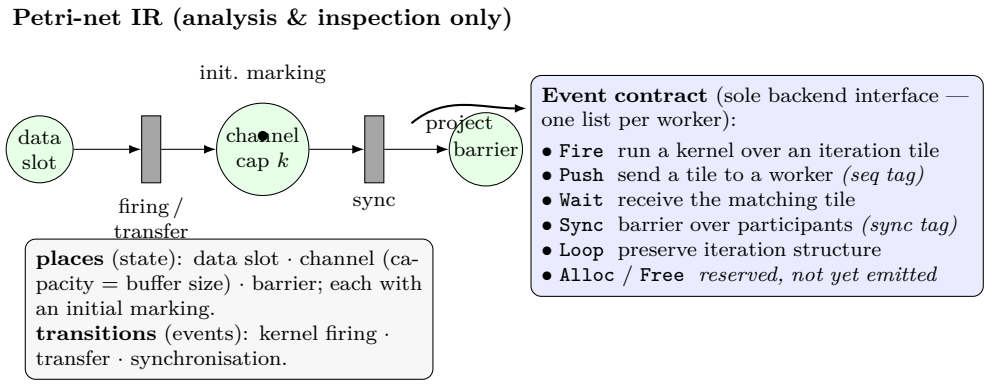


Figure 6.5: The Petri-net IR and the event contract the backend actually consumes. In the net, *places* hold state — a data slot, a communication channel (whose capacity is the declared buffer size), or a barrier — each with an initial marking that encodes a pipelined or double-buffered head-start; *transitions* are kernel firings, transfers, and synchronisations. The net exists only for analysis and the soundness gate. Each transition is projected onto the worker that owns it to form the *event contract*: a per-worker list drawn from a small fixed vocabulary (`Fire`, `Push` and `Wait` carrying a sequence tag, `Sync` carrying a shared sync tag, `Loop`), with `Alloc`/`Free` reserved but not yet emitted. A *sequence tag* is the compile-time identity pairing one `Push` with its matching `Wait`; a *sync tag* is a single identifier shared by all participants of one barrier. (Circles are places, bars are transitions, and a filled dot is a token, in the standard Petri-net convention.) This is the sole interface to the backends; the concrete net for the stencil is fig. 8.3.

regression tripwire rather than the source of the guarantee: it is a complete search on small contested nets and a sound (never-false-rejecting) single-order under-approximation on the larger multi-worker nets, where an exhaustive search would be costly and the by-construction argument has already ruled conflict out. A *boundedness* check replays the firing order and rejects any firing that would push a place past its capacity, reporting the place and the overflowing marking. A *deadlock-freedom* check replays the same order and rejects the first stall — a required transition whose input place lacks tokens — reporting the stalled transition and the deficient place. The error labelling is careful: a stall observed while checking boundedness is reported as a deadlock, not a capacity error, so the diagnosis matches the underlying fault.

The crucial point is *why* this is sound despite being far cheaper than a general Petri-net analysis. The firing order is statically determined: the compiler derives one deterministic order — source order, except where a dataflow dependence forces a producer to fire before its consumer. These nets are structurally bounded by construction — every place has a finite

declared capacity, so the reachable state space is finite — and, as just argued, free of non-deterministic token routing; for that class replaying the single derived order is equivalent to exploring every reachable marking. The gate is therefore an exact replay over one order, not a reachability or coverability search. This is a real limitation, and it is stated as such: the gate is sound for the statically-ordered nets this language produces, and is emphatically not a general model checker for arbitrary Petri nets. Within its class it is both sound and inexpensive. Figure 6.6 contrasts a sound net with the three unsound shapes the gate is built to reject — a boundedness violation, a deadlock, and a free-choice conflict.

6.8 Per-worker projection

Projection turns the elaborated ACFG into one totally-ordered event list per worker (fig. 6.7). The walk is depth-first in source order: a sequence visits its children left to right; a repeat wraps each worker's projected body in a single loop event, preserving the iteration structure rather than unrolling it; an operation emits a fire on each worker it is placed on; a synchronisation emits a barrier event on each participant; and a transfer emits a push on its source worker and a wait on its destination worker. Because the walk is deterministic and worker lists are kept in a sorted map, two compilations of the same program yield byte-identical event lists — a prerequisite for the reproducibility the validation strategy depends on.

The event lists are the sole interface to the backend. Everything the backend needs to know about decomposition, transfers, and synchronisation is in them; the backend never sees the algorithm, the schedule, or the net.

6.9 Presentation-layer codegen

The final step renders each worker's event list into source code for a chosen backend. The same event has different renderings in different backends: a push is a memory write under a shared-memory backend, a socket write under a message-passing backend, a non-blocking send under MPI, and a DMA descriptor enqueue on an embedded target; a barrier is a thread barrier, an `MPI_Barrier`, or a coordinated interrupt rendezvous. The kernel bodies are emitted verbatim from the algorithm's sibling Rust file, and array indices are flattened to row-major offsets using the declared shapes.

A large fraction of the rendering logic is shared across backends rather than duplicated: the event-list walker, the expression and index renderers, the check-loop templates, and the host-election rule (the policy that designates one worker as the coordinating host for operations needing a single orchestrator, such as program IO) all live in a common library that every backend consumes, with each backend providing only the thin, transport-specific pieces.

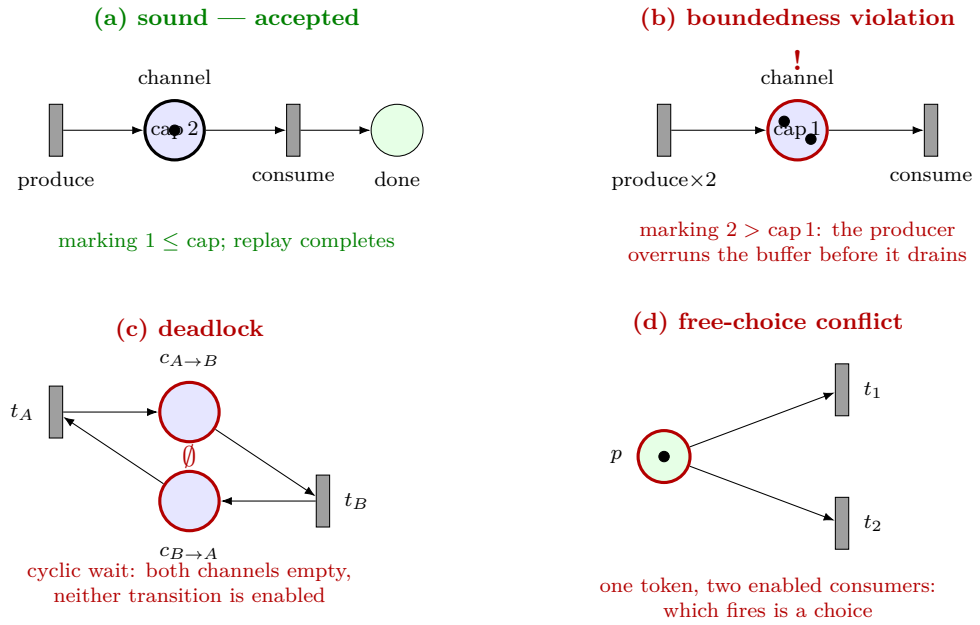


Figure 6.6: One sound net and the three unsound shapes the soundness gate is built to reject. Circles are places, bars are transitions, filled dots are tokens. **(a)** A sound net: the channel's marking never exceeds its declared capacity, every required transition eventually fires, and the single replayed firing order runs to completion. **(b)** A *boundedness* violation: the producer fires twice into a capacity-one channel before the consumer drains it, so the place would hold two tokens — caught by the replay as a capacity overflow. **(c)** A *deadlock*: two workers each wait on a channel the other must fill first; with both channels empty no transition is enabled and the replay stalls. **(d)** A *free-choice conflict*: one place offers its single token to two enabled consumers, so which one consumes it is a nondeterministic choice. In the nets this compiler emits, case (d) cannot arise — the per-worker control places serialise each worker's transitions by construction (section 6.7) — so the in-gate conflict check is a regression tripwire rather than the source of the guarantee; cases (b) and (c) are what the boundedness and deadlock replays catch on a genuinely unsound schedule.

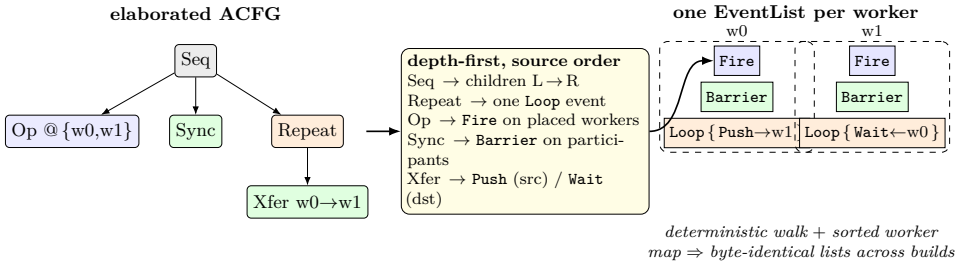


Figure 6.7: Per-worker projection turns the elaborated ACFG into one totally ordered event list per worker. The walk is depth-first in source order: a sequence visits its children left to right; a repeat wraps each worker's projected body in a single `Loop` event (preserving the iteration structure rather than unrolling it); an operation emits a `Fire` on each worker it is placed on; a synchronisation emits a barrier on each participant; and a transfer emits a `Push` on its source worker and a `Wait` on its destination. Because the walk is deterministic and worker lists are kept in a sorted map, two compilations of the same program yield byte-identical event lists — the prerequisite for the reproducibility the validation strategy depends on. (`@{w0,w1}` is the schedule's placement annotation; `Loop{…}` denotes the events enclosed by a worker's loop.)

With codegen described, both boundaries this chapter set out to defend are now visible in the artefact. The algorithm/schedule boundary is the link step: nothing target-specific exists above it, because the algorithm IR is built before any schedule is consulted. The middle-end/presentation boundary is the projection step: every pass from parsing to projection is backend-free, and the backend enters only here, consuming event lists and nothing else. A leak across either boundary would surface as a backend that disagrees with the others — which is exactly what the validation strategy of chapter 9 is built to detect. Chapter 7 first describes the backends this codegen targets and the capability matrix that governs which schedules each can accept.

Chapter 7

Backends and the Target Ladder

A backend turns the per-worker event lists of section 6.8 into code for one combination of runtime and transport. Because the middle-end/presentation boundary is real, a backend sees only an event list, never the algorithm or schedule — and this chapter argues that the restriction is enabling rather than limiting: the same event list drives ten backends across three qualitatively different tiers, and the low cost of adding one (section 7.5) is itself evidence that the boundary is doing its job. The chapter presents the ten backends, organised as a three-tier ladder from cheap commodity CPU through message-passing clusters to embedded microcontrollers (fig. 7.1), and defends the capability matrix that makes backend choice orthogonal to the schedule.

The three tiers differ not only in target but in how thoroughly each is tested. Tier-1 backends are both compiled and run on every continuous-integration invocation — they are the cheap rig. Tier-2 backends must compile for every required schedule and are run and compared wherever a message-passing runtime is available. Tier-3 backends must likewise compile for every required schedule and are run under simulation where a hardware shim exists. This *compile-mandatory, run-best-effort* discipline is referred to by name in the tier sections below.

7.1 The capability matrix and backend orthogonality

Each backend declares its capabilities as a small committed text file — a capability matrix — listing its transport, its supported notification mechanisms, whether it supports asynchronous and buffered transfers, its maximum buffer depth, and its worker classes and memory regions (fig. 7.2). Table 7.1 reproduces the matrix for all ten backends.

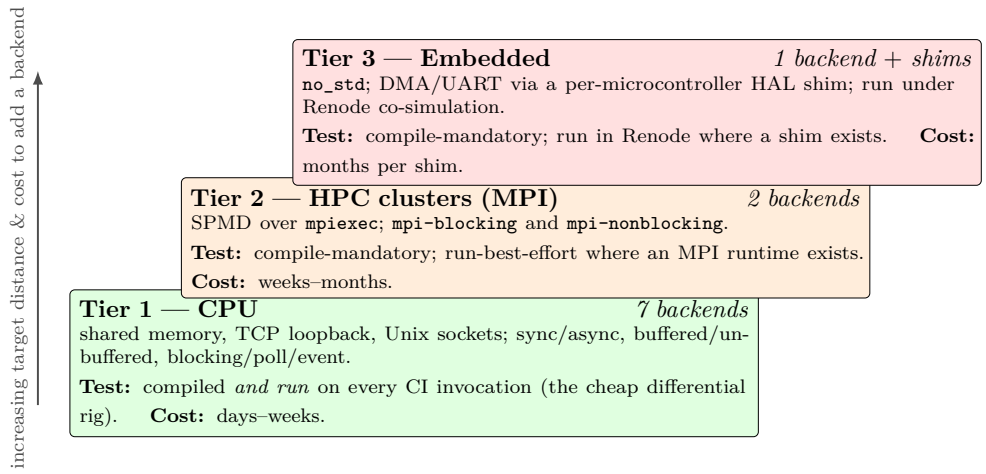


Figure 7.1: The ten backends organised as a three-tier ladder, ascending in target distance and in the cost of adding a backend. **Tier 1** (seven CPU backends) is compiled *and run* on every continuous-integration invocation — the cheap differential rig — and a new one costs days to weeks (copy a sibling, swap the wire and notification calls, reuse the shared single-worker emitter). **Tier 2** (two MPI backends) and **tier 3** (one embedded backend plus per-microcontroller shims) follow a *compile-mandatory, run-best-effort* discipline: they must compile for every required schedule and are run where a runtime (an MPI `mpiexec`, a Renode shim) is available. Their costs rise to weeks–months and months-per-shim respectively. (Green, orange, and red shade tiers 1, 2, and 3; "target distance" is how far the execution environment is from a standard hosted operating-system runtime.)

Orthogonality is the property that backend choice is independent of the schedule, mediated only by this matrix. A schedule compiles against any backend whose capabilities are a superset of what the schedule demands. A schedule that asks for an asynchronous, event-notified, double-buffered transfer compiles on `pthreads-async` or `mp-tcp-event` but is *rejected at compile time* on `pthreads-sync`, whose matrix offers neither async nor buffering — rejected with a message naming the missing capability, never papered over with a silent downgrade that would make the same schedule mean different things on different backends. Two backends can also be made deliberately capability-identical to isolate a variable: `openmp-rs` and `pthreads-sync` expose the same matrix but run on different substrates (a work-stealing thread pool versus raw threads), so a disagreement between them would implicate the substrate, not the schedule.

Backend	Tier	Transport	Notify	Async	Buffer
`openmp-rs`	1	shared memory	barrier, blocking	no	1
`pthreads-sync`	1	shared memory	barrier, blocking	no	1
`pthreads-async`	1	shared memory	event	yes	64
`mp-tcp-bufsync`	1	TCP loopback	barrier, blocking	no	1
`mp-tcp-poll`	1	TCP loopback	poll, barrier, blocking	no	1
`mp-tcp-event`	1	TCP loopback	event	yes	64
`mp-uds-event`	1	Unix socket	event	yes	64
`mpi-blocking`	2	MPI	barrier, blocking	no	1
`mpi-nonblocking`	2	MPI	barrier, blocking, event	yes	64
`embedded-pattern`	3	DMA/UART	event, poll, barrier, blocking	yes	2

Table 7.1: The capability matrix of the ten backends, reproduced from each backend's committed declaration. "Notify" lists the notification mechanisms the backend supports; "Async" is whether it supports asynchronous transfers; "Buffer" is the maximum in-flight buffer depth. `mpi-nonblocking` is a strict superset of `mpi-blocking`: it accepts every synchronous schedule the blocking backend does, plus asynchronous ones. Its *event* notification is the MPI analogue of a reactor — non-blocking point-to-point (`MPI_Ibsend`/`MPI_Imrecv`) with completion tracked by `MPI_Wait` on request handles — not an operating-system event loop of the kind the tier-1 reactor backends use. `embedded-pattern`'s *event* notify is a third flavour again — an interrupt-driven DMA-completion wait on a depth-2 descriptor ring (section 7.4) — and its asynchrony and buffering apply only on the DMA transport path; a programmed-IO transfer ignores them and renders a blocking byte loop.

7.2 Tier 1 — CPU

The seven tier-1 backends run on a commodity CPU and emit hosted Rust. They are the cheap test harness and the falsification rig: because they are inexpensive to build and run, the cross-backend differential test (chapter 9) can exercise the full schedule-by-backend matrix on a laptop. They span the IO design space of section 2.2 along two axes — where a worker lives, and how data moves between workers.

Shared-memory backends map a worker to a thread. `pthreads-sync` uses operating-system threads with condition-variable rendezvous and synchronous, unbuffered transfers; `pthreads-async` uses threads with ring buffers for asynchronous, buffered, event-notified transfers; and `openmp-rs` maps workers onto a work-stealing thread pool, reusing the synchronous shared-memory lowering. *Message-passing backends* map a worker to an operating-system process. `mp-tcp-bufsync`, `mp-tcp-poll`, and `mp-tcp-event` communicate over TCP loopback sockets, differing in their notification discipline — blocking receive, non-blocking poll, and an event reactor respectively (an event reactor is an IO loop that waits on operating-system readiness notifications, via `epoll` or `kqueue`, rather than spinning or blocking per transfer) — and `mp-uds-event`

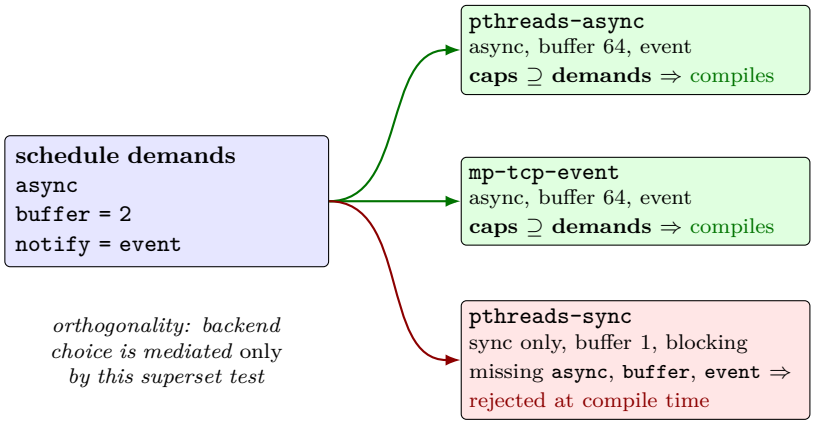


Figure 7.2: Backend orthogonality (backend choice does not change what the schedule computes, only whether it compiles) made mechanical. Each backend declares a capability vector (transport, notify, async, buffer depth); a schedule declares a demand vector. A schedule compiles against a backend exactly when the backend's capabilities are a *superset* of what the schedule demands. The stencil's distributed schedule (`async`, `buffer=2`, `notify=event`) compiles on `pthreads-async` and `mp-tcp-event`, but is *rejected at compile time* on `pthreads-sync`, whose matrix offers neither async nor buffering — with a diagnostic naming each missing capability, never a silent downgrade that would make one schedule mean different things on different backends. The full ten-backend matrix is table 7.1.

is the event reactor over Unix domain sockets instead of TCP. The synchronous TCP backends share one wire-protocol substrate and the two event-reactor backends share another, each differing only in the transport-specific calls, so that the same scheduling logic is exercised across distinct transports.

Together the seven cover the cross-product that matters: synchronous and asynchronous, buffered and unbuffered, event-driven and polled, over shared memory, TCP, and Unix sockets. That spread is what gives the differential test its power — a leak in the algorithm/schedule or middle-end/presentation boundary is unlikely to manifest identically across seven such different realisations.

7.3 Tier 2 — HPC clusters

The two tier-2 backends target distributed-memory clusters through MPI, the dominant message-passing standard. Both emit single-program multiple-data (SPMD) code: one Rust binary that dispatches on its MPI rank — the integer index identifying a process within the MPI communicator — so rank r executes worker r's event list (fig. 7.3). SPMD is the natural fit for MPI,

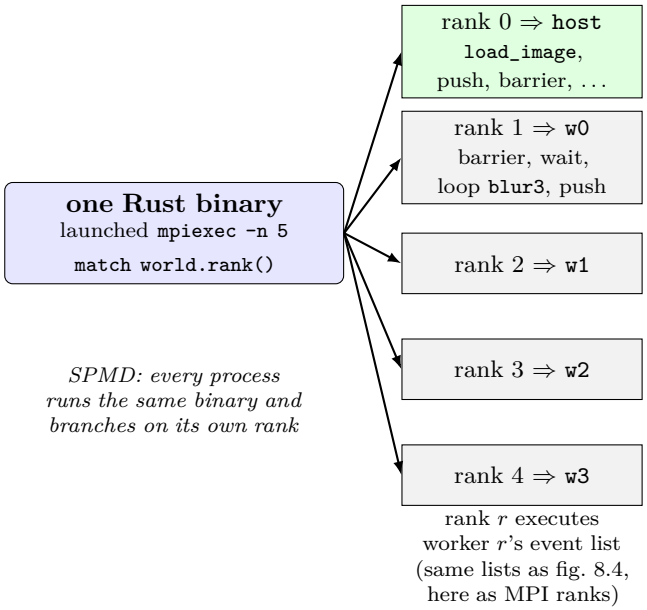


Figure 7.3: The single-program multiple-data (SPMD) dispatch the tier-2 MPI backends emit. Rather than one binary per worker, the backend emits *one* Rust binary, launched as five processes with `mpiexec -n 5`, that branches on its MPI rank: a `match world.rank()` with five arms sends rank 0 to the host's event list and ranks 1 through 4 to the four compute workers' lists. Rank 0 (shaded) is the host; ranks 2–4 run the same structure as rank 1 over their own bands. SPMD is the natural fit for MPI, which launches many copies of one binary and lets each discover its identity by querying its rank.

which launches many copies of one binary and lets each discover its identity by querying its rank; emitting a separate binary per worker, as the embedded tier does, would require external orchestration that MPI does not provide. The generated project depends on the established Rust MPI bindings, and runs under `mpiexec`.

`mpi-blocking` renders a push as a blocking `MPI_Send` and a wait as a blocking `MPI_Recv`, with barriers as `MPI_Barrier`; `mpi-nonblocking` renders a push as a buffered non-blocking send and a wait as a non-blocking receive completed by `MPI_Wait`, which is what its asynchronous, buffered capability advertises. The two share a common MPI planning substrate and differ only in the rendezvous prelude and the send/receive calls they emit; the rank assignment, barrier handling, and event-walk are identical. Following the tier's compile-mandatory, run-best-effort discipline, tier-2 backends must compile for every required schedule, and are run and compared where an MPI runtime is available (a local `mpiexec` suffices). Fourteen MPI cells spanning twelve examples are run and byte-diffed against the committed reference this way: ten

under `mpi-blocking` — spanning a row-partitioned matrix multiply, a sparse matrix–vector product, a data-dependent scatter, a transpose, a separable filter, and an iterative stencil, alongside the original element-wise, split, and reduction shapes — and four asynchronous schedules under `mpi-nonblocking`. This is a loopback launcher on a single oversubscribed host: it stands in for, but is not, a production cluster, which remains future work. Collective recognition — emitting `MPI_Allreduce` or `MPI_Scatter` when the scheduler recognises the pattern — is likewise deliberately left to future work, so the backends emit correct point-to-point communication that is merely not optimal.

7.4 Tier 3 — Embedded

The tier-3 backend, `embedded-pattern`, targets `no_std` microcontrollers (`no_std` is Rust's term for a target compiled without the standard library, and therefore without an operating system, heap allocator, or file system). It emits target-agnostic event-list lowering and delegates all hardware specifics — DMA descriptors, interrupt-vector binding, memory-region addresses — to a per-microcontroller *shim* that implements a small hardware-abstraction trait. The generic backend is written once; each microcontroller family is a separate shim crate, of which the system ships two: an STM32H7 (Cortex-M7) reference shim using a USART transmit-data register plus a DMA controller, and a second nRF52840 (Cortex-M4F) shim using a UARTE EasyDMA engine whose deliberately different transmit-register interface is what shows the trait is a real portability seam rather than a single-target abstraction dressed up as one.

The backend emits in two modes. A library mode produces a self-contained `no_std` crate per worker that compiles against a do-nothing stub shim — enough to prove the lowering type-checks on the embedded target without hardware. A binary mode, given a concrete shim, produces bare-metal binaries that run under the Renode instruction-set simulator [1] — an open-source, scriptable tool that emulates a configurable set of microcontrollers and their peripherals wired together. The interesting case is multiple microcontrollers co-simulated together (fig. 7.4): the backend emits one binary per worker plus a generated Renode machine description that wires the binaries together over a UART hub, with one bounded FIFO per transfer channel.

The embedded tier is no longer synchronous-only. On the DMA transport path a schedule may now ask for an asynchronous, double-buffered, event-notified transfer, and the backend lowers it to a producer-side depth-2 descriptor ring: the producer arms one transfer and runs ahead, deferring completion until the ring is full, so two descriptors are enqueued at the steady-state peak, and an *event* notify routes completion through a distinct interrupt-completion hook on the hardware-abstraction trait (whose modelled default is itself a spin — a true `wfi` wait is per-silicon work the simulator's synchronous DMA model cannot exercise). This is the embedded realisation of the IO-as-schedule-directive

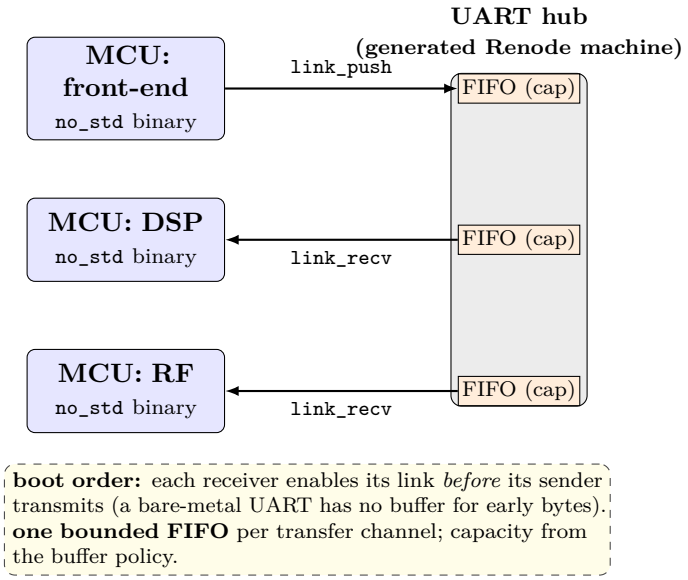


Figure 7.4: The tier-3 backend's multi-microcontroller path. The backend emits one `no_std` binary per worker plus a generated Renode machine description that wires them together over a UART hub, with one bounded FIFO per transfer channel (depth from the buffer policy). A cross-worker `Push` becomes a `link_push` on the hardware-abstraction trait and a `Wait` a matching `link_recv`. Two things have no analogue on the hosted tiers, where the operating system absorbs startup races: a boot order is computed so that every receiver enables its input before its sender transmits (a bare-metal UART has no buffer to hold an early byte), and a control-only synchronisation with no data edge is rejected, since there is no transfer to carry the ordering. The load-bearing example is the three-core hearing aid (front-end, DSP, RF), which reproduces its reference output byte-exactly under co-simulation.

claim — the same algorithm driven blocking on one edge and double-buffered-DMA on another by changing only the schedule. The honesty is in the depth and the model: the maximum buffer is two, so a deeper ring is rejected at compile time rather than silently truncated, and under the Renode model the DMA completes synchronously inside the arm, so the two in-flight slots are an occupancy property the generated code provably reaches, not a temporal overlap on this non-cycle-accurate emulator — genuine concurrent DMA on real silicon is future work. A programmed-IO transfer ignores the buffer and notify directives and renders the unchanged blocking byte loop.

Two subtleties are handled here that have no analogue on the hosted tiers, where the operating system and its socket buffers absorb startup races. First, a boot order is computed so that a receiver enables its input before a sender transmits: on a bare-metal UART channel there is no buffer to hold early

bytes, so a byte sent before the receiver is listening is simply lost. The compiler stages the releases of the cores until every transmit is preceded by its matching receive-enable. Second, a control-only synchronisation that would order two cores' external IO with no data edge between them is rejected at compile time, because no data transfer is present to carry the ordering on this substrate.

Four examples exercise this path end-to-end and reproduce their reference output byte-exactly under co-simulation: a two-worker split-and-add; a three-core hearing-aid pipeline with heterogeneous front-end, DSP, and RF-controller workers and bidirectional dataflow between them; a two-microcontroller demonstration that contrasts DMA-driven and programmed-IO transports for the same transfer; and a three-core producer–consumer pipeline whose cross-worker stream edge is the asynchronous, double-buffered, event-notified DMA-ring schedule described above, co-simulated byte-exact over a 64-byte output with the ring reaching its depth-2 occupancy. The hearing-aid example is the load-bearing demonstration that the typed, heterogeneous worker form lowers to a real multi-microcontroller system. Following the tier's discipline, tier-3 backends must compile for every required schedule, and are run in Renode where a shim and machine description exist.

7.5 Adding a backend: cost classes and the shim model

The three tiers have genuinely different cost classes, and conflating them would misrepresent the engineering. The bulk of each backend's logic — the event-list walker, the expression and index renderers, the host-election rule, the check-loop templates, and the per-transport planning substrates — lives in a shared library; each backend crate is correspondingly thin, providing only the transport-specific renderings — a few hundred lines for the simplest synchronous backends, more for the event-reactor and embedded backends, but in every case small relative to the shared codegen they sit on. This is what makes a new tier-1 backend cheap: copy a sibling, swap the wire and notification calls, and reuse the shared single-worker emitter — days to weeks of work.

A tier-2 backend is more expensive — weeks to months — because it brings in MPI library bindings, SPMD code generation, and cluster continuous integration. A tier-3 shim is the most expensive: the *first* family is months of work, because it must supply concrete DMA controllers, interrupt-vector tables, memory layout, linker scripts, and real-time validation under `no_std` constraints — much of it the generic machinery the first shim builds alongside itself. A *second* family that shares the target instruction set and has a simulator model is far cheaper: the nRF52840 shim reused the generic backend, the lowered run loop, and the input-injection and capture harness unchanged, swapping only the concrete register sequence, the memory map, and one linker

flag. This shim sprawl is nonetheless real and, in principle, unbounded: every new family is another shim, and one needing a target the toolchain does not yet build, a simulator model that does not yet exist, or a transport the trait does not express would cost more again. The system caps its own committed surface at two reference shims and treats further shims as out-of-tree contributions, which is why the capability matrix and the hardware-abstraction trait are committed, reviewable text: they are the contract a contributor's shim must satisfy. Chapter 11 returns to this maintenance burden as one of the honest costs of the approach.

Chapter 8

A Compilation, End to End

The previous chapters described the language, the pipeline, and the backends one concern at a time. This chapter puts them together. It takes a single program — the 3×3 stencil of listing 5.1 under the distributed schedule of listing 5.2, already familiar from sections 5.3 and 5.4 — and follows it through every stage of the compiler, from the two source files to the per-worker event lists and then, in the second half, to generated code on three different backends. Nothing here is new machinery; everything is a concrete instance of a pass defined in chapter 6. The purpose is to make the abstract pipeline legible by watching one program move through it, with the actual intermediate forms shown at each step.

The stencil is a good specimen because it is small enough to draw in full yet exercises the parts of the compiler that do real work: it has a neighbourhood access pattern that forces halo inference, an outer loop that is partitioned across workers, an inner loop that is both tiled and reuse-optimised, and two transfers with different IO semantics. Every figure and every intermediate form in this chapter was produced by the shipping compiler on this exact input; the generated code in the second half is reproduced from its output verbatim.

8.1 The two inputs

Recall the division of labour. The algorithm file (listing 5.1) declares two 16×16 arrays of 32-bit integers, `img_in` and `img_out`; three kernels — a side-effecting (*effectful*) `load_image`, a pure `blur3` taking nine neighbours, and an effectful `save_image` (the purity distinction is made precise in section 8.2); and a dataflow body that loads the image, writes each interior `img_out[y][x]` from the nine `img_in` neighbours around it, and saves the result. It says nothing about workers, transfers, tiling, or buffering.

The distributed schedule (listing 5.2) supplies exactly those things, and only those things. It declares five workers — a `host` and four compute workers `w0`–`w3`; places `load_image` and `save_image` on the host and `blur3` on the four

compute workers; partitions the outer `y` loop into row bands across them; tiles and reuse-optimises the inner `x` loop; and asks that `img_in` be streamed asynchronously with a two-deep, event-notified buffer while `img_out` is written back synchronously. The compiler's task is to reconcile these two files into one deterministic, sound, per-worker execution.

8.2 The algorithm sublanguage, precisely

Before tracing the passes, it is worth pinning four properties of the algorithm to this example, because several later passes are sound only because of them (section 5.2 introduced them in general).

Shaped, statically-sized arrays. The constants `H` and `W` evaluate to 16 during lowering, so `img_in` and `img_out` have the fully concrete type `i32[16][16]`; every index expression is therefore an affine function of the loop variables, reasoned about at compile time.

Shape-typed kernels with declared purity. `blur3` is *pure*, so the compiler may reorder, duplicate, or eliminate its calls: the reuse pass uses the elimination licence (carrying a previously-read value over rather than re-reading it) and the partition pass uses replication (each worker runs it independently over its own band, since a pure kernel has no hidden shared state). `load_image` and `save_image` are *effectful*: their order within a block is preserved and they are never duplicated, because they touch the outside world.

Single assignment. Each interior `img_out[y][x]` is written exactly once, which makes each datum's producer *unique*; together with the affine static indices below, that producer is also statically *identifiable*, so linking can compute each array's producing and consuming worker sets and the partition pass can later refine this to which worker owns which cell.

Affine loops and the static firing model. The loop bounds (`1..H-1`) and every array index (`y-1`, `x+1`, and so on) are affine in the loop variables with constant coefficients, and there is no data-dependent control flow or trip count. The entire *firing schedule* — which kernel instances run, in what order, reading and writing which cells — is therefore fixed by the program text, before any value is known. This is what lets the middle-end reason about decomposition, transfers, and soundness once, at compile time, with an answer that holds for every input: it is a claim about the schedule and its coordination, not about the numerical results. The price is the restricted class (section 5.5); the return is everything that follows in this chapter.

8.3 Parsing and lowering to the algorithm IR

The two files are parsed independently into abstract syntax trees and then lowered into typed intermediate representations. Lowering the algorithm performs the work the semantics above describe: it evaluates the constants (`H`, `W` $\rightarrow$ 16), resolves every array and kernel to a fully concrete shape-typed form, checks the single-assignment discipline, and checks that each effectful statement calls an effectful kernel. The result is the algorithm IR: a map of resolved constants, a map of resolved data symbols with their concrete types, a map of resolved kernels with their signatures and purity, and the dataflow body as a list of statements — a load, a doubly-nested loop whose innermost statement writes `img_out` from nine `img_in` reads, and a save. Crucially, the algorithm IR still contains no notion of a worker, a transfer, or a buffer; it is the *what*, fully typed, and nothing else.

The schedule is lowered in parallel into the schedule IR, which resolves the worker declarations, the placements, the loop directives (the `partition` on `y`; the `block` and `reuse` on `x`), and the two transfer policies. At this point the two IRs are independent objects that have never referred to each other.

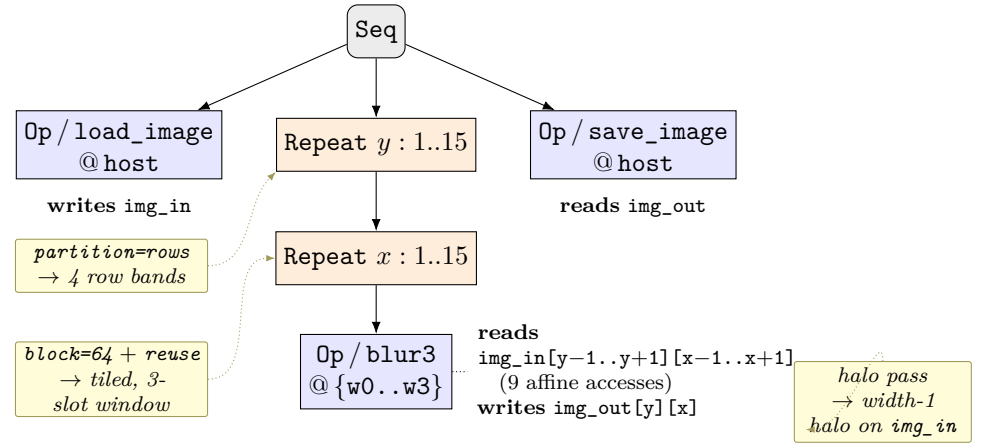
8.4 Linking: binding the algorithm to the schedule

Linking is where the two halves first meet (section 6.3). It produces the linked IR, which holds both source IRs together with the cross-references the compiler derives by reconciling them: the placement of each kernel onto its worker set, and, for each array, its producing worker set and its consuming worker sets. For the stencil this resolution is the first concrete sign of the decomposition. `load_image` and `save_image` bind to `host`; `blur3` binds to the set $\{$`w0`, `w1`, `w2`, `w3`$\}$. `img_in` is therefore produced on the host and consumed on all four compute workers; `img_out` is produced on the compute workers and consumed on the host. Those producer–consumer facts are precisely the data movements the later transfer pass must synthesise, but linking only records them; it inserts nothing yet. Linking also enforces completeness: every kernel must be placed, and every cross-worker producer–consumer pair must be reconcilable, or linking fails with a diagnostic.

8.5 The ACFG and its transforms

From the linked IR the compiler builds the application control-flow graph (section 6.4), a tree whose nodes are operations, sequences, and loops (fig. 8.1). For the stencil the initial tree is a sequence of three items: an operation firing `load_image` on the host; a repeat over `y` containing a repeat over `x` containing an operation that fires `blur3`; and an operation firing `save_image` on the host. Each operation carries a small dataflow record naming the arrays it reads and

(a) ACFG control tree (transforms tagged in yellow)



(b) dataflow DAG (the *what*, before any decomposition)

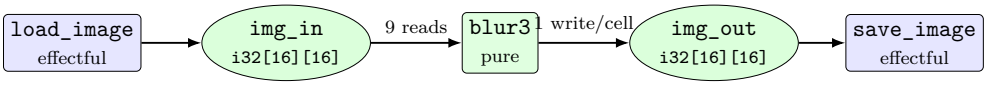


Figure 8.1: The application control-flow graph the compiler builds for the stencil. **(a)** The control tree: a `Seq` of an effectful load, a doubly-nested `Repeat` firing the pure `blur3`, and an effectful save (`@` denotes placement, so `@{w0..w3}` means the operation runs on all four compute workers). Each operation carries a dataflow record naming the arrays it reads and writes as affine index expressions (dotted). The schedule's transforms (yellow) rewrite or annotate specific nodes: `partition=rows` cuts the y repeat into four bands, `block=64` with `reuse` tiles the x repeat and adds a three-slot delay line, and halo inference records a width-one halo on `img_in`. **(b)** The same body as a dataflow DAG — still the *what*, with no worker, transfer, or buffer in sight. Loop ranges are half-open (1..15 denotes $1, \ldots, 14$).

writes and the index expressions it uses — the nine `img_in` accesses and the one `img_out` write, recorded as affine index expressions, which the inference passes will read.

The schedule's transforms are then applied to this tree, in the fixed order the pipeline defines, each pass either rewriting the tree or annotating it. Throughout, loop and band ranges are written half-open: $a..b$ denotes the integers $a, a{+}1, \ldots, b{-}1$.

- **Tiling.** The `block=64` directive on `x` strip-mines the inner loop into a tile loop over an outer tile index and an intra-tile loop. The interior `x` range is 1..15, of length 14; since the block size 64 exceeds it, there is a single tile here. The structure is nonetheless materialised, so the same

code path handles the partial-tile case that arises when the block does not divide the range.

- **Partitioning.** The `partition=rows` directive on `y` is consumed by the row-band pass, which overrides each compute worker's `y` range with its band. The outer range 1..15 likewise has length 14, which does not divide evenly by four; the compiler distributes the remainder by handing $\lceil 14/4 \rceil = 4$ rows to the first two workers and $\lfloor 14/4 \rfloor = 3$ to the last two — a floor-with-spillover policy that concentrates the surplus in the leading workers — giving the contiguous bands 1..5, 5..9, 9..12, 12..15 (fig. 8.2). The partition passes for per-worker ranges, row bands, and two-dimensional grids are mutually exclusive alternatives selected by the directive; here only the row-band pass fires.

- **Halo inference.** The halo pass reads `blur3`'s access pattern and finds that each output row y reads input rows $y - 1$, y, and $y + 1$ (and analogously $x - 1, x, x + 1$): offsets of -1, 0, $+1$ in both dimensions, recognised because each index is affine in the loop variable. It records a halo of width one on `img_in` along the partitioned `y` axis. This is the fact that makes a row-band partition correct: a worker computing rows 1..5 needs input rows 0..6 — its band plus one halo row above and below.

- **Reuse inference.** The `reuse` directive on `x` is discharged by the reuse pass, which recognises that as `x` advances by one the three-wide window of `img_in` reads shifts by one, so two of every three reads can be carried over. It annotates the loop with a three-slot circular delay line per input row — a ring buffer of three values that shifts by one on each `x` step, so the two values from the previous step are reused and only one new `img_in` read is needed per column. This is the optimisation the 2013 prototype (section 3.6) could not express; here it falls out of a one-word directive and the access pattern.

After these passes the ACFG is no longer a faithful echo of the source loop nest: it encodes four per-worker bands, a tiled and reuse-windowed inner loop, and the knowledge that `img_in` carries a one-row halo — yet still contains no transfers or synchronisations. Those are injected next.

8.6 Injecting synchronisation and transfers

Two final middle-end passes turn the decomposition implied by the ACFG into explicit coordination: the synchronisation pass and the transfer pass (section 6.5). The synchronisation pass inserts the barriers that bracket the distributed phase: one before the compute workers begin, after the host has the input ready, and one after they finish, before the host collects the output. Each barrier names its participants — here all five workers — and is given

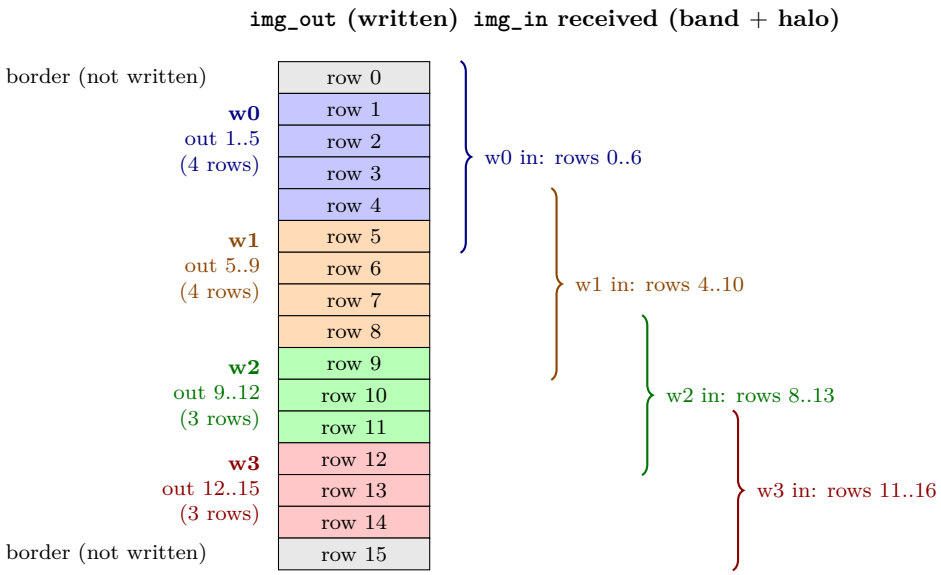


Figure 8.2: The row-band partition of the stencil's outer loop. The interior range 1..15 (rows 1 through 14; rows 0 and 15 are the never-written border) has length 14, which does not divide evenly by four. The floor-with-spillover policy hands $\lceil 14/4 \rceil = 4$ rows to `w0` and `w1` and $\lfloor 14/4 \rfloor = 3$ to `w2` and `w3`, giving the contiguous output bands 1..5, 5..9, 9..12, 12..15 (left). Halo inference adds one row on each side along y, so each worker receives a *larger* slice of `img_in` (right braces): `w0` computing 1..5 receives input rows 0..6, `w1` computing 5..9 receives 4..10, and so on. The brace overlaps are the shared halo rows. All ranges are half-open.

a stable *sync tag*, a compile-time identifier shared by every participant of that barrier (section 6.6), so the projection can match the same barrier across workers locally, without consulting any other worker's event list.

The transfer pass then consumes the producer–consumer facts from linking and the halo width from inference, and synthesises matched send/receive pairs. For `img_in` it emits, for each compute worker, a transfer from the host carrying that worker's band *plus its halo rows*: worker `w0`, which computes rows 1..5, receives input rows 0..6; worker `w1`, computing 5..9, receives rows 4..10; and so on. These transfers carry the schedule's `img_in` policy — asynchronous, two-deep buffer, event notification. For `img_out` it emits, for each compute worker, a transfer back to the host carrying just that worker's band of results, under the synchronous policy (a single-deep, blocking channel: the host receives each band before the worker could send another). The result is the fully elaborated ACFG: decomposition, halos, synchronisation, and IO semantics all explicit, and still not one line of backend-specific code.

8.7 The Petri net and the soundness gate

The elaborated ACFG is now lowered to a Petri net and checked (section 6.7 and fig. 8.3). In a Petri net, *places* hold tokens up to a capacity and *transitions* fire by consuming tokens from their input places and producing tokens in their output places. Here places model the things that hold state — each array region a worker owns, each transfer channel, and each barrier — and transitions model the things that happen: the host's `load_image` firing, the four `img_in` sends and their matching receives, each worker's `blur3` firings, the two barriers, the four `img_out` sends and receives, and the host's `save_image` firing. A channel place has a finite capacity drawn from the transfer's buffer policy — two for the asynchronous `img_in` channels, one for the synchronous `img_out` channels.

The gate runs its three checks in a fixed order (section 6.7). The first concerns *conflict-freedom*: that no place offers its token to two competing consumers. This is the load-bearing precondition, and for the nets this language produces it holds *by construction* — the lowering threads a single-token control place through each worker, serialising its transitions, so no data-dependent token routing can arise. With that source of choice removed, every firing order is a permutation of one partial order and reaches the same states, which is what makes a single replayed order representative of all of them; the argument is specific to this class and is not a general Petri-net property. The in-gate conflict check guards against a regression in the lowering rather than supplying the guarantee. On that basis the gate derives one deterministic firing order and replays it, checking *boundedness* — no channel place ever exceeds the capacity its buffer policy sets, which also catches a producer blocked because its output channel stays full — and *deadlock-freedom* — no required transition is left permanently un-fireable because an input place never receives its token. For the stencil the net passes: the two-deep input buffers never overflow, because the host pushes exactly one band into each per-worker channel before reaching the barrier, well within the two-deep capacity; and the barriers admit a completing order. Had the schedule asked for a single-deep buffer where two were needed, or arranged a cyclic wait, the gate would have rejected the program here, at compile time, naming the offending place or transition — not produced code that hangs. This check runs on this program, as on every program, before any code is emitted.

8.8 Projection to per-worker event lists

The last middle-end step projects the elaborated ACFG into one totally-ordered event list per worker (section 6.8 and fig. 8.4). The walk is deterministic, so the lists are reproducible byte-for-byte across compilations. The events are drawn from a small fixed vocabulary — fire a kernel, push a data region to a

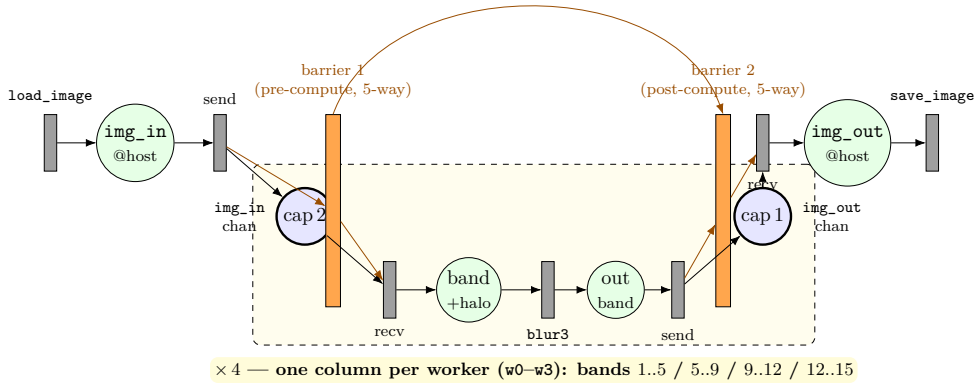


Figure 8.3: The Petri net the elaborated ACFG lowers to. *Places* (circles) model state — the array regions a worker owns and the transfer channels — and *transitions* (bars) model events: the host's `load_image`, the four `img_in` sends and matching receives, each worker's `blur3` firings, the four `img_out` sends and receives, and the host's `save_image`. The four compute workers form a 4-fold-symmetric column, drawn once inside the dashed plate. Channel places (marked "cap") carry the capacity their buffer policy sets: **2** for the asynchronous `img_in` channels, **1** for the synchronous `img_out` channels. The two orange bars are the 5-way (host + `w0`–`w3`) pre- and post-compute barriers; the orange arcs are their barrier-place arcs (no participant proceeds until all five have arrived), and the arc arching over the top is the host idling from the first barrier to the second. The gate checks, in order, conflict-freedom (no place offers a token to two competing consumers), then — on one replayed firing order — boundedness (no `img_in` channel ever exceeds 2) and deadlock-freedom, before any code is emitted.

worker, wait for a region from a worker, participate in a barrier, and loop — and they are the *sole* interface to the backend: everything the decomposition decided is encoded in them, and nothing else crosses the line.

For the stencil the host's list reads: fire `load_image`; push `img_in` to `w0`, `w1`, `w2`, `w3`; barrier; barrier; wait for `img_out` from `w0`, `w1`, `w2`, `w3`; fire `save_image`. The host stages all four input bands into the buffered channels, then the two barriers bracket the phase during which it has nothing to do, then it collects the four output bands. The two barriers are the same pre- and post-compute synchronisations of section 8.6: the first releases the workers once every input band has been pushed, the second waits until every worker has finished and pushed its output. Each compute worker's list is the mirror image. Worker `w0`'s reads: barrier; wait for `img_in` from the host; a loop over its band rows 1..5, whose body is the tiled, reuse-windowed inner loop firing `blur3`; push `img_out` to the host; barrier. The loop event carries the iteration structure

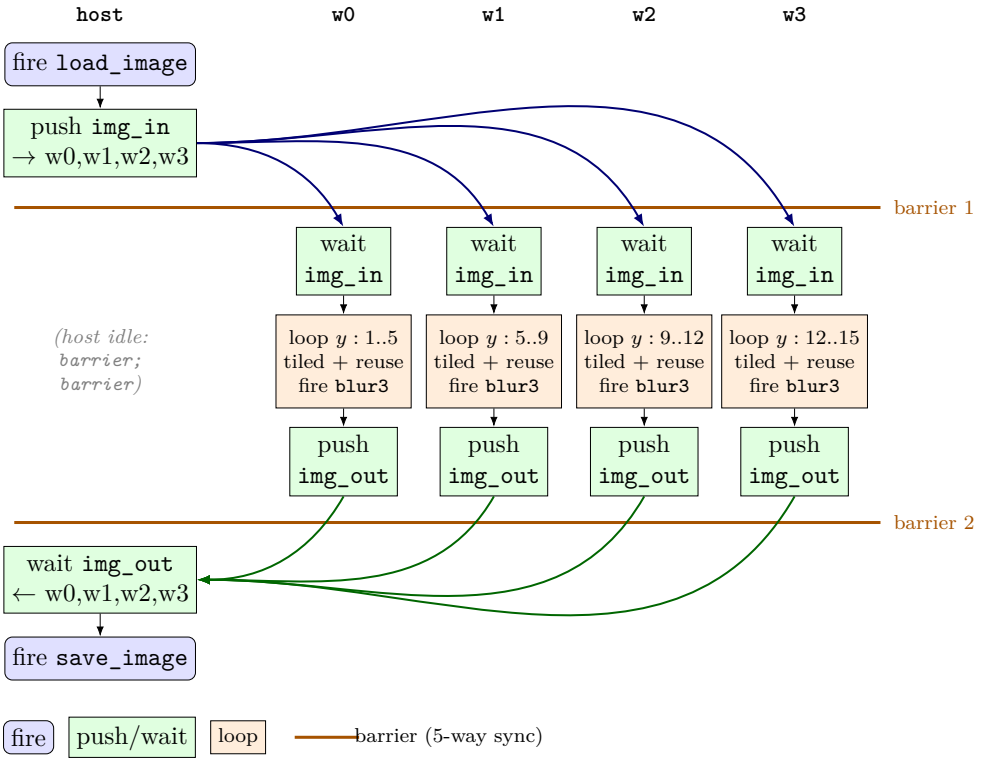


Figure 8.4: The per-worker event lists, the sole interface between the middle end and any backend, as a swimlane timeline (time flows down). The host fires `load_image`, pushes one `img_in` band into each worker's buffered channel, meets both barriers, collects the four `img_out` bands, and fires `save_image`. Each worker meets barrier 1, waits for its `img_in` band, runs the tiled, reuse-windowed `blur3` loop over its row band, pushes its `img_out` band back, and meets barrier 2. The two horizontal bars are the 5-way pre- and post-compute barriers; between them the host lane is empty — exactly the back-to-back `barrier; barrier` in the host's list — while the workers compute. Blue arrows are `img_in` transfers (host to worker), green arrows are `img_out` transfers (worker to host); the `img_in` pushes cross barrier 1 because the channels are buffered (depth 2). The loop event carries the iteration structure, not an unrolled body.

rather than an unrolled body, so a backend emits a loop construct rather than one replicated body per iteration, keeping generated code compact regardless of trip count.

These event lists are the output of the middle-end and the input to every backend. The same host-and-four-workers lists just described are what the next sections lower to shared-memory threads, to MPI ranks, and to bare-metal microcontrollers — the same events, three very different renderings.

8.9 One event list, three renderings

A backend consumes the per-worker event lists and nothing else (section 6.9). It renders each event into code for one combination of runtime and transport, and the central observation of this chapter is visible only once that rendering is laid side by side across backends: the events that compute — fire a kernel, loop — go through a *shared* renderer, while only the events that communicate — push, wait, barrier — are rendered per-backend (fig. 8.5). This is the middle-end/presentation boundary (section 6.2) — the line separating the shared, backend-independent lowering from the per-backend rendering of transport — made mechanical: there is one emitter for the inside of a worker and a small per-backend emitter for the seams between workers.

The consequence is literal, not approximate. Emitting the stencil's distributed event lists to the shared-memory `pthreads-async` backend and to the distributed-memory `mpi-nonblocking` backend produces, for each compute worker, *byte-for-byte the same compute body* — the same tiled, reuse-windowed loop, the same nine-argument `blur3` call, the same circular-buffer index arithmetic, down to the character (the transport sections of the two files, of course, differ). The two generated programs diverge only where a worker sends or receives a band or meets a barrier. The sections below show each of the three renderings of the same event lists from section 8.8: two use the hosted backends running that exact distributed schedule, and the third uses the embedded backend running a synchronous variant, as its transport requires. The final section explains why all three agree to the byte.

8.10 Shared memory: the pthreads rendering

The `pthreads-async` backend maps each worker to an operating-system thread and each transfer channel to a bounded ring buffer shared between two threads (section 7.2). The host's `main` allocates one ring per transfer — four for the `img_in` sends, each a two-slot ring matching the schedule's `buffer=2`, and four single-slot rings for the synchronous `img_out` returns — and two five-party barriers, then spawns the four compute-worker threads and runs the host's own event list inline.

Each event renders to a shared-memory primitive. A *push* becomes a `ring.push(payload)` into the destination worker's channel; a *wait* becomes a `ring.wait()` followed by a copy of the received band into the worker's local array; a *barrier* becomes a `barrier.wait()` on the shared five-party barrier; and a *fire* becomes a direct call to the kernel. The ring is a condition-variable queue — a producer blocks when it is full, a consumer blocks when it is empty — so the asynchronous, event-notified semantics the schedule asked for are realised by the operating system's condition variables rather than by any polling. The host stages the four input bands into the two-deep rings, both

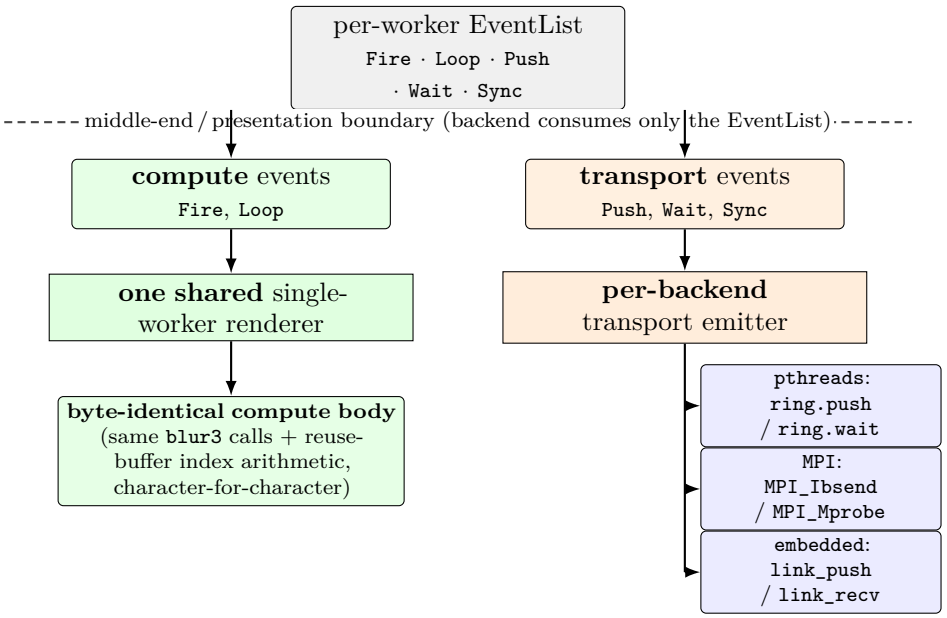


Figure 8.5: Why one event list yields byte-identical output across very different backends. A backend splits each per-worker event list into two classes. The events that *compute* — `Fire` and `Loop` — go through *one shared* single-worker renderer, so the compute body (the `blur3` calls and their reuse-buffer index arithmetic) is the *same source text* on every backend, character-for-character. Only the events that *communicate* — `Push`, `Wait`, and the barrier `Sync` — are rendered per-backend (a `Wait`, for instance, lowers to `ring.wait`, `MPI_Mprobe`, or `link_recv`), into ring buffers (`pthreads`), buffered non-blocking MPI sends (`MPI`), or HAL trait calls (`embedded`); the dashed line marks the middle-end/presentation boundary, above which is the compiler's output and below which the backend's rendering. The shared compute is what makes the cross-backend differential test pass; the per-backend transport is value-preserving, so it cannot perturb a result.

barriers pass, and the four output bands are collected; the compute worker for the first band runs exactly the loop described in section 8.5, reading its received rows 0..6 and writing its band rows 1..5.

8.11 Distributed memory: the MPI rendering

The `mpi-nonblocking` backend maps each worker to an MPI rank and emits a single program that every rank runs, branching on its rank to execute that worker's event list (section 7.3 and fig. 7.3). Where the shared-memory backend spawned four threads from a host `main`, the MPI backend emits one `match world.rank()` — a branch on the process's rank within the MPI communicator — with five arms. Rank 0 is the host; ranks 1 through 4 are the compute workers; the program is launched as five processes with `mpiexec -n 5`. The same event list that drove the host thread now drives rank 0; the same list that drove the first compute thread now drives rank 1.

The transport primitives are entirely different from the ring buffers, yet they realise the same event semantics. A transfer channel becomes a small wrapper over an MPI peer and a message tag (a small integer the receiver uses to select which incoming message to take). A *push* is a buffered non-blocking send (`MPI_Ibsend` into a process-attached send buffer) whose completion is local: it returns once the payload has been copied into that buffer, without waiting for the peer to post its receive. This is what gives the asynchronous, buffered behaviour the schedule requested, and what keeps a wait-before-push exchange from deadlocking — if two workers each waited to receive before sending, neither would proceed, whereas a local-completion send lets each post its send first and then wait. A *wait* is a matched probe (`MPI_Mprobe`, which learns the incoming element count) followed by a non-blocking matched receive and its completion. A *barrier* is `MPI_Barrier` over the whole communicator. Each channel pins its MPI message tag to the transfer's compile-time identity — every distinct transfer edge in the event list gets a unique tag — so two concurrent transfers between the same pair of ranks carry different tags and cannot be matched by the wrong receive. The rank assignment, the message tags, and the receive slices are all computed by the same projection that produced the event lists; the backend only chooses how to render a push and a wait.

And the inside of each rank is the shared compute body unchanged. Rank 1 receives rows 0..6 into the same array offset (`img_in[0..96]`: six rows of sixteen in row-major order), runs the byte-identical tiled, reuse-windowed `blur3` loop, and sends its output array back to the host, which recovers this worker's band rows 1..5 at the same offset the pthreads host used (`img_out[16..80]`) — the identical arithmetic the pthreads thread ran, wrapped in `MPI_Ibsend` and `MPI_Mprobe` instead of `ring.push` and `ring.wait`.

8.12 The capability boundary

The two renderings above were both possible because both backends advertise the capabilities the stencil's distributed schedule demands: asynchronous transfers, a buffer of depth two, and event notification. Not every backend does, and the event contract is where the mismatch is caught. The schedule's `img_in` transfer asks for `async`, `buffer=2`, and `notify=event`; a backend whose capability declaration offers only synchronous, single-buffer, blocking transfers cannot honour it. When the same program is aimed at such a backend — `mpi-blocking`, or the embedded backend of the next section — compilation *fails*, with a diagnostic naming each unmet demand: that the transfer requests asynchrony the backend does not support, a buffer depth beyond the backend's maximum of one, and an event notification mode the backend does not offer. This is not a runtime surprise or a silent downgrade to blocking behaviour — which would make the same schedule mean different things on different backends — but a compile-time rejection (section 7.1). A schedule and a backend compose only when the backend's capabilities are a superset of what the event list demands; here a capable sibling such as `pthreads-async` carries the schedule, and the synchronous backends carry the synchronous schedules of the same algorithm instead.

8.13 Bare metal: the embedded rendering

The embedded backend renders an event list to a `no_std` crate for a microcontroller, with no operating system, no heap, and no threads (section 7.4). Because its transport is a synchronous UART or DMA fabric, it declares only synchronous, single-buffer, blocking transfers, so — as just shown — it rejects the stencil's asynchronous distributed schedule. The same stencil *algorithm* under a synchronous schedule — one whose transfers drop the `async`, `buffer`, and event-notify directives in favour of plain blocking transfers — lowers cleanly, and exhibits the third rendering of the event contract.

Three things change relative to the hosted backends, and all three follow from the target rather than from the algorithm. First, storage: the heap-allocated `Vec` arrays of the hosted backends become fixed-size stack arrays (`[i32; 256]`), since there is no allocator. Second, the effectful kernels: where the hosted backends call `load_image` and `save_image` as ordinary functions, the embedded backend lowers them to calls on a *hardware-abstraction trait* — a Rust interface that each per-microcontroller shim implements concretely — so loading the image becomes a region allocation and a DMA wait, and saving it becomes a DMA push to a peripheral followed by a wait. Third, the transport: a cross-worker *push* becomes a `link_push` on the trait, keyed by the transfer's sequence tag (the same compile-time transfer identity the MPI backend mapped to a message tag), and a *wait* becomes a matching

`link_recv`; a *barrier* becomes an interrupt-driven synchronisation hook. The generic backend emits against the trait once; each microcontroller family supplies a shim that binds these hooks to real DMA descriptors, interrupt vectors, and memory addresses — the system commits two such shims, an STM32H7 and an nRF52840, against the same trait.

The *compute kernel*, though, is the same: the pure `blur3` body is copied verbatim into the embedded crate, character for character. The loop that calls it differs from the hosted renderings — this is the synchronous naive schedule, with no partitioning across workers and no reuse window, so the loop is an unpartitioned doubly-nested sweep over the whole image rather than the tiled, reuse-windowed per-band loop the distributed schedule produced. What does not change is the arithmetic each `blur3` call performs: the only kernel that changes *form* is the effectful IO, because only it touches hardware. For a multi-worker embedded schedule the cross-worker pushes and waits become real inter-microcontroller UART transfers, and a boot order is computed so that each receiver enables its link before its sender transmits, because a bare-metal UART has no buffer to hold an early byte (section 7.4); the dedicated multi-microcontroller examples exercise that path under co-simulation and reproduce their reference output byte-exactly (section 10.4).

8.14 Why the three agree to the byte

The three renderings just walked through differ in almost everything a systems programmer would notice — threads versus processes versus bare-metal cores; ring buffers versus MPI messages versus UART links; a heap versus fixed arrays — and yet the hosted two produce byte-identical output, and the embedded one reproduces the same reference bytes under co-simulation. The walkthrough makes the reason concrete rather than asserted. For the two backends running the *same* (distributed) schedule, the compute that determines every output byte is emitted by one shared single-worker renderer, so it is the *same source text*: the `blur3` calls and their reuse-buffer index arithmetic are character-for-character identical between the shared-memory and MPI programs. The embedded backend runs a different (synchronous) schedule and so emits a different loop, but the value-determining part — the `blur3` kernel body — is the same verbatim kernel. In every case that arithmetic is integer arithmetic, which is deterministic and reorder-independent by the language's design (section 5.5), so no backend's scheduling choices can perturb a value. What the backends *do* differ in — how a band is moved from one worker to another — is value-preserving: a ring, an `MPI_Ibsend`, and a `link_push` each move the band as an opaque byte payload, none reinterpreting or recomputing it, and the soundness gate has already established that none overflows or deadlocks for this program. For this stencil, whose workers write disjoint output bands with no cross-worker reduction, identical compute over a value-

preserving transport yields identical output. Where workers instead combine into a shared accumulator, byte-identity additionally requires the combine to be order-independent; the integer-arithmetic restriction supplies that guarantee (section 10.2). The cross-backend differential test of chapter 9 is the empirical check that this reasoning holds, and the walkthrough is why it holds.

Chapter 9

Validation

Chapter 8 traced one program through the compiler and argued, concretely, why the same algorithm under different backends must produce the same bytes. A genericity claim — one algorithm, many schedules, many backends, all in agreement — is only worth as much as the instrument that could refute that reasoning empirically. This chapter describes that instrument. The central design decision is to make the claim *mechanically falsifiable* rather than merely plausible: every worked example is run through every applicable schedule on every capable backend, and any disagreement, by a single byte, is a hard failure that names the offending cell. The validation strategy is therefore not a smoke test that confirms the system runs; it is a falsification rig built to make the separation of algorithm, schedule, and target fail loudly if it leaks anywhere. The chapter presents that rig (sections 9.1 and 9.2), the compile-time soundness gate as a second, orthogonal guarantee (section 9.3), the negative tests that prove every check actually bites (section 9.4), the compile-mandatory, run-best-effort discipline for the cluster and embedded tiers (section 9.5), and the reproducibility substrate the whole apparatus rests on (section 9.6). Chapter 10 reports what the rig has so far established and, as importantly, what it has not.

9.1 The test suite is the specification

The project adopts a deliberately austere rule: a capability is considered to exist only when a committed example exercises it end-to-end. There is no separate prose specification that the implementation is measured against; the corpus of worked examples, together with the matrix that declares which schedule and backend combinations each example requires, *is* the specification. A feature with no example is not a feature.

This inverts the usual relationship between document and artefact. A written specification drifts from the implementation the moment either changes, and the drift is silent — nothing forces the two back into agreement. An

executable specification cannot drift, because it is run on every change and a divergence is a build failure. The cost is that the specification is only as complete as the example corpus, and gaps in the corpus are gaps in the guarantee; this is an honest limitation, returned to in section 9.5 and section 10.5. The benefit is that every claim this document makes about what the system does is, by construction, backed by an artefact that a reader can run.

Each example is a small directory: an algorithm in the algorithm sublanguage, one or more schedules, a set of scalar kernels, a fixed input, and a reference output — a golden file produced once by an independent reference implementation and then held immutable. The reference output is the oracle. The unit of testing is a *cell* — one triple of (example, schedule, backend), the smallest thing the matrix can require and the harness can run — and a cell's emitted program is correct exactly when, fed the fixed input, it reproduces the reference output byte-for-byte. Which cells the matrix *requires* to pass is itself a committed, reviewable declaration, so the distinction between a required cell and a deliberately skipped one is explicit rather than incidental. Because the oracle is a fixed byte string rather than a re-execution of the same generator, it cannot drift in lockstep with a bug in the compiler: a defect that corrupts both the emitted code and a freshly computed expectation would still be caught, because the committed reference predates the defect.

9.2 The tier-1 bit-identical differential rig

Differential testing — running the same input through multiple implementations and treating any disagreement as a bug, without needing to know in advance which implementation is wrong — is a well-established way to find compiler defects without a complete formal oracle [15, 25]. The classical form compares independent compilers for the same language. The form used here is sharper, because the multiple implementations are not independent tools that merely ought to agree: they are the *same algorithm* compiled under the *same schedule* to different backends, and the entire thesis of the system is that they *must* agree. A disagreement is therefore not just a bug in one backend; it is direct evidence that the algorithm, schedule, or target boundary has leaked — that some decision which should have been confined to one layer has bled into another (fig. 9.1).

The rig is the seven tier-1 backends of section 7.2. They are chosen to be cheap to build and run, so the full schedule-by-backend matrix fits on a developer laptop and on every continuous-integration invocation, and they are chosen to be *maximally diverse* in the dimension that matters. They span shared memory, TCP loopback, and Unix-domain sockets; synchronous and asynchronous transfers; buffered and unbuffered channels; and blocking, polled, and event-reactor notification disciplines. The diversity is the point. A leak in the algorithm/schedule or middle-end/presentation boundary that

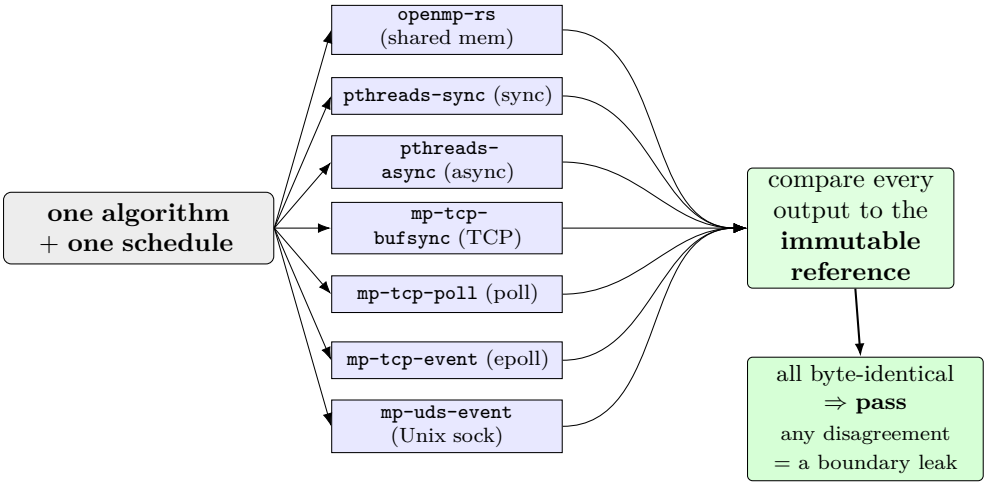


Figure 9.1: The cross-backend differential test, the thesis's falsification rig. One algorithm under one schedule is compiled to the seven tier-1 backends, chosen to be *maximally diverse* — shared memory, TCP loopback, and Unix sockets; synchronous and asynchronous; buffered and unbuffered; blocking, polled, and event-reactor notification. Each backend emits, builds, and runs, and every output is compared against the example's immutable reference. Byte-identity across all seven is the pass condition; any disagreement is direct evidence that the algorithm/schedule or middle-end/presentation boundary has leaked — a decision that should have been confined to one layer has bled into another. The diversity is the point: an error would have to corrupt the output *identically* across seven such different runtimes to escape detection.

happened to corrupt the output would have to corrupt it *identically* across seven such different runtimes to escape detection — and an error that produces the same wrong bytes on a spinning poll loop, an `epoll` reactor, and a barrier-synchronised thread pool is far less likely than one that manifests differently, or only on some, and is thereby caught. Two of the seven, `openmp-rs` and `pthreads-sync`, are deliberately capability-identical but run on different substrates, so a disagreement between just those two would isolate the fault to the substrate rather than the schedule.

Concretely, the harness enumerates every required cell and for each one emits the project, builds it with the host toolchain, runs it against the fixed input, and compares the result to the example's reference output. A cell passes only if it builds, runs, and matches. Because all seven backends are compared against the same immutable reference, a set of cells that all pass are transitively byte-identical to one another: the reference is the shared fixed point through which mutual agreement is established. The harness reports a one-line-per-cell table and a totals line; a non-zero count of required-cell failures fails the build.

A generative arm over a synthesised subclass. The curated matrix is complemented by a generative differential harness that relies on no hand-written example at all. From a seed it synthesises random programs in a structured subclass of the affine, single-assignment, integer class, now across five program families: one-dimensional element-wise integer pipelines of one to four pure scalar stages; a two-dimensional stencil whose neighbour reads in the partition axis force halo inference; a partitioned binned reduction exercised over all six combine operators (sum, bitwise or, xor, and, minimum, maximum) with identity-aware initialisation; a multi-compute-worker partition; and a bounded `for..until` early-exit loop. Each program is over randomly-sized arrays, decomposed across a real worker boundary, and is subjected to the same test the curated rig applies: compile across the tier-1 backends required for its family, run against generated input, and require mutual byte-identity, here additionally cross-checked against an in-process reference computed directly from the synthesised program. Generation is deterministic in the seed, so any divergence reproduces exactly and the harness prints the offending program and the divergent backend. Every spawned build and run carries a wall-clock timeout that kills the whole process group on expiry and reports the offending program, so a synthesised deadlock surfaces as a reproducible failure rather than a stalled run; the timeout guarantees the arm terminates and reports — it does not prove liveness, which remains the soundness gate's responsibility (section 9.3). It is run on demand rather than on every build, because each synthesised program costs several independent backend builds; a representative seeded run synthesises over a hundred programs across all five families with no counterexample. The subclass it samples is still narrower than the full grammar — it generates no floating-point, recursive, or unbounded data-dependent shapes — and one family carries a documented honesty: the generator checks the `for..until` family on the single `pthreads-sync` backend only, not seven-way. This is now a property of the *generator*, not of the construct: the curated matrix exercises the same single-worker `for..until` machinery byte-identically across all seven tier-1 backends (section 10.6), so the construct itself is no longer single-backend. The generator's narrower scope is a not-yet-lifted harness limitation — its per-family backend set still pins this family to `pthreads-sync` — rather than a gap in the backends. The arm therefore sharpens the corpus-coverage limitation of section 9.1 rather than removing it; the honest accounting of what it does and does not buy is deferred to section 11.4.

9.3 The soundness gate as a compile-time guarantee

The differential rig is a dynamic check: it runs emitted code and inspects the bytes it produces. The soundness gate is its static complement. As described in section 6.7, before any backend code is emitted the Petri-net intermediate

representation [17] is checked for conflict-freedom, boundedness, and deadlock-freedom, and a failure is reported as a compile error naming the offending place or stalled transition rather than as a runtime hang discovered later.

For validation the gate matters for a reason beyond the guarantee it gives the user: it runs on *every* build of *every* cell in the matrix. Each of the hundreds of differential cells therefore also exercises the gate on a real, shipping net, and every committed schedule is, by the fact that it builds at all, a positive witness that the gate accepts sound nets without false rejection. The gate currently rejects nothing in the example corpus — which is the correct outcome, since every committed schedule is sound, and which is only meaningful because the negative tests of section 9.4 prove the gate is capable of rejecting. A check that accepts everything because it is empty looks identical, in a green build, to one that accepts everything because everything is genuinely sound; only an adversarial test that hands it a deliberately broken net can tell the two apart.

The gate's soundness is bounded, and the bound is stated plainly. It is an exact replay over a single, statically-determined firing order, sound for the restricted class of nets this language produces precisely because those nets have no non-deterministic token routing, and it is emphatically not a general model checker for arbitrary Petri nets (section 6.7). Within its class it is both sound and inexpensive; outside that class it makes no claim. Conflating the two would overstate the result, so the document does not.

9.4 Negative falsifiers: proving the checks bite

A passing test suite establishes that the checks accept what they should. It says nothing about whether they would *reject* what they should — a check that can never fail provides no assurance at all, however green the build. The project therefore carries a set of *negative falsifiers*: tests that deliberately inject a fault and fail unless the corresponding check catches it. Each is paired with an explicit signal proving the injection actually perturbed something, so that a falsifier which silently became a no-op — the most dangerous failure mode for a negative test, since a no-op test is permanently and invisibly green — is itself caught.

Three such falsifiers guard the dynamic rig at the level of the test harness. The *determinism falsifier* injects nondeterminism into the code generator and requires that the determinism check — which compiles each cell twice and compares the two emissions — detects the perturbation; it reports the number of cells it perturbed and fails if that number is zero. The *cross-backend falsifier* corrupts the byte-level message framing of one message-passing backend and requires that the differential test detects the resulting byte divergence, defined precisely as a comparison failure on exactly that backend, so the falsifier cannot be satisfied by an unrelated failure. The *required-coverage falsifier* injects a typo'd required cell — a cell the matrix demands but that can never run

— and requires that the coverage guard, which checks that every declared-required cell was actually run, refuses to let it silently vanish; this defends the specification itself against erosion, since without the guard, deleting a required cell would quietly shrink the matrix and weaken every claim built on it. The same coverage falsifier is run a second time scoped to the cluster tier, so the message-passing matrix is protected by the identical mechanism rather than an approximation of it.

The static soundness gate is exercised adversarially by a separate, complementary means: rather than an end-to-end injection, its rejection capability is established by unit tests that hand the gate deliberately unsound nets — an over-capacity net that must be rejected as a boundedness error, a stalling net that must be rejected as a deadlock, and a net in which two consumer transitions compete for one place's token (a free-choice conflict) that must be rejected as a conflict — and the tests assert that each is refused with the correct diagnosis. The gate's acceptance of sound nets, in turn, is witnessed by every passing cell in the matrix. So the positive and negative directions are both covered for the gate, just not by the same instrument that covers the harness.

These falsifiers and adversarial tests are not decoration. They convert each check from an assertion that *happens* to hold on the current corpus into one that has been shown to fail on a corpus engineered to break it — the difference between a claim that is true and a claim that is testable. A green differential matrix is only meaningful because the cross-backend falsifier proves the matrix can go red.

9.5 Tier-2 and Tier-3 validation

The tier-1 backends are cheap enough to run in full on every build. The cluster and embedded tiers are not: they need a message-passing runtime or an instruction-set simulator that is not present in every environment. The project handles this with the *compile-mandatory, run-best-effort* discipline introduced in chapter 7, and what follows is its validation-specific reading. Every backend, on every tier, must *compile* for every schedule it is required to support; compilation is never waived, because a backend that fails to generate buildable code for a required schedule is a hard failure regardless of whether a runtime is on hand to execute it. The *running and diffing* of the emitted code, by contrast, is performed wherever the necessary runtime exists and is treated as a best-effort confirmation on top of the mandatory compile.

For the two MPI backends [16] of section 7.3, the runtime is a local `mpiexec`, which suffices to launch the single-program multiple-data binary across ranks on one machine; a full cluster is not required to establish value-correctness, only to measure performance, which is out of scope here. Where `mpiexec` is available the cluster cells are run and their output diffed against the same

reference files the tier-1 rig uses, so the MPI backends are held to byte-exact value-correctness, not merely to compiling.

For the embedded backend of section 7.4, the runtime is the Renode instruction-set simulator, which can co-simulate several microcontrollers wired together over a virtual UART fabric. The compile-mandatory half is exercised by cross-compiling each worker as a `no_std` library for the embedded target; the run-best-effort half is a standing gate that generates the bare-metal binaries, cross-compiles them, co-simulates them, captures the output bytes, and diffs them against the reference file — failing loud on any byte mismatch. Because the embedded tier needs a target toolchain and a simulator closure that the default development shell does not carry, its cells are marked as handled by this dedicated gate rather than by the tier-1 runtime matrix; the separation is explicit, so a reader is never misled into thinking an embedded cell ran inside the tier-1 totals when it was in fact validated by the simulator gate.

9.6 Reproducibility

The entire apparatus rests on two reproducibility properties, one of the toolchain and one of the compiler.

The toolchain is pinned. The build environment — the Rust compiler version, every dependency, the MPI libraries, the embedded cross-compiler, and the simulator — is described by a Nix flake — a content-addressed, declaratively pinned specification — so that the same inputs produce the same environment on any machine that evaluates it [8]. This matters for a differential test in particular: if two backends were built with two different compiler versions, a disagreement between them could be an artefact of the toolchain rather than evidence of a boundary leak, and the falsification rig would be measuring the wrong thing. Pinning removes that confound. The single pinned toolchain is also the sole source of truth for the minimum supported compiler version, so there is no second place for it to drift out of agreement with.

The compiler is deterministic. The projection that turns the elaborated program into per-worker event lists walks the structure in a fixed source order and keeps all worker collections in sorted maps, so two compilations of the same program emit byte-identical *source* (section 6.8). This is not a convenience; it is a precondition of the whole validation strategy. Byte-identical emission is what makes the determinism check meaningful — it compiles each cell twice and compares — and it is what lets the differential test attribute any divergence to a genuine difference between backends rather than to incidental nondeterminism in the generator. Determinism of the compiler and pinning of the toolchain are, together, the foundation on which every result in the next chapter stands.

Chapter 10

Results

This chapter reports what the validation apparatus of chapter 9 has established. Its core is correctness and coverage: how much of the schedule-by-backend space is exercised, whether the backends agree, whether the soundness gate behaves as designed, and how far the cluster and embedded tiers have been confirmed. The *runtime speed* of the emitted code is out of scope: the contribution this document defends is the separation and its falsifiability, not the performance of the generated programs, and a credible speed claim would demand a benchmarking methodology this work does not undertake (section 12.5). The chapter does, however, close with a set of *static* engineering characteristics — compile cost, generated-code footprint, and how the soundness gate's analysis scales (section 10.6) — reported as descriptive facts about the compiler, the cost of its genericity, rather than as performance claims. Every *structural* number below (cell counts, code and generated-code sizes, net node counts) is drawn from the live toolchain; the wall-times of section 10.6 are indicative medians on one machine, not measurements.

10.1 Coverage: examples, schedules, and backends

The example corpus comprises twenty-nine worked examples spanning twenty-eight distinct algorithms — the convergence Jacobi appears as two boundary variants, a converged early-exit and a cap-hit run that exercises the worst-case bound — ranging from elementwise addition and reduction through stencils, separable filters, matrix multiplication, histogram, producer–consumer pipelines, a wavefront, the game of life, bitonic sort, a small convolutional-network inference, a three-stage hearing-aid pipeline, transpose, Jacobi iteration, sparse matrix–vector multiplication, gather and scatter kernels, and a family of binned-combine reductions (reductions that accumulate a partial result per bin, such as a per-bin exclusive-or or a per-bin minimum). Each example carries one or more schedules expressing different decompositions and IO disciplines, and the differential matrix declares, for each schedule, the set of backends required to

support it.

The full tier-1 matrix consists of 504 declared cells across the seven shared-memory and message-passing CPU backends. Of these, 443 pass — they emit, build, run, and reproduce the reference output byte-for-byte — 61 are skipped for a stated reason, and none fail. The skipped cells are not silent gaps: the matrix records a reason for every one, and they fall into a few well-defined categories. The largest group, thirty-five cells, are embedded-tier fixtures that are validated by the simulator gate of section 10.4 rather than by the tier-1 runtime rig, since the embedded backend emits `no_std` code that the hosted run-and-diff harness cannot execute. The remaining twenty-six divide cleanly: twenty-four are genuine capability mismatches — an asynchronous, buffered schedule asked of a synchronous, single-buffer backend, which is rejected at compile time as the capability matrix prescribes (section 7.1), with the schedule's differential requirement met instead by a backend that does offer asynchrony (the four newest, the synchronous tier-1 backends correctly refusing the double-buffered DMA-ring schedule of section 7.4) — and two are a language shape honestly unsupported on the multi-worker path: an in-loop worker-to-worker push, documented as such (section 10.5). The two `for..until` break-loop examples (a converged and a cap-hit variant) no longer contribute skips: their single-worker schedule was lifted from one backend to all seven, so their twelve formerly-skipped sibling cells now pass and agree byte-for-byte, leaving only the genuinely distributed break as future work. Figure 10.1 renders the matrix as an example-by-backend heatmap.

The shape of this coverage is what gives the next section its force: it is not a handful of examples on one backend, but a broad cross-product of decompositions, IO disciplines, and runtimes, exercised together on every build.

10.2 Bit-identity across the tier-1 matrix

The central result is that the seven tier-1 backends agree byte-for-byte. Across the 443 passing cells, every backend required for a given example and schedule reproduces that example's immutable reference output exactly, and the backends are therefore byte-identical to one another, transitively, through that shared reference. This holds uniformly across the diversity catalogued in section 9.2: the same algorithm under the same schedule produces the same bytes whether its workers are threads sharing memory, processes exchanging messages over TCP loopback, or processes over Unix-domain sockets; whether transfers are synchronous or asynchronous, buffered or unbuffered; and whether the notification discipline is a blocking receive, a non-blocking poll, or an event reactor. The decomposition and IO machinery moves the same values to the same places by the same final ordering, regardless of the runtime mechanism used to do so. That is the genericity claim, and across the exercised matrix it is not falsified.

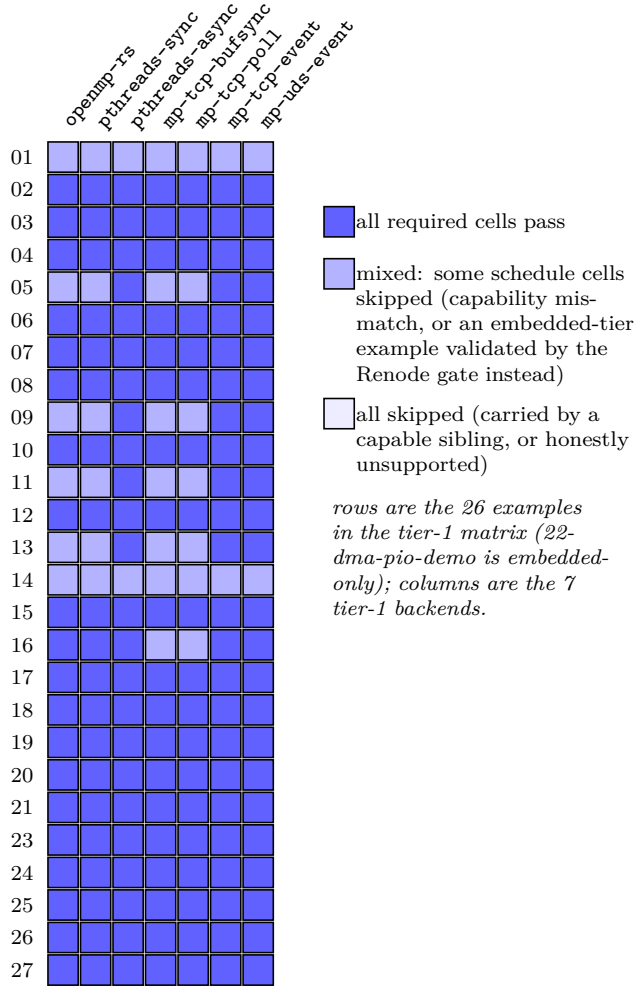


Figure 10.1: The tier-1 differential matrix as an example-by-backend heatmap (rows are the 26 examples in the tier-1 matrix — `22-dma-pio-demo` is embedded-only, hence the gap; columns are the seven tier-1 backends, of which `mp-uds-event` uses Unix-domain sockets). Each cell aggregates, over an example's schedules, its status on one backend: a dark cell (P) means every required schedule passes there, a mid cell (M) means some schedule is skipped, and a light cell (s) means all are skipped. Of 504 declared cells across the seven tier-1 backends, 443 pass (emit, build, run, and reproduce the reference byte-for-byte), 61 are skipped for a recorded reason — the largest group embedded-tier fixtures validated by the simulator gate, the rest genuine capability mismatches carried by a capable sibling backend or a few honestly-unsupported language shapes — and none fail. The broad band of dark cells is the cross-product of decompositions, IO disciplines, and runtimes exercised together on every build.

One family of examples is worth singling out because it stresses the part of the claim most likely to break under reordering. A distributed reduction that combines per-worker partial results at a host accumulator is only order-independent if the combining operation is associative and commutative; otherwise the order in which workers arrive — which legitimately varies across backends and runs — would change the result, and byte-identity would be lost. The system makes the combining operation explicit — declared as an attribute on the reducing kernel in the algorithm (section 5.2) — and supports six of them: integer sum, bitwise or, bitwise xor, minimum, maximum, and bitwise and, each with its correct algebraic identity element so that the accumulator is initialised soundly. Dedicated examples exercise the non-trivial cases — an xor-combined binned reduction, an integer-minimum reduction whose identity is the type maximum rather than zero, and the same in floating point. The discipline is enforced rather than assumed: a floating-point *sum* combine is *rejected at compile time*, because IEEE-754 addition is not associative and a distributed float-sum could not be made byte-identical across arrival orders. Refusing the operation is more honest than silently emitting code that happens to agree on the test input and diverges on another. Figure 10.2 draws this as a fan-in net: the gate fixes the firing order, but byte-identity of the folded value additionally needs the combine to be order-independent, which the six admitted operators are and a float sum is not.

10.3 Soundness-gate outcomes

The soundness gate runs on every build of every cell, so the matrix above is also a record of the gate's behaviour on hundreds of real, shipping nets. On the committed corpus the gate rejects nothing: every schedule that is supposed to be sound is accepted, with no false rejections. This is the outcome a correct gate should produce on a corpus of correct schedules, and on its own it would be weak evidence — an empty check produces the same green build. It is made meaningful by the negative result alongside it. As described in section 9.4, the gate's three properties are each exercised adversarially by a unit test that hands it a deliberately unsound net: an over-capacity net rejected as a boundedness error, a stalling net rejected as a deadlock, and a free-choice net rejected as a conflict. The gate accepts every sound net in the corpus *and* is demonstrated capable of rejecting unsound ones; both halves are necessary, and both hold. The harness's own checks — determinism, the cross-backend differential, and required-coverage — are guarded separately by the three injection falsifiers of section 9.4, so that no check in the pipeline, static or dynamic, is trusted on its green result alone.

It bears repeating where the boundary of this result lies. The gate is sound for the statically-ordered net class the language produces, by exact replay over one firing order, and is not a general Petri-net model checker (section 9.3).

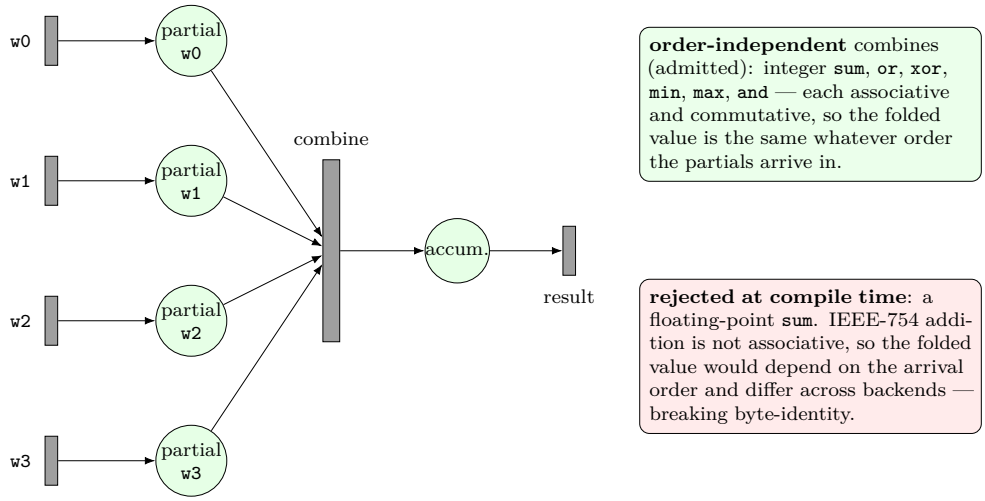


Figure 10.2: A distributed reduction as a fan-in net: each worker fires a partial result into its own place, and a single *combine* transition folds all the partials into one accumulator. The soundness gate fixes the *firing order*, but byte-identity of the *value* across backends needs more: the combine must be order-independent, since the partials may arrive in any order the schedule permits. The six admitted combine operators are associative and commutative, so the result is invariant under arrival order; a floating-point `sum` is not (IEEE-754 addition does not associate) and is therefore rejected at compile time rather than allowed to produce backend-dependent bytes. This is the determinism boundary of section 11.3 drawn on the net.

The result is "the gate correctly classifies the nets this system generates," not "the gate decides boundedness for arbitrary nets," and the document claims only the former.

10.4 Cluster and embedded value-correctness

Beyond the tier-1 rig, the cluster and embedded tiers are confirmed to the extent their runtimes allow, under the compile-mandatory, run-best-effort discipline of section 9.5.

The two MPI backends compile for every schedule they are required to support and, where a local `mpiexec` is available, are run and diffed against the same reference files as the tier-1 backends. On that runtime every exercised MPI cell reproduces its reference output byte-exactly: the single-program multiple-data binary, dispatched across ranks, computes the same values as the shared-memory and loopback backends. The blocking and non-blocking MPI backends thus extend the byte-identity result from the laptop rig to a distributed-memory runtime, for the cells exercised. The limit to be clear

about is that this is value-correctness on a local launcher, not a measurement on a production cluster, and that the backends emit correct point-to-point communication rather than recognising and emitting collective operations — a deliberate deferral noted in section 7.3.

The embedded tier is confirmed by co-simulation. A standing gate generates the bare-metal binaries for four multi-microcontroller examples, cross-compiles them for the embedded target, co-simulates the microcontrollers wired over a virtual UART fabric, captures the output bytes, and diffs them against the reference file. All four reproduce their reference output byte-exactly: a two-microcontroller split-and-add over a 1024-byte payload; the real three-core hearing-aid pipeline, with heterogeneous front-end, DSP, and RF-controller workers and bidirectional dataflow, over a 512-byte output; a two-microcontroller demonstration that drives two transfers by DMA and one by programmed-IO over the same fabric, byte-exact over 1024 bytes; and a three-core producer–consumer pipeline whose cross-worker stream edge is the asynchronous, double-buffered, event-notified DMA-ring schedule (section 7.4), byte-exact over 64 bytes with the producer's depth-2 descriptor ring observed reaching full occupancy. The hearing-aid result (section 7.4) is the load-bearing embedded demonstration: the typed, heterogeneous, multi-worker form lowers all the way to a real multi-microcontroller system whose per-frame values are correct end-to-end, including the staged boot-order handling that a bare-metal UART channel requires and that the hosted tiers never need.

A second, separate gate exercises hardware-family portability rather than the multi-microcontroller wiring. It generates three *single-worker* examples — the element-wise add, the stencil, and the producer–consumer — for a second microcontroller family, the nRF52840 (a Cortex-M4F), and confirms each byte-exact against the same frozen reference the hosted tiers use. The nRF52840's UARTE EasyDMA transmit — a pointer-and-length DMA engine — is a genuinely different register interface from the STM32's USART, so reaching the second family through the *unchanged* hardware-abstraction trait is what shows the trait is a portability seam rather than a single-target abstraction: the two families share the generic backend, the lowered run loop, and the kernel extraction, and differ only in the concrete shim, the memory map, and one linker flag. This second family is single-worker only; it does not participate in the multi-microcontroller co-simulation above (a multi-MCU nRF52840 wiring is out-of-tree work), and its byte-exact evidence covers the value-transport path; the trait's timing and diagnostic hooks are structurally mirrored from the first family but, the examples carrying no timing assertion, are not separately exercised on the second (section 11.3).

10.5 What the falsification rig establishes, and its limits

It is worth stating precisely what these results do and do not establish, because the value of a falsification rig lies as much in the boundary of its claim as in the claim itself.

What the rig establishes is this. For the exercised matrix — twenty-nine examples, their schedules, and the backends each requires — the separation of algorithm, schedule, and target holds to the strongest available standard: byte-identical output across seven structurally diverse CPU runtimes, extended to a distributed-memory MPI runtime and to bare-metal multi-microcontroller co-simulation for the cells those tiers reach. The soundness gate correctly accepts every sound schedule in the corpus and is demonstrated able to reject unsound nets. Every check that underwrites these claims has a negative test proving it bites, so a green result is informative rather than vacuous, and the whole apparatus is reproducible from a pinned toolchain and a deterministic compiler.

What the rig does not establish is equally important. Byte-identity is shown over the exercised cells, not proved for all programs the grammar admits; the specification is exactly as complete as the example corpus, and a shape absent from the corpus is outside the guarantee. Several such shapes are known and recorded rather than hidden: a small number of cells are skipped because a language construct is supported on the single-worker path but not yet on the multi-worker (distributed) path. The surviving such case is a worker-to-worker transfer nested inside a loop body, reported as a skip with a stated reason, not papered over. A data-dependent loop break, of the kind the language's termination restriction already circumscribes (section 5.5), used to sit in this category too, but its single-worker form now runs byte-identically across all seven tier-1 backends; only its distributed (collective-decision) form remains future work. The cluster result is value-correctness on a local launcher, not a cluster-scale measurement, and the MPI backends emit point-to-point rather than collective communication. The embedded result covers two reference microcontroller shims (an STM32H7 and an nRF52840, across two distinct transmit architectures), but only the STM32H7 family is exercised in the multi-microcontroller wiring; the nRF52840 is the single-worker portability check described above, and further hardware families are out-of-tree work. And, as the gate's design requires (section 9.3), its soundness guarantee is confined to the statically-ordered net class the language produces.

The honest summary is that the falsification rig has had every opportunity to refute the central claim over a broad and diverse matrix, and has not — while the matrix's documented edges mark exactly where the claim has not yet been put to the test. That distinction, between what has survived falsification and what has not been exposed to it, is the one this chapter is most concerned

to keep clear.

10.6 Quantitative characteristics: the cost of genericity

The results above concern correctness. This closing section reports a small set of *static* engineering characteristics of the compiler and its output. They are deliberately not performance claims: none measures how fast the emitted programs run, which would require a benchmarking methodology this work does not undertake (section 12.5). What they characterise instead is the cost of the genericity the document argues for — how much code a backend takes to add, how large the generated projects are, and how the soundness gate's analysis scales with problem size. The structural figures (code lines, generated lines, net node counts) are *exact and reproducible*, because the compiler's emission is deterministic (section 9.6); the wall-times are indicative medians on the development machine, reported only to bound an order of magnitude. The measurement commands are given with the reproducibility material of appendix D.

Backend footprint

Section 7.5 argued that a backend is cheap to add because the bulk of its logic is shared. Table 10.1 quantifies that claim by code lines. The shared codegen library is 5405 lines and the backend-independent middle-end a further 19385; against that common base, the two thinnest tier-1 backends are 161 and 164 lines — a synchronous transport over a socket, expressed almost entirely as a swap of the wire and notification calls. The event-reactor backends are larger (around a thousand lines) because they carry a reactor and per-channel queue machinery, and the embedded backend is the heaviest at 5016 lines, consistent with the tier's far higher cost. The numbers bear out the shape the architecture predicts: per-backend code is small relative to the shared base it sits on, and grows with the genuine novelty of the transport, not with boilerplate.

Generated-code footprint

A second cost is the size of the projects the compiler emits. Table 10.2 reports the generated Rust for one representative cell — a two-worker split-and-add — across the seven tier-1 backends, counting every emitted source line except the kernel bodies, which are copied verbatim from the algorithm's sibling file rather than generated. The shape is informative. The shared-memory backends emit a single compact driver of seventy to ninety-odd lines; the message-passing backends emit an order of magnitude more — seven hundred to sixteen hundred lines — because each materialises a wire-protocol module and one binary per worker, and the event-reactor variants add reactor scaffolding on top. The

Crate	Role	Code lines
`nucleus-compiler`	backend-independent middle-end	19,385
`backend-common`	shared codegen library	5,405
`mp-tcp-common`	shared socket-transport substrate	612
`mp-tcp-bufsync`	tier-1 (TCP, sync)	161
`mp-tcp-poll`	tier-1 (TCP, poll)	164
`openmp-rs`	tier-1 (shared memory, rayon scope)	444
`pthreads-async`	tier-1 (threads, async)	864
`mp-tcp-event`	tier-1 (TCP, event reactor)	974
`pthreads-sync`	tier-1 (threads, sync)	985
`mp-uds-event`	tier-1 (Unix socket, reactor)	1,165
`mpi-blocking`	tier-2 (MPI, blocking)	581
`mpi-nonblocking`	tier-2 (MPI, non-blocking)	658
`embedded-pattern`	tier-3 (`no_std` + shim)	5,016

Table 10.1: Backend footprint, in code lines (comments and blanks excluded) of each crate's source tree, measured with `tokei`; dedicated integration-test directories are excluded uniformly across all rows, and the small amount of inline unit-test code that remains in a few crates is likewise counted uniformly. The shared base (`backend-common` plus the middle-end) is large; individual tier-1 backends are small, the thinnest under two hundred lines. The `embedded-pattern` figure is an outlier not because of large per-chip shims — the two committed shims are small — but because the generic tier-3 backend carries multi-microcontroller machinery the hosted tiers do not need (boot-order computation, the UART-hub plan, cross-MCU control-sync) together with its inline tests.

footprint is therefore a direct readout of how much machinery a transport genuinely needs: crossing an address-space boundary costs generated code that staying within one does not.

Crucially, this footprint is a function of the *program text and worker count*, not of the problem size. The generated driver preserves loops rather than unrolling them (section 6.8): the 16×16 stencil compiles to a driver with two real `for` loops in eighteen lines, and a hypothetical 1024×1024 stencil would compile to the same eighteen lines. Generated-code size is bounded by the schedule, not the data.

Cross-worker communication volume

A third characteristic is how much data a decomposed program moves between workers. Each cross-worker edge whose inferred per-worker region is a single contiguous span now transmits only that span rather than the whole array, on every tier-1 backend and under MPI (section 11.3). The reduction is exact, because the wire shape is a deterministic function of the partition: on the four-

Backend	Worker model	Generated lines	Bytes
`openmp-rs`	shared memory (one driver)	70	2,779
`pthreads-sync`	shared memory (one driver)	71	2,726
`pthreads-async`	shared memory (one driver)	94	3,391
`mp-tcp-bufsync`	processes (wire + per-worker)	716	29,803
`mp-tcp-poll`	processes (wire + per-worker)	720	30,029
`mp-uds-event`	processes (wire + reactor)	1,133	44,700
`mp-tcp-event`	processes (wire + reactor)	1,607	66,741

Table 10.2: Generated source for one cell (the two-worker `02-split-add` schedule) across the seven tier-1 backends, excluding the verbatim kernel file. Shared-memory backends emit a single small driver; message-passing backends emit a wire-protocol module plus one binary per worker, an order of magnitude larger.

worker stencil, where each worker reads its own row band plus a one-row halo, the cross-worker traffic falls from 8 192 to 2 304 bytes, a 71.9 percent cut; on the four-worker row-partitioned matrix multiply, where each worker owns a row band of the left operand and the full right operand, the combined operand-and-result traffic falls from 12 288 to 6 144 bytes, a 50 percent cut. The figures are static counts derived from the partition arithmetic, not measurements, and they are honestly partial: only edges with a contiguous span narrow (strided column bands, non-prefix halos, accumulator fan-ins, and conservatively-widened data-dependent transfers stay whole-array), the embedded tier stays whole-array entirely, and per-worker *allocation* is unchanged — every worker still allocates the array full-shaped and pastes its received span into place, so this is a wire-volume saving, not yet a memory-footprint one.

The analysis net and how the gate scales

The one place where problem size does drive cost is the soundness gate. The gate replays a single firing order over the Petri net (section 6.7), and that net is *expanded over the iteration space*: it carries roughly one place and one transition per kernel firing — about two nodes per firing — so its size grows linearly with the number of firings the program performs. Table 10.3 makes the contrast explicit. The generated driver stays around twenty lines across every example, but the analysis net ranges from about a hundred nodes for the small convolutional-network example to 8199 nodes for the $16 \times 16 \times 16$ matrix multiply — the latter about twice the $16^3 = 4096$ kernel firings its triple loop performs, since each firing contributes both a transition and a place. Decomposition adds to this too but far less steeply: the stencil's net grows from 397 nodes on one compute worker to 1,041 across four, as per-worker channels and barriers are introduced.

Example (naive schedule)	Approx. firings	Net nodes	Generated lines
`13-cnn-inference`	(layered)	101	20
`05-stencil`	14^2 (interior of 16^2)	397	18
`01-elementwise-add`	256	519	17
`03-reduction`	256	523	21
`07-matmul`	16^3	8,199	21

Table 10.3: The two cost regimes. Generated-code size is flat in problem size (loop-structured), while the expanded Petri net is spread over the iteration space — roughly one place and one transition per kernel firing, so node count is about twice the firing count. Net node counts are exact (deterministic emission); "firings" is the dominant loop volume. Every row here is a single-worker (naive) schedule, hence buffer-free, so the gate now proves it sound symbolically from the rolled ACFG and never builds the expanded net (section 10.6); the counts are the expansion the symbolic path avoids. The `13-cnn-inference` firing count is marked *layered* because it is spread across several heterogeneous convolution and pooling kernel invocations rather than a single dominant loop volume.

This expanded net is what the gate *would* build for every program; it is also where the cost lived, since both its construction and the single-order replay are linear in the number of firings. That was once an unqualified limitation. It has now been lifted for the common cases, and the gate decides soundness in one of three regimes.

For a *buffer-free* program — one with no cross-worker transfer, which includes every single-worker schedule and so the entire table 10.3 corpus, the matrix multiply included — soundness is proved *symbolically* from the rolled ACFG, without building the expanded net at all. Such a net consists only of per-worker control chains, and these are bounded, deadlock-free and conflict-free by construction: every control place has capacity one, a single producer transition that fires once, and a single consumer, so none of the three failure modes the gate checks (capacity overflow, stall, free-choice conflict) can arise — each of those requires a shared buffer place, which only a cross-worker transfer creates. The gate discharges such a net in one structural walk of the rolled ACFG, whose size is a function of the program text, not the iteration count. Compiling the matrix multiply at progressively larger dimensions — raising N from 16 to 256 in the source, from about four thousand to about sixteen million kernel firings — leaves the driver compile time essentially unchanged, since the gate no longer expands the net; the expansion the fast path skips would itself reach about $2N^3 \approx 33$ million nodes at the upper end.

The second regime extends the symbolic proof across a cross-worker boundary, to the *single-shot matched-pair* communicating subclass: a distributed schedule in which each transfer is one `Push` matched by one `Wait`, both at

loop depth zero, with no pre-marking, so the buffer place's peak occupancy is exactly one and at most the declared capacity. This is the shape of the corpus's distributed reductions, stencils, matrix multiply, sparse mat–vec, histogram, and scatter — the dominant distributed class. For it the gate proves boundedness, deadlock-freedom, and single-consumer conflict-freedom directly from the rolled ACFG, against a firing-order invariant pinned by its own property tests, and an A/B harness confirms the symbolic verdict equals the expanded-replay verdict on every gated corpus cell plus a set of synthetic negatives. The payoff is the same flatness the buffer-free case enjoys: building the four-worker distributed matrix multiply at $N = 512$ — whose expanded net would project to several hundred million nodes and over a hundred gigabytes — gates at about 7.2 megabytes of peak resident memory, only some 350 kilobytes more than at $N = 16$ across a thirty-two-thousand-fold rise in firings. The wall the symbolic path removes here is the one that would otherwise make compiling exactly the decompositions one would want to scale-test infeasible on commodity hardware.

The third regime is the honest residual. A communicating schedule that does *not* fit the single-shot shape — a loop-carried transfer whose buffer occupancy depends on cross-iteration drainage (the `16-jacobi` distributed stencil), or a pipelined transfer that intentionally keeps several frames in flight (the producer–consumer and convolutional-network pipelines) — still falls back to the expanded single-order replay, because its boundedness and liveness depend on the firing-order interleaving, which the symbolic analysis does not yet cover for these shapes. For those nets the cost remains linear in the number of firings. The fallback is loud and never silent: anything outside the proven subclass is routed to the expanded replay rather than guessed at, because approximating the occupancy argument imprecisely could accept a net the replay would reject, trading soundness for scaling. Lifting the bound for the loop-carried and pipelined shapes too — a steady-state occupancy argument over their looped buffer structure — is left as future work (section 12.5).

Compile cost

End to end, a single cell compiles in roughly 6–10 milliseconds on the development machine: parse, link, elaborate, lower, run the gate, project, and emit. For the smaller examples this wall-time is dominated by process startup rather than by the pipeline, so the figures should be read as an order of magnitude, not a measurement. With the symbolic fast path (section 10.6) both the single-worker examples — the matrix multiply included — and the single-shot-matched-pair distributed schedules (the distributed matrix multiply, reductions, stencils, and the rest of that dominant class) compile in time that does not grow with their firing count, because the gate proves them symbolically and never expands their nets. The compiles that now stand out are the schedules outside that subclass — the loop-carried and pipelined distributed shapes

(`16-jacobi` distributed, the producer–consumer and convolutional-network pipelines) — which still expand their nets and so carry the gate's linear replay cost; they are the heaviest in the corpus and grow with the worker count and firing volume. At these sizes the effect is small, and this work did not instrument per-stage timing to attribute it precisely. The practical reading is that for this class of program the entire compile, soundness gate included, is comfortably sub-second, and qualitatively a small fraction of the downstream Rust build of the emitted project (which is seconds, not milliseconds); the genericity is not bought at a compile-time cost that would matter in practice at these sizes.

Chapter 11

Discussion

The preceding chapters described what the system is and what the validation rig has established. This chapter steps back to weigh it. The central question is not whether the system works on its corpus — chapter 10 answered that — but whether the one design decision underneath everything was worth making, and at what price. That decision is a single trade, made once and propagated everywhere: the model gives up expressiveness in exchange for a statically determined, deterministic firing order. Section 11.1 examines the trade directly; section 11.2 sets out what it buys; section 11.3 sets out what it costs, in equal detail and without softening; and section 11.4 asks where the evidence of the previous two chapters could itself be misleading.

11.1 The central trade

Every restriction the language imposes — affine and constant-bounded loops, single-assignment arrays, mandatory purity annotations, no data-dependent control flow, no recursion, an explicit rather than discovered decomposition — is a facet of one underlying commitment: that the order in which the program's operations fire is fixed at compile time rather than at run time. A conventional concurrent program decides, dynamically, which of several ready operations runs next, according to which inputs happen to arrive first. This system forbids that. It derives one firing order from the dataflow and the source order, and that order is the program's behaviour on every backend and every run.

The cost of this commitment is expressiveness, and it is not a small cost. A whole class of useful programs cannot be written at all: anything whose control flow depends on the values it computes — an iterative solver that runs until it converges, a search that stops when it finds what it is looking for, a workload balanced dynamically across workers as data arrives. These are not exotic programs; they are the bread and butter of numerical computing and systems work, and the model excludes them by construction, not by omission.

What the commitment buys is the subject of the next section, but the

shape of the bargain can be stated now. A statically determined firing order is precisely the condition under which two otherwise intractable questions become cheap. The first is whether the concurrent program is sound — bounded and deadlock-free — which for an arbitrary concurrent system is a state-space search, and which here collapses to replaying one known order. The second is whether two backends compiled from the same algorithm and schedule must agree — which for nondeterministic programs is not even well-posed, since a nondeterministic program need not agree with itself, and which here becomes a sharp, falsifiable, byte-for-byte test. Neither the soundness gate of section 6.7 nor the differential rig of chapter 9 would be possible without the static firing order. They are not separate features bolted on; they are the two things the central restriction was traded for. The rest of this chapter attempts to judge whether the things bought are worth the expressiveness given up — a judgement that depends entirely on what one wants to build, stated in terms concrete enough for a reader to apply to their own case.

11.2 Advantages

Portability for the algorithm class, demonstrated rather than asserted. Within the restricted class, the separation delivers on its promise: one algorithm file, unchanged, is decomposed across shared-memory threads, message-passing processes, distributed-memory ranks, and bare-metal microcontrollers by editing only the schedule. This is the property that motivated the work, and it is not merely claimed — it is the thing the differential matrix tests on every build. Twenty-nine worked examples, each written once, are driven across the ten backends on three tiers; every required cell a runtime can execute agrees byte-for-byte, and the documented exceptions — cells a backend cannot run for a stated capability reason — are recorded as skips rather than hidden. One construct is realized but not yet universal: the bounded `for..until` convergence loop runs byte-identically across all seven tier-1 backends on a single-worker schedule, while its *distributed* (multi-worker) form, whose early exit must become a collective decision, is named future work rather than silently passed. The arithmetic was written once and never rewritten for a target; the expensive, error-prone part — the decomposition and the IO — became a short, swappable description.

IO semantics swappable per target without touching the algorithm. The contribution that distinguishes this work from the compute-scheduling languages it borrows from is that IO is a first-class schedule concern, not a property of a fixed runtime. Synchronous or asynchronous, buffered or unbuffered, event-driven or polled, over shared memory or sockets or DMA: each is a directive the schedule sets and the compiler resolves against the chosen backend's declared capabilities. The same algorithm can be given a

blocking, unbuffered transport on one target and an asynchronous, double-buffered, event-notified one on another, and the change is local to the schedule. Hand-written parallel code couples these decisions to the source; here they are dials.

Compile-time soundness in place of runtime surprise. Boundedness and deadlock-freedom are decided before any code is emitted, and a violation is a compile error that names the offending place or stalled transition rather than a hang discovered under load. For the targets this work most cares about — embedded systems with no operating system to absorb a buffer overrun and no debugger attached in the field — moving these failures from run time to compile time is not a convenience but a change in kind. A class of bugs that would otherwise be found late, intermittently, and expensively is instead found at the desk, every build, deterministically.

Falsifiable genericity. The separation claim is unusual among software-architecture claims in being mechanically refutable. "The algorithm is independent of schedule and backend" is not a matter of taste or of code review; it is a prediction that a specific test will stay green, and any leak that makes two backends diverge — which a leak across the algorithm/schedule or middle-end/presentation boundary will, unless it happens to corrupt every backend identically — turns it red by naming a backend that disagrees. The one case the rig cannot catch by construction is a common-mode leak in the shared middle-end that corrupts all backends alike; it is the residual risk taken up in section 11.4. The negative falsifiers of section 9.4 close the loop by proving the test can go red, so a green result carries information. This is a higher evidential standard than separation claims usually meet, and it is a direct consequence of the determinism the central trade bought.

A genuinely thin backend boundary. The event contract is narrow enough that most of a new backend is shared code. Adding a tier-1 backend is days to weeks, not a rewrite, because the event-list walker, the renderers, the host-election rule, and the check-loop templates are reused, and only the transport-specific calls differ (section 7.5). The low marginal cost of a backend is itself evidence that the middle-end/presentation boundary is real: if target-specific knowledge had leaked above the contract, a new backend could not be so thin.

11.3 Honest shortcomings

The advantages are real, but so are the costs, and an honest accounting gives them equal weight. None of the following is a defect to be fixed by more

engineering on the same design; each is a consequence of the central trade, and several are permanent given that trade.

The restricted algorithm class is narrow. The headline limitation is the class of programs admitted at all. Loops must be affine and constant-bounded; arrays are single-assignment with no aliasing; arithmetic is integer-valued, or floating-point only under the reduction-determinism restrictions discussed below (order-independent combines, or the opt-in fixed-order `fsum`); data-dependent control flow is admitted only in a bounded single-worker form (a `for..until` early exit under a compile-time iteration cap, section 5.5), and there is no recursion, no higher-order kernels, and no dynamic scheduling or work-stealing. This excludes multi-worker convergence-driven iterative solvers, recursive divide-and-conquer, irregular and pointer-chasing computations, and any workload whose balance must be decided at run time. The corpus reflects the boundary honestly: the two examples written to express data-dependent loop termination compile and run on the single-worker path — one converging early, one running the full cap so the worst-case bound is itself exercised — byte-identically across all seven tier-1 backends, while their *distributed* (multi-worker) form, whose early exit must become a collective decision, is named future work rather than papered over. The class is large enough to contain stencils, dense linear algebra, separable filters, histograms, sorting networks, small neural-network inference, and a real multi-core signal pipeline — but it is a class, and a reader whose problem lies outside it is not served by this system.

The schedule is greedy and non-optimal. The compiler is faithful to the schedule it is given; it does not search for a better one. There is no cost model, no autotuner, and no attempt to choose a block size, a partition, or a pipeline depth automatically. A schedule that decomposes badly compiles into a correct program that runs badly, and the system will not warn that a different decomposition would be faster. This is a deliberate scoping decision — the contribution is the separation and its soundness, not performance — but it means the system delivers portability and correctness, not speed, and the burden of finding a good schedule rests entirely on the programmer.

Soundness is scoped to coordination, not value-correctness, and to one net class. Two distinct boundaries on the soundness claim must both be kept in view. First, the gate proves that the concurrent coordination is sound — that buffers do not overflow and the program does not deadlock — and says nothing about whether the arithmetic is correct. A kernel that computes the wrong function passes the gate as readily as a correct one; value-correctness is established only by the differential rig against a reference output, and only over the exercised cells. The gate and the rig are complementary, and neither

subsumes the other. Second, even the coordination guarantee is bounded to the statically-ordered net class the language produces. The gate is an exact replay over one firing order, sound for that class precisely because those nets have no nondeterministic token routing; it is not a general boundedness or reachability decision procedure for arbitrary Petri nets, and it makes no claim outside its class. Both boundaries are stated wherever the gate is discussed because conflating "the coordination is sound" with "the program is correct," or "sound for these nets" with "sound for all nets," would overstate the result.

Bit-identity for floating point is conditional, and float *sum* is where the line is drawn. The byte-for-byte guarantee rests on the arithmetic being deterministic, and IEEE-754 addition is not associative: a distributed floating-point sum reduced in an order that varies across backends would produce different bytes. The system's default response is to refuse the operation — a plain floating-point `sum` combine is rejected at compile time — rather than emit code that happens to agree on the test input and diverges on another. Two routes cross the line without abandoning determinism. Order-independent combines — a float `min` or `max` — are admitted directly, because their result does not depend on the reduction order at all (subject to the caveat below). And float *sum* is admitted through an explicit opt-in, `combine=fsum`: the host folds the per-worker partials with a *fixed*, worker-id-sorted order that is identical on every backend (the same deterministic event-list order the rest of the schedule already uses), and the immutable reference oracle reproduces that exact fold, so all seven CPU backends agree to the byte and agree with the oracle (demonstrated by the `28-bin-fsum` example). The honesty is in the residual, which is larger here than for `min`/`max`: `fsum` is reproducible *across backends for a given schedule*, not order-independent. It is not the naive single-pass IEEE sum, and a different worker count folds the partials differently and can yield different bits — so it carries the "same bytes across backends" guarantee of the differential rig (section 10.2), not a "same bytes across schedules" one. The plain `sum` combine stays rejected, and a fully order-independent (schedule-invariant) reduction — the wide-superaccumulator or pre-rounding route — remains future work (section 12.2). The `min`/`max` caveat still applies: they are order-independent only for distinct, finite, non-NaN values, because IEEE-754 treats -0.0 and $+0.0$ as equal and ignores NaN, so a bin mixing signed zeros or containing only NaNs is not guaranteed bit-stable under reordering (the committed fixtures are chosen NaN-free with distinct finite values).

Check assertions are descriptive, not prescriptive. The schedule can attach a timing assertion — a latency bound on a loop, say — and the compiler will inject code that measures and reports whether the hand-written schedule meets it. But it is a thermometer, not a thermostat. There is no cost model

behind it and no closed loop: the compiler does not adjust the schedule to satisfy the bound, and cannot, because it has no model of what any schedule would cost on any target. A reader expecting the assertion to *drive* scheduling will be disappointed; it only observes.

Conservative transfer inference: the wire is narrowed, the footprint is not. The compiler infers, for each worker, the region of a partitioned array that worker will actually read: when a partitioned index is affine and contiguous, that region is a precise tiled slice; when it is not — a sparse, multi-variable, or data-dependent index — the inference conservatively widens to the whole array, always a correct superset. That inferred region now governs the wire. For a cross-worker edge whose inferred region is a single contiguous span, the producer transmits only that span and the receiver pastes it from offset zero, on every tier-1 backend and under MPI; the send-buffer sizings and the MPI buffered-send budget shrink in lock-step, all derived from the one shared shape descriptor so a sender slice and a receiver paste cannot disagree by construction. The reduction is real and measured: on the four-worker stencil it cuts cross-worker traffic by 71.9 percent, and on the row-partitioned matrix multiply by 50 percent (section 10.6). This is the saving the inference was always the groundwork for, now taken for the common contiguous case — and it touches neither the firing order, the soundness gate (which counts messages, not bytes), nor the byte-level results, exactly as predicted.

Two residuals keep this honest, and both are deliberate rather than overlooked. First, the narrowing is confined to contiguous-span edges: a strided per-worker region (a column band of a row-major matrix), a halo strip that is not a clean prefix, an accumulator fan-in, and any conservatively-widened data-dependent gather or scatter all stay whole-array on the wire, because narrowing them safely needs a banded pack-and-scatter the backends do not yet emit; the embedded tier likewise stays whole-array, since its receiver fills a fixed-shape buffer with no slice-paste and a banded send without a banded receive would corrupt. Second, and more fundamental, only the *wire* shrinks, not the *footprint*: every worker still allocates the array full-shaped and pastes its received span into place, so per-worker memory is unchanged. Reducing the allocation to the owned band needs either index rebasing at every kernel call or a banded-view abstraction — a deeper change than the wire flip, because downstream reads index with global coordinates — and it remains open. So the over-communication floor is no longer absolute, but it is lowered for one class of edge and along one axis only: contiguous spans move less data; everything else, and all per-worker allocation, still pays the whole-array cost.

The inference's two data-dependent forms differ in shape, which is why one stays whole-array on the wire and the other is tied to the reduction algebra: a gather widens the whole indexed input and needs no combine, whereas a scatter replicates the whole target accumulator across workers and recombines

the partials under the declared combine operator. A scatter whose target would have to be split along an affine partition axis is rejected outright rather than mis-compiled. The boundary is drawn for safety, not for communication efficiency.

Shim sprawl is real and, in principle, unbounded. The embedded tier's generic backend is written once, but every microcontroller family needs its own shim supplying concrete DMA controllers, interrupt-vector tables, memory layout, linker scripts, and real-time validation under `no_std` constraints. The *first* family is the most expensive class of backend work — months, because the first shim builds much of the generic embedded machinery alongside itself. The system now ships *two*: the original STM32H7 reference shim and a second nRF52840 shim, byte-exact on the same examples (chapter 10). The second is the load-bearing one for this claim, because its UARTE EasyDMA transmit — a pointer-and-length DMA engine — is a genuinely different register interface from the STM32 USART, so reaching it through the unchanged hardware-abstraction trait shows the trait is a real portability seam, not a single-target abstraction. The byte-exact evidence exercises the trait's value-transport methods — allocation, transmit, and the run loop — on both families; the trait's timing and diagnostic hooks are structurally mirrored from the first family but not separately exercised on the second, because the examples used carry no timing assertion, so that narrower sub-surface is verified on one family only. The second family also calibrates the cost honestly: the second family was days, not months, because it reused the generic backend, the lowered run loop, and the input-injection and capture harness unchanged, swapping only the concrete register sequence, the memory map, and one linker flag — but only because it happened to share the Cortex-M target the toolchain already builds and to have a ready simulator model. A family lacking either would cost more. So this is an honest containment of the cost, not an elimination of it: the trait is now demonstrated across two families rather than asserted from one, but shim sprawl remains in principle unbounded, and hardware portability beyond the two demonstrated families is still future work that someone must do, family by family.

The cheap tier carries the formal claim. The full bit-identical matrix runs only on the seven tier-1 CPU backends, because they are the only ones cheap enough to build and run on every continuous-integration invocation. The cluster and embedded tiers run under a compile-mandatory, run-best-effort discipline: every backend must compile for every required schedule, but the cluster tier is executed only where a local message-passing launcher is present and the embedded tier only under a simulator gate that the default development shell does not carry. The practical consequence is that the strongest, most-exercised evidence comes from the commodity-CPU tier, while the distributed

and embedded tiers are confirmed more narrowly. The distributed tier is exercised as a byte-exact differential against the same frozen reference the cheap tier uses, but on a local launcher rather than a production cluster: fourteen MPI cells spanning twelve examples — ten schedules under the blocking backend (`mpi-blocking`) and four asynchronous schedules under the non-blocking one — each launched under `mpiexec` and byte-diffed against the committed reference, on one oversubscribed host rather than across many nodes. The blocking arm in particular now spans a row-partitioned matrix multiply, a sparse matrix–vector product, a data-dependent scatter, a transpose, a separable filter, and an iterative stencil, not just the original element-wise, split, and reduction shapes. The embedded tier is narrower still — byte-exact co-simulation of four examples (now including the asynchronous double-buffered DMA-ring schedule of section 7.4), on two microcontroller families (an STM32H7 and an nRF52840) rather than a broad embedded matrix. So the claim is strongest where the targets are least exotic, which is the reverse of where extra assurance would be most valuable; loopback MPI is not a cluster and a co-simulated microcontroller is not silicon, and both remain genuine residuals.

11.4 Threats to validity

Beyond the design limits, it is worth asking where the evidence itself could mislead — where a green result might mean less than it appears to.

The specification is the corpus, so coverage bounds the claim. The test suite is the specification: a capability exists only when a committed example exercises it. The strength of this is that the specification cannot drift from the implementation; the weakness is that the guarantee is exactly as complete as the corpus, and a program shape absent from the corpus is outside the guarantee entirely. Bit-identity is shown over the exercised cells, not proved for every program the grammar admits. The corpus was assembled to be diverse, and the known gaps are recorded rather than hidden, but "no counterexample in twenty-nine examples" is empirical evidence, not a proof, and should be read as such. The natural way to push the claim from curated witnesses toward the boundary is generative: a property-based harness that synthesises affine, single-assignment programs and checks cross-backend byte-identity over them attacks the separation harder than a fixed corpus can. Such a harness now exists for a structured subclass (section 9.2): it synthesises programs in five families — element-wise pipelines, a two-dimensional stencil with halo inference, a binned reduction over all six combine operators, a multi-compute-worker partition, and a bounded `for..until` loop — each decomposed across a real worker boundary, and checks that the tier-1 backends required for the family and an in-process reference agree byte-for-byte; across the seeds run to date it has produced no counterexample. This narrows the gap

but does not close it, and two things must be said plainly about how far. First, the synthesised subclass is still smaller than the grammar the system admits — the generator emits no floating-point, recursive, or unbounded data-dependent programs, and the `for..until` family is checked by the *generator* on one backend only (a not-yet-lifted limit of the generator's per-family backend set, not of the construct: the curated matrix already exercises the same single-worker `for..until` machinery seven-way and byte-identically) — so the boundary of the empirical claim has moved outward, with the widening from one family to five, but not vanished, and every shape the generator cannot yet reach remains covered only by the curated corpus. Second, the result is still empirical and still intent-bounded: "no counterexample over the sampled programs" is stronger than "no counterexample in twenty-nine examples" but is not a proof, and the generator's in-process reference shares the arithmetic intent of the kernels it emits — indeed it is a second transcription of the same operator definition, so *less* independent than the separately-authored curated oracle, not more. At the level of author intent the bound is the same as the curated case: it guards against a compiler fault that mistranslates a kernel, not against an author-level error in the intent itself. Extending the generator further — to floating-point shapes, and widening the generator's `for..until` family from its single backend to the seven-way single-worker set the curated matrix already covers (a generator-side change, since that construct's single-worker cross-backend path has landed) — remains among the most direct pieces of validation work for the correctness claim, a more direct one than the performance study of section 12.5. The deeper remaining lift, the *distributed* multi-worker `for..until` with a collective early exit, is a backend-capability gap (section 12.3) rather than a generator one.

Agreement could be common-mode rather than genuine. The differential rig infers mutual byte-identity transitively, through a shared immutable reference file. This is sound only if a defect could not corrupt all backends and the reference identically. The design defends against this on two fronts: the reference is produced once by an independent implementation and then frozen, so it cannot drift in lockstep with a later compiler bug; and the seven backends are chosen to be maximally diverse in transport and notification discipline, so an error would have to produce the same wrong bytes on a spinning poll loop, an event reactor, and a barrier-synchronised thread pool to escape. A common-mode fault in the shared middle-end — above the point where the backends diverge — is the residual risk the rig cannot catch by construction, since every backend inherits the same middle-end. The soundness gate offers only partial protection here, being itself middle-end code that such a fault could equally corrupt; the genuinely orthogonal defence is the independent, frozen reference of the previous paragraph, produced by a separate implementation that shares none of the compiler's middle-end. That code-level

independence is now a mechanically enforced invariant rather than a reviewer convention: a continuous-integration fence rejects any reference that declares a dependency on the compiler or a backend crate, that links into the compiler's build, or that includes a generated source file, so a reference cannot silently come to share the very middle-end it exists to check. That defence is real but bounded along two distinct axes. It catches a middle-end fault only where the fault changes the output bytes, not where it leaves them unchanged; and it removes only *code*-level common mode — a shared implementation — not *author-intent* common mode, since the reference still re-encodes the same arithmetic specification (the same operator definitions, the same input and output byte layout), so a misreading of that specification would corrupt the reference and the algorithm alike. The fence makes the first axis mechanical; the second is irreducible, reduced only where a reference encodes the algorithm a structurally different way — the sparse matrix–vector reference indexes the input vector directly where the compiled program reaches the same values through a masked accumulator, so the two cannot be wrong in the same way by construction — and otherwise left as a documented residual.

The negative falsifiers could themselves rot. A negative test that silently becomes a no-op is permanently and invisibly green, and would leave a check unguarded while appearing to guard it. The apparatus addresses this directly: each falsifier reports a signal proving its injection actually perturbed something — a count of perturbed cells, a divergence pinned to exactly the corrupted backend (section 9.4) — and fails if that signal is absent. This is one further level of meta-testing, and it reduces but does not abolish the regress; at some level a test apparatus must be trusted.

Toolchain confounds are pinned out, but the pin is itself an assumption. A disagreement between two backends built with two different compiler versions would be an artefact of the toolchain, not evidence of a boundary leak. The Nix flake pins the entire environment so that this confound cannot arise, and the single pinned toolchain is also the sole source of truth for the minimum supported compiler version. The residual assumption is that the pinned environment is itself faithfully reproduced, which content-addressing makes very strong but not absolute.

The honest summary of this chapter is that the central trade is favourable for exactly the audience the work targets — people moving a regular, deterministic algorithm across heterogeneous parallel and embedded targets, who value correctness and portability over peak performance and can live within an affine, static, single-assignment class — and unfavourable for everyone else. The next chapter takes the most pressing of these limits and asks which could be lifted, and at what cost to the guarantees that make the rest of the system worth having.

Chapter 12

Future Work and Open Problems

The limitations weighed in chapter 11 are not all equal: some are permanent consequences of the central trade, while others are boundaries the current system draws conservatively and could move with more work. This chapter is concerned with the second kind. Each direction below is grounded in a limit the system actually has, and each is described with an honest estimate of what it would take and — crucially — what it would risk, because several of these extensions push directly against the static-determinism property that the soundness gate and the differential rig depend on. A direction that quietly sacrifices that property would not be an extension of this work but a different system; where that danger exists, it is named.

12.1 Prescriptive scheduling

The schedule today is hand-written and the compiler is faithful to it; the `check` assertions observe whether a bound is met but do nothing to meet it (section 5.3). The natural next step is to close that loop: turn a checkable latency assertion into a solved latency budget, so that a schedule states a goal — a per-loop latency ceiling, a memory footprint, a worker count — and the compiler searches the schedule space for a decomposition that satisfies it.

This is the direction in which the compute-scheduling languages have invested most heavily, through cost models and autotuners, and the same machinery is applicable here. But it is genuinely hard, and it changes the contract: a solver needs a cost model of each target, and a cost model is exactly the thing this work deliberately does not have, because a faithful one is target-specific and a wrong one would silently choose bad schedules. A credible version would likely begin narrowly — solving for one parameter, such as block size or pipeline depth, against a measured rather than modelled cost on one target — and treat full multi-objective schedule synthesis as a long-term goal

rather than a near-term feature. The value of the present architecture for this work is that the search space is already explicit and the soundness gate already rejects unsound candidates, so a solver could explore schedules knowing that every accepted one is sound by construction.

12.2 Floating-point determinism

The bit-identity guarantee once stopped at floating-point summation, because IEEE-754 addition is not associative and a distributed float-sum reduced in a varying order produces varying bytes (section 11.3). That boundary has now been crossed for the cross-backend case, but only partway, and the remaining distance is worth stating precisely.

What is implemented is the *fixed-order* route: an opt-in `combine=fsum` under which the host folds the per-worker partials in a fixed worker-id-sorted order that is identical on every backend, and the reference oracle reproduces that exact fold. The reduction lowering, the kernel contract, and the reference implementation therefore move in lockstep — the subtlety anticipated below, made concrete — and a distributed float sum now carries the same cross-backend byte-identity the integer combines enjoy, demonstrated across all seven CPU backends by the `28-bin-fsum` example. The plain `sum` combine stays rejected, so the guarantee is opt-in and the default stays strict.

What remains open is the stronger, *order-independent* route: a reproducible reduction that computes a fixed result regardless of summation order, for instance by accumulating into a wide fixed-point superaccumulator — an integer accumulator wide enough to represent any partial sum exactly — that absorbs each addend without rounding, or by pre-rounding addends to a common granularity [7]. The fixed-order fold delivered here is reproducible across backends for a *given* schedule, but it is not the naive left-to-right IEEE-754 sum, and a different worker count folds the partials differently — so it does not yet give "same bytes across schedules." An order-independent scheme would close that gap, making every decomposition agree bit-for-bit on a single reproducible result independent of worker count (the result being the scheme's own — the exact sum for a superaccumulator, the pre-rounded sum for pre-rounding — not necessarily the naive single-pass IEEE sum). The subtlety that made even the fixed-order step less self-contained than it first appears — that the immutable oracle, the rig's trust anchor, must adopt the same scheme so cross-backend agreement is also agreement with the oracle — applies with full force to the order-independent route, and is the reason it is the higher-value but heavier remaining direction: it lifts the schedule-invariance limit that still excludes part of a whole numerical domain, but it is not the trivial drop-in the opt-in framing alone would suggest.

12.3 Beyond the affine, static class

The hardest and most consequential limits are the ones that define the algorithm class itself, and they come in several grades of difficulty.

Distributed gather and scatter. Data-dependent indexing is supported today only conservatively: a gather broadcasts the whole indexed array and a scatter replicates and recombines its target, and a scatter onto an affine-partitioned target is rejected (section 6.5). A precise distributed gather/scatter — one that moves only the elements a worker actually touches — would require a communication-synthesis pass that reasons about the index array's contents, which are not known at compile time, and so would need either a runtime exchange phase or a two-pass inspector/executor scheme — a first pass that inspects the index array to plan the communication and a second that executes that plan — of the kind sparse libraries use [21]. This is well-trodden ground in distributed sparse computing, but importing it here means admitting a runtime-determined communication pattern, which sits in tension with the static firing order. The difficulty is sharper than the bounded-iteration case treated below in this section. Bounded data-dependent *iteration* (the capped `until` loop exercised by `21-jacobi-converge` and `29-jacobi-cap-hit`) works because its only runtime unknown is a scalar trip count: a compile-time cap fixes the worst-case firing order and any shorter run replays as a sub-trace of it. A precise gather/scatter's runtime unknown is not a scalar but an entire address map — which element moves to which worker — whereas the soundness gate replays a communication structure fixed at compile time. The only compile-time-fixed structure that covers every possible index pattern is the conservative whole-array broadcast the backends already emit; that broadcast is, in effect, the trivial statically sound envelope, merely an untightened one. Tightening it is therefore not blocked by bounding the exchange's *size* — that bound is the part the iteration work already demonstrates — but by deciding *which* elements move while the gate still sees a fixed structure. Closing the gap needs either a gate that admits a bounded family of runtime communication patterns or a static envelope provably sound for any pattern yet tighter than whole-array; both are verification advances the present soundness argument does not reach, which is why the conservative broadcast remains the only sound option today.

Data-dependent control flow. Convergence-driven termination — a loop that runs "until converged" — is the single most-requested capability the model still lacks in general, and lifting it *fully* is the deepest change, because an unbounded data-dependent trip count means the firing order is no longer statically determined, which is the property everything else rests on. The bounded middle path, however, is no longer hypothetical. Bounded data-dependent iteration — a loop with a compile-time maximum trip count and a

runtime early exit, so the worst-case firing order remains statically analysable even though the actual run may stop early — is *realized on the single-worker path*. Two corpus examples (`21-jacobi-converge` and `29-jacobi-cap-hit`, the optional `until` clause of appendix B) compile and run a Jacobi convergence loop value-correctly: one exits early, and one runs the full cap so the worst-case bound the soundness gate analyses is itself an exercised, byte-identical execution path rather than an assertion. This holds because the gate replays the full capped unroll and any early exit is a sub-trace of it (soundness preserved by the cap), and because the exact-integer halt predicate is a deterministic, backend-agnostic function of the data, so every backend stops at the same generation (the differential test preserved). On this single-worker path the construct is no longer one-backend: both examples now run byte-identically across all seven tier-1 backends, because a host-only schedule delegates to the shared single-worker emitter on every backend (section 10.6). Two residuals remain. First, the break is single-worker only: on the multi-worker (distributed, `partition=workers`) path the convergence scalar is a cross-worker reduction, so the early exit must become a collective all-reduce-and-broadcast at a barrier — every worker branching on the identical global value and exiting at the same generation by construction — which keeps the firing order static but is a genuinely new collective pattern not yet built. This distributed form is named future work rather than recorded as a skipped cell: it cannot become an executable cell until the distributed Jacobi base it builds on lands, so the multi-worker break-condition path stays fail-loud-rejected in the meantime rather than silently passed. Second, byte-identity is at risk under a *floating-point* convergence predicate, whose comparison value depends on accumulation order, tying this direction back to the float-determinism limit above. Even so, the realized form already admits a useful subset of single-worker iterative solvers — those with a known iteration cap and an exact termination test — and the residual is a bounded, well-scoped lift rather than the open-ended problem unbounded data-dependent control flow poses.

Sparse iterative solvers. A full sparse solver needs both of the above — data-dependent indexing and convergence-driven termination — which is why it sits entirely outside the present model. It is listed here not as a near-term target but as the natural destination if the two preceding directions both succeed, and as an honest marker of how far the current class is from general numerical computing.

Precise inference, and the remaining halves of the region saving. The groundwork here is now partly built, which sharpens what is left to do. The backends already transmit only the inferred region on a cross-worker edge whose region is a single contiguous span, cutting wire volume measurably (section 10.6); two distinct extensions remain, one on the inference side and

one on the backend side, and they are independent. On the inference side, the transfer-inference pass still degrades to whole-array broadcast whenever an index is not an affine, contiguous function of the partition variables. A polyhedral analysis, using an established integer-set library, could compute precise transferred regions for a broader class of affine and piecewise-affine index expressions than the current prefix-based inference handles, without changing the language — and because the wire now follows the inferred region, a sharper inference would directly narrow more edges rather than merely improving receiver-side correctness. On the backend side, two cases the wire flip deliberately left whole-array remain: a *strided* or non-prefix region (a column band of a row-major matrix, an irregular halo) needs a banded pack-and-scatter the backends do not yet emit, and the embedded tier needs a banded receive to match a banded send. And the *footprint* half is wholly untouched: every worker still allocates the array full-shaped and pastes its received span into place, so per-worker memory is unchanged; reducing the allocation to the owned band needs index rebasing at kernel-call sites or a banded-view abstraction, because downstream reads index with global coordinates (section 11.3). All of these are internal precision improvements that risk nothing in the guarantees — the firing order, the soundness gate, and the byte-level results are unchanged, only the volume moved and the memory held — which keeps the direction comparatively safe, but it is a several-part one, of which only the contiguous-span wire flip is so far done.

12.4 New target tiers

The three tiers — commodity CPU, message-passing cluster, embedded microcontroller — cover a wide span but stop short of massively parallel accelerators. A GPU or neural-processing-unit tier is a plausible fourth, and the event contract is in principle agnostic enough to drive one. But the fit is not free: a GPU's execution model is bulk-synchronous (alternating compute phases with global synchronisations across many parallel threads) over thousands of lanes, with a memory hierarchy the current contract does not model, and the `Alloc` and `Free` events (section 6.6), currently reserved in the event contract but emitted by no backend, would have to become load-bearing to place data across that hierarchy. The honest assessment is that a GPU tier is a research project in its own right, not a matter of writing one more shim, and that claiming the architecture "extends to GPUs" without having done it would be exactly the kind of overclaim this document has tried to avoid. It is listed as an open direction, not a promised one.

The cheaper and more certain tier work is further breadth within the existing embedded tier. A second microcontroller family has already been added — an nRF52840 (Cortex-M4F) shim whose UARTE EasyDMA transmit is a genuinely different register interface from the STM32 USART — byte-exact

on the same examples (chapter 10), which both demonstrates the hardware-abstraction trait is a real portability seam and calibrates the cost: that second family reused the generic backend and the test harness unchanged, swapping only the concrete shim, the memory map, and one linker flag, because it shared the Cortex-M target the toolchain already builds and had a ready simulator model. Additional families — each a separate shim against the committed trait — would broaden the evidence further. This is bounded, well-specified engineering rather than research; cheap when a family shares the target and has a simulator model, more expensive when it needs a new target triple or a model that does not yet exist. It remains the most direct way to widen the part of the claim that section 11.3 identifies as the narrowest.

12.5 A quantitative performance and scaling study

This document confined its main results to correctness and coverage, because the contribution it defends is the separation and its falsifiability, not the speed of the emitted code (chapter 10). It does report a set of *static* engineering characteristics — per-backend code footprint, generated-code size, and how the soundness gate's analysis net scales with problem size (section 10.6) — so the cost of the genericity is at least described. Two quantitative directions remain genuinely open, and it is worth stating each precisely.

The first is a *runtime* performance and scaling study: how the emitted programs' wall-clock time varies with worker count, schedule, and backend on real hardware — and, for the cluster tier, on an actual cluster rather than a local launcher. This is the measurement that would turn the portability claim from "the same values everywhere" into "the same values, and here is what each decomposition costs," and it is the input the prescriptive-scheduling direction of section 12.1 would need. It is deliberately deferred here because a credible runtime study demands a benchmarking methodology — worst-case inputs, controlled hardware, warm-up and variance discipline — that is a project in itself, and because no runtime number bears on the correctness claims already established: a slow but byte-identical program still falsifies nothing.

A scoping observation explains why this was, until recently, more than a matter of securing time on a cluster, and it once coupled the runtime study tightly to the gate-scaling direction below. At the corpus's problem sizes the emitted programs' wall-clock is dominated by fixed per-invocation overhead rather than by computation — a single-worker matrix multiply at the corpus dimension finishes in about a millisecond, of which the few thousand integer multiply–adds account for only microseconds; the remainder is process spawn, input loading, and thread setup, not arithmetic — so a meaningful scaling signal emerges only at substantially larger problem sizes. The gate is not separable from the build: there is no option to emit a decomposed program without analysing it, so the cost of compiling the decompositions one would

want to time used to bound what could be timed at all. That coupling has now loosened for the dominant case. The gate proves the single-shot matched-pair communicating subclass — which is the shape of most of the corpus's distributed schedules — symbolically, so building the four-worker distributed matrix multiply at $N = 512$ now costs about 7.2 megabytes of compiler memory rather than the hundred-plus gigabytes its expanded net would project (section 10.6). One can therefore now compile exactly those large distributed decompositions, which removes the gate as the binding obstacle to timing them. What remains coupled is narrower: the loop-carried and pipelined distributed shapes still expand, so a runtime study that needed to scale *those* specific schedules would still wait on the gate work below. A credible runtime study therefore now waits primarily on a representative cluster and a disciplined methodology rather than on the compiler — except for the residual loop-carried and pipelined shapes, where the two open problems remain coupled.

The second direction is the gate-scaling limitation that section 10.6 measured, and it has been *substantially* closed, in two steps. Because the soundness gate replayed a net expanded over the iteration space — roughly one place and one transition per kernel firing — its cost was linear in the number of firings, cheap for the corpus but unable to scale to a problem with billions of firings. First, for the *buffer-free* subclass (a single worker, or any schedule with no cross-worker transfer) the gate now reasons about the *looped* structure directly: it proves soundness from the rolled ACFG in time independent of the iteration count, never building the expansion. Second, and the harder step, the symbolic proof now reaches across a cross-worker boundary, to the *single-shot matched-pair* communicating subclass: a distributed schedule whose every transfer is one `Push` matched by one `Wait` at loop depth zero, so the shared buffer place's peak occupancy is provably one and at most its capacity. This is the dominant distributed shape — the corpus's distributed reductions, stencils, matrix multiply, sparse mat–vec, histogram, and scatter all fall in it — and proving it symbolically, against a firing-order invariant pinned by property tests and cross-checked against the expanded replay on every gated cell, is what keeps the $N = 512$ distributed matmul compile flat. What remains open is the *loop-carried* and *pipelined* communicating shapes — a transfer whose buffer occupancy depends on cross-iteration drainage, or one that deliberately keeps several frames in flight — whose boundedness still depends on a firing-order interleaving the symbolic analysis does not yet cover. A steady-state occupancy argument over their looped `Push`/`Wait` structure is the concrete compiler work still outstanding. It is deliberately not approximated: an imprecise buffer analysis could accept a net the expanded replay would reject, trading soundness for scaling, which the gate's whole purpose forbids — so anything outside the proven subclasses is routed loudly to the sound expanded replay until the symbolic argument is extended to it too.

Taken together, these directions share a common discipline: each is judged

not only by what it would add but by what it would risk to the determinism the system is built on. Two of them sit off that risk axis entirely — prescriptive scheduling and the quantitative study change no guarantee, the first being an optimisation question and the second an empirical one. Of the rest, the order-independent float reduction and the polyhedral precision improvement add capability without touching the guarantees, even though the polyhedral work is grouped, by topic, inside the affine-class section; bounded data-dependent iteration and a precise distributed scatter push against the static firing order and must be done carefully if at all; and unbounded data-dependent control flow and a GPU tier would, if done naively, dismantle the very properties that make the present system worth having. The interesting future work is therefore not simply to lift the limits, but to lift them while keeping the soundness gate sound and the differential test meaningful — which is the same constraint that shaped the original design.

Chapter 13

Conclusion

This document began with a narrow, concrete complaint: that porting a parallel algorithm across CPU threads, a message-passing cluster, and an embedded microcontroller means rewriting it, because the decomposition and the IO — not the arithmetic — are written by hand and tangled into the source, separately for each target. Compute scheduling has long had first-class language support; IO scheduling has not. The work set out to give it that support and to make the resulting separation claim something stronger than a design aspiration.

The contribution is best stated soberly. The system rests on a strict separation between an algorithm sublanguage, which says what to compute and contains no worker, buffer, transport, or synchronisation, and a schedule sublanguage, in which decomposition, IO semantics, and target are first-class directives the algorithm never sees. The two are unified by a single Petri-net intermediate representation that carries transfer synthesis, synchronisation, and a compile-time soundness gate together, so that boundedness and deadlock-freedom are reported as compile errors rather than discovered as runtime hangs. And the genericity claim is made mechanically falsifiable by a cross-backend differential test: the same algorithm, under every schedule and capable backend, must produce byte-identical output, and any leak across the algorithm/schedule or middle-end/presentation boundary that makes two backends diverge turns that test red. The implementation drives twenty-nine worked examples through ten backends across three tiers — seven commodity-CPU backends, two message-passing cluster backends, and one embedded backend with multi-microcontroller co-simulation.

What was demonstrated is correspondingly bounded. The seven CPU backends agree byte-for-byte across the exercised schedule-by-backend matrix; the soundness gate accepts every sound schedule in the corpus and is shown, by adversarial tests, able to reject unsound nets; every check that underwrites these claims has a negative test proving it can fail, so a green result is informative rather than vacuous; and the cluster and embedded tiers are confirmed to value-

correctness on a local message-passing launcher and to byte-exact co-simulation respectively. None of this is a proof that every program the grammar admits is byte-identical everywhere; it is the result of a falsification rig that has had every opportunity over a broad and diverse corpus to refute the central claim and has not, with its documented edges marking exactly where the claim has not yet been tested.

The work is equally clear about its price. The whole architecture is bought with a single trade — expressiveness for a statically determined, deterministic firing order — and that trade defines a real boundary. The algorithm class is affine, static, single-assignment, and integer-valued at its core — unordered floating-point summation is excluded by the determinism it would break, and the opt-in fixed-order variant is reproducible across backends but not across schedules; there is no data-dependent control flow, no recursion, and no dynamic scheduling; the schedule is checked, not optimised, and carries no cost model; the soundness guarantee is scoped to coordination within a restricted net class, not to value-correctness or to arbitrary nets; and the cheap CPU tier carries the strongest form of the formal claim while the more exotic tiers are confirmed more narrowly. These are consequences of the central commitment, not oversights, and the document has tried throughout to weigh them with the same seriousness as the advantages.

The bet the work makes is that for a particular and not uncommon audience — people moving regular, deterministic algorithms across heterogeneous parallel and embedded targets, who value portability and provable coordination over peak performance and can live within the affine, static class — a system that makes IO a first-class schedule concern, decides soundness at compile time, and renders its genericity claim falsifiable is worth more than a more expressive system whose portability is asserted rather than tested. Whether that bet generalises beyond the demonstrated class is the substance of the future work — including order-independent floating-point reduction, bounded data-dependent iteration, precise distributed communication, further hardware families, prescriptive scheduling, and the deferred runtime performance study — each of which would widen the audience or sharpen the evidence. The discipline that shaped the original design — lift a limit only while keeping the soundness gate sound and the differential test meaningful — remains the constraint under which that widening must happen.

The smallest honest summary is this. The expensive part of portable parallel software is not the arithmetic; it is the decomposition and the IO. This work factored those out into a separate, swappable schedule, unified them in one analysable representation, and then — rather than asking to be believed — built the instrument that would expose the separation if it leaked, and ran it on every build. Across the corpus, it did not leak. That is a modest claim, precisely scoped, and it is the one the document stands behind.

Appendix A

Example Catalogue

The corpus of worked examples is the system's specification (section 9.1). This appendix catalogues the twenty-nine examples and, for each, names what it computes and the scheduling or compiler property it is there to stress. Most examples carry more than one schedule — typically a single-worker reference schedule and one or more distributed or transformed schedules that must reproduce the reference output byte-for-byte — so the number of differential cells is several times the number of examples.

The examples are grouped by the concern they principally exercise, in six tables (tables A.1 to A.6). The grouping is expository only; many examples touch more than one concern, and the table notes the primary one.

A.1 Foundational dataflow and decomposition

A.2 Stencils, halos, and loop transforms

A.3 Reductions and the combine algebra

A.4 Data-dependent indexing: gather and scatter

A.5 Larger pipelines and networks

A.6 Embedded and boundary cases

The last three rows of table A.6 are the most informative for a reader probing the limits: `04-prefix-sum` shows how a pattern outside the surface grammar (a carried scan) is expressed within it as a kernel-level idiom, and the two Jacobi convergence examples show where the static-firing-order boundary is drawn — `21-jacobi-converge` exiting before the cap and `29-jacobi-cap-hit` running the full cap so the worst-case bound is itself exercised — and how the corpus

Example	Computes	Stresses
`01-elementwise-add`	$c_i = a_i + b_i$	Smoke test: single worker, no transfers; the minimal positive control.
`02-split-add`	The same sum, split across two workers	First cross-worker dataflow and the first inferred transfer pair; also a tier-3 multi-microcontroller fixture.
`23-dot-product`	$r = \sum_k a_k b_k$	Map-then-reduce composition over two inputs; an explicit intermediate product, since nested kernel calls are not written inline.
`24-outer-product`	$c_{ij} = a_i b_j$	Rank expansion (the dual of reduction): two one-dimensional reads under two distinct index variables, one write per cell, no accumulation.

Table A.1: Foundational examples.

Example	Computes	Stresses
`05-stencil`	A 3×3 box-blur over an image	Halo inference from kernel access offsets; blocking; row-band and two-dimensional partitions; the worked example of section 5.4.
`06-separable-filter`	A two-pass separable box filter	An intermediate buffer whose lifetime spans two passes on one worker; clamp-to-edge boundary handling.
`16-jacobi`	A fixed number of 4-tap Jacobi sweeps	A multi-iteration stencil with a cross-iteration read-after-write; the banded-slice copy the distributed form requires.
`10-wavefront`	An accumulated-cost recurrence over a grid	A kernel-level recurrence the algorithm surface sees only as a wrapping dataflow edge; order-independent combine operators.
`11-game-of-life`	Eight generations of a toroidal life rule	A cross-iteration stencil with a double-buffered, pipelined transfer between generations.

Table A.2: Stencil and loop-transform examples.

Example	Computes	Stresses
`03-reduction`	A sum reduction to a scalar	The distributed accumulator pattern: per-worker partials combined at a host under an associative, commutative operator.
`25-bin-parity`	Per-bin parity (count modulo two)	A bitwise-xor combine with a zero identity; an operator-form rather than method-form accumulator.
`26-bin-min`	Per-bin minimum over keyed values	A minimum combine whose identity is the type maximum, not zero; identity-aware accumulator initialisation, with an empty bin forcing the identity to show.
`27-bin-fmin`	Per-bin floating-point minimum	An order-independent *float* combine that is admitted directly (with its $+\infty$ identity), in contrast to a plain float *sum*, which is rejected for non-associativity.
`28-bin-fsum`	Per-bin reproducible floating-point sum	The opt-in `combine=fsum`: a distributed float *sum* made cross-backend bit-identical by a fixed worker-id-sorted fold (the reference oracle moving in lockstep), as distinct from plain `sum` (still rejected) and from a schedule-invariant order-independent reduction (future work).

Table A.3: Reduction examples spanning the supported combine operators.

records, rather than hides, a shape it supports on the single-worker path but not yet across all paths.

Example	Computes	Stresses
`08-histogram`	A binned count over an input stream	A data-dependent *write* (scatter); the replicate-and-recombine handling of a conservatively-distributed scatter target.
`19-histogram-unconstrained`	A histogram over an unconstrained input	A non-trivial bucketing kernel in index position, with an input whose post-bucket distribution is genuinely non-uniform.
`17-spmv`	A sparse matrix–vector product (dense-stored)	A data-dependent *read* (gather) in the right-hand side; conservative whole-array handling of the indexed dimension.
`18-multigather`	Two arrays crossing one channel	When both pushes are lifted to the same schedule point, their order must still match the producer's declaration order, caught by the strict-FIFO message-passing backend if reversed.
`20-index-cast-permute`	A reversal permutation	A pure kernel in array-subscript index position taking a bare iteration variable, forcing the lowering that coerces an iteration variable to an array-index type before codegen.

Table A.4: Data-dependent indexing examples.

Example	Computes	Stresses
`07-matmul`	A blocked integer matrix product	Two-dimensional blocking on both axes and the triple-nested accumulator.
`09-producer-consumer`	A two-stage producer–consumer pipeline	Buffered, asynchronous, event-notified streaming between two compute workers.
`12-bitonic-sort`	A bitonic sorting network	A static compare-exchange network checked byte-for-byte against a structurally different reference sorter.
`13-cnn-inference`	A small convolutional-network forward pass	One algorithm under three orthogonal decompositions — serial, batch-parallel, and pipeline-parallel.
`15-transpose`	A matrix transpose	The first example whose output axis disagrees with its input axis, exercising the compiler's handling of non-matching output–input axis order and the conservative whole-array transfer it falls back to.

Table A.5: Pipeline and network examples.

Example	Computes	Stresses
`14-hearing-aid`	A three-core hearing-aid signal pipeline	The load-bearing embedded demonstration: heterogeneous worker classes, named memory regions, bidirectional dataflow, and staged boot order, byte-exact under co-simulation.
`22-dma-pio-demo`	A fixed-point gain stage	Per-edge transport-mode hints contrasting a DMA-driven bulk transfer with a programmed-IO control transfer on the same fabric; tier-3 value-correctness.
`04-prefix-sum`	An inclusive prefix sum	A carried recurrence expressed as a multi-pass rectangular accumulator, since the surface forbids a carried scan directly.
`21-jacobi-converge`	A Jacobi sweep with convergence-driven termination	The deliberate boundary marker: a bounded `for..until` early exit that converges before the cap, compiling and running value-correctly and byte-identically across all seven tier-1 backends on the single-worker path; the distributed (multi-worker) form is future work.
`29-jacobi-cap-hit`	The same Jacobi sweep with the cap set below the convergence generation	The companion boundary case: the `until` predicate never fires inside the cap, so the loop runs the full worst case and extracts the last computed generation — the cap-bounded replay the soundness gate analyses, exercised end-to-end seven-way on the single-worker path; distributed is future work.

Table A.6: Embedded examples and the boundary cases that mark the edges of the supported class.

Appendix B

The Nuc Grammar

This appendix gives the grammar of the two Nuc sublanguages introduced by example in chapter 5. The notation is a lightweight EBNF: `|` separates alternatives, `[]` encloses an optional element, `{}*` a zero-or-more repetition, and terminals are set in `typewriter`. The grammar describes the surface syntax; semantic restrictions that the parser accepts but later passes reject — single assignment, affine bounds, mandatory placement, capability resolution — are stated as notes rather than encoded in the productions, since they are enforced by typed diagnostics downstream rather than by the grammar.

B.1 Lexical conventions

Comments run from `//` to end of line. Identifiers are the usual alphanumeric-with-underscore form. Integer and size literals may carry a unit suffix in the schedule's resource and timing positions (for example `64KB`, `10ms`, `1ns`). A type is a scalar element type — one of the fixed-width integer types, the pointer-width `usize`/`isize`, the two floating-point widths, or `bool` — optionally followed by one or more bracketed dimension extents, each a constant-folding expression over declared constants. The worked examples in this document exercise only `i32`, `f32`, and `usize`, but the grammar admits the full set below.

B.2 The algorithm sublanguage

An algorithm file declares constants, data arrays, kernel contracts, and a dataflow body of single-assignment writes and affine loops.

```
algo-file   ::= { item }*
item        ::= const-decl | data-decl | kernel-decl | stmt

const-decl  ::= "const" ident ":" type "=" expr ";"
data-decl   ::= "data" ident ":" type ";"
kernel-decl ::= "kernel" ident ":" "(" [ type-list ] ")" "->" type
```

```
                purity [ combine-spec ] ";"
purity      ::= "pure" | "effectful"
combine-spec::= "combine" "=" combine-op
combine-op  ::= "sum" | "or" | "xor" | "min" | "max" | "and"
type-list   ::= type { "," type }*

type        ::= scalar-type { "[" expr "]" }*
scalar-type ::= "i8" | "i16" | "i32" | "i64" | "isize"
              | "u8" | "u16" | "u32" | "u64" | "usize"
              | "f32" | "f64" | "bool"

stmt        ::= write-stmt | call-stmt | for-stmt
write-stmt  ::= lvalue "<--" kernel-call ";"
call-stmt   ::= kernel-call ";"
for-stmt    ::= "for" ident ":" expr ".." expr [ "until" rel-expr ]
                "{" { stmt }* "}"

kernel-call ::= ident "(" [ arg-list ] ")"
arg-list    ::= rvalue { "," rvalue }*
rvalue      ::= kernel-call | expr
lvalue      ::= ident { "[" expr "]" }*

rel-expr    ::= expr [ cmp-op expr ]
cmp-op      ::= "<=" | "<" | "==" | "!=" | ">" | ">="
expr        ::= mul-expr { ( "+" | "-" ) mul-expr }*
mul-expr    ::= unary-expr { ( "*" | "/" | "%" ) unary-expr }*
unary-expr  ::= [ "-" ] atom
atom        ::= literal | ident | kernel-call
              | ident { "[" expr "]" }* | "(" expr ")"
```

Notes. The expression grammar is stratified by precedence — additive over multiplicative over unary over atom — so it is unambiguous without a separate precedence declaration; index and loop-bound positions reuse the same `expr` nonterminal. Loop bounds and array-index expressions are affine in the enclosing iteration variables and declared constants, and fold to compile-time constants where a constant is required; a non-affine or non-constant bound is rejected by a later pass, not by the grammar. A comparison (`cmp-op`) is admitted only inside an `until` clause, never in an index, bound, or shape position, where it is rejected as a typed error. The optional `until` clause is the bounded early-exit surface of section 5.5: the `..`-bound remains the compile-time iteration cap that keeps the worst-case firing order statically analysable, while the `until` condition is a runtime halt predicate. Each array location is written at most once across the whole body (single assignment). A `write-stmt` whose kernel is `pure` is a value-producing dataflow edge; a `call-stmt` invokes an `effectful` kernel for its side effect (program IO) and preserves its order within the enclosing block. Kernel *bodies* are not part of this grammar: a `kernel-decl` is a contract only, and the body is an ordinary Rust function in a sibling file checked against the contract separately.

B.3 The schedule sublanguage

A schedule file binds an algorithm's kernels and loops to a worker topology and chooses IO semantics. It has a simple flat form, suitable for homogeneous CPU and cluster targets, and a richer typed form that additionally declares worker classes and memory regions for heterogeneous targets. The two share the placement, loop, transfer, and check directives.

```
sched-file  ::= "schedule" "for" string "{" { sched-item }* "}"
sched-item  ::= class-decl | region-decl | workers-decl
              | place-decl | place-data-decl
              | loop-decl | transfer-decl | check-decl

workers-decl ::= "workers" "=" "{" worker-entry { "," worker-entry }* "}" ";"
worker-entry ::= ident                      // flat form
               | ident ":" ident            // typed form: worker : class

place-decl   ::= "place" ident "on" placement ";"
placement    ::= ident | "{" ident { "," ident }* "}"

loop-decl    ::= "loop" ident ":" loop-opt { "," loop-opt }* ";"
loop-opt     ::= "partition" "=" ( "rows" | "blocks2d" | "workers" )
               | "block" "=" int
               | "reuse"
               | "pipeline" "=" int
               | "unroll" "=" int

transfer-decl ::= "transfer" ident ":" xfer-opt { "," xfer-opt }* ";"
xfer-opt      ::= "sync" | "async"
                | "buffer" "=" int
                | "notify" "=" ( "event" | "poll" )
                | "mode" "=" ( "dma" | "pio" )

check-decl    ::= "check" "loop" ident ":" check-prop
                  [ "," "on_violation" "=" viol-action ] ";"
check-prop    ::= "latency_max" "=" duration
viol-action   ::= "panic" | "count" | "log"
```

The typed form adds worker-class and memory-region declarations and a directive that binds data to a region:

```
class-decl   ::= "worker_class" ident "{" { class-attr }* "}" ";"
class-attr   ::= "simd" "=" simd-spec ";"
               | "memory" "=" mem-spec ";"
simd-spec    ::= "none" | ident                    // e.g. neon128
mem-spec     ::= region-ref { "+" region-ref }*
region-ref   ::= "shared" | ident [ "[" size "]" ]

region-decl  ::= "memory_region" ident "{" { region-attr }* "}" ";"
region-attr  ::= "size" "=" size ";"
               | "accessible_by" "=" "{" ident { "," ident }* "}" ";"
               | "per_worker" "=" bool ";"
```

```
place-data-decl ::= "place_data" ident "in" ident ";"
```

Notes. Every kernel the algorithm defines must have exactly one `place-decl`; an unplaced kernel, or a placement of a kernel the algorithm does not define, is a compile error (section 5.3). A `loop-decl` and a `transfer-decl` refer to a loop variable or data symbol by the name the algorithm gave it. The three partition kinds cut a distributed loop differently: `rows` into contiguous bands of the outer index, `blocks2d` into a two-dimensional grid, and `workers` into one contiguous band per worker over the loop range. The `unroll` option is parsed and validated (its divisibility against `block` is checked) but no transform pass yet consumes it, so the compiler *rejects* a schedule that uses it, with a diagnostic naming the option as accepted-but-unimplemented — a loud refusal rather than a silent no-op, consistent with the no-silent-downgrade rule below. The production is retained in the grammar for the planned consumer. A data symbol that crosses workers must carry a `transfer-decl`; omitting it is an error naming the symbol. Each transfer option is resolved against the chosen backend's capability declaration, so an `async` or buffered request on a backend that supports neither is rejected rather than silently downgraded. On a homogeneous backend the typed form collapses to the flat form: the worker classes carry a single default class and the memory regions a single default region, so the same algorithm and schedule extend from a thread pool to a heterogeneous system-on-chip without a second sublanguage. The `check` directive's three violation actions — `panic`, `count`, and `log` — are each exercised by a dedicated schedule in the corpus, so the surface the grammar admits here is one the example suite demonstrates rather than one merely declared.

Appendix C

Capability Matrix

Section 7.1 introduced the capability matrix and reproduced its transport, notification, asynchrony, and buffering columns for all ten backends (table 7.1). This appendix gives the matrix from the complementary direction — the one a schedule author needs — by recording, for each backend, the concurrency substrate it maps a worker onto, and then tabulating which schedule *demands* each backend accepts. Together these answer the practical question the matrix exists to answer: given a schedule, which backends can compile it, and which reject it and why.

C.1 Backends and their concurrency substrate

The capability columns of table 7.1 say what IO mechanisms a backend offers; table C.1 says what a worker physically becomes on each backend and over what substrate workers communicate. The two together are the full declaration each backend commits as reviewable text.

C.2 Schedule demands and backend acceptance

Orthogonality means backend choice is mediated only by the matrix: a schedule compiles against a backend exactly when the backend can satisfy what the schedule demands, and is rejected at compile time, with a message naming the missing capability, otherwise (section 7.1). The demands fall into two kinds, and the resolution rule differs between them. The `async` and `buffer` demands are *ordered*: asking for asynchrony, or for a buffer depth N, succeeds against any backend whose declared maximum meets or exceeds it, so this axis is a genuine superset relation. The notification discipline is instead a *partition*: a schedule's `notify` selects exactly one discipline, and a backend accepts it precisely when that discipline is in the backend's declared notify-set, rejecting the others. The disciplines do not form a lattice — `notify=poll` is not "less than" `notify=event` — so a backend offering only the event discipline rejects

Backend	Tier	Worker maps to	Substrate
`openmp-rs`	1	task on a work-stealing pool	shared memory
`pthreads-sync`	1	operating-system thread	shared memory
`pthreads-async`	1	operating-system thread	shared memory, ring buffers
`mp-tcp-bufsync`	1	operating-system process	TCP loopback socket
`mp-tcp-poll`	1	operating-system process	TCP loopback socket
`mp-tcp-event`	1	operating-system process	TCP loopback socket
`mp-uds-event`	1	operating-system process	Unix-domain socket
`mpi-blocking`	2	MPI rank	MPI
`mpi-nonblocking`	2	MPI rank	MPI
`embedded-pattern`	3	microcontroller core	UART fabric, DMA/PIO

Table C.1: The concurrency substrate of each backend: what one worker becomes, and how workers communicate. The two shared-memory synchronous backends, `openmp-rs` and `pthreads-sync`, are deliberately capability-identical but run on different substrates (a work-stealing pool versus raw threads), so a disagreement between just those two would implicate the substrate rather than the schedule.

a poll-notified schedule and vice versa. Table C.2 tabulates both kinds for the four IO demands a schedule can raise. A schedule that raises none of them — a single-worker schedule, or a multi-worker schedule using only synchronous, unbuffered, blocking transfers — compiles on every backend.

Backend	`async`	`buffer>1`	`notify=event`	`notify=poll`
`openmp-rs`	—	—	—	—
`pthreads-sync`	—	—	—	—
`pthreads-async`	yes	yes (to 64)	yes	—
`mp-tcp-bufsync`	—	—	—	—
`mp-tcp-poll`	—	—	—	yes
`mp-tcp-event`	yes	yes (to 64)	yes	—
`mp-uds-event`	yes	yes (to 64)	yes	—
`mpi-blocking`	—	—	—	—
`mpi-nonblocking`	yes	yes (to 64)	yes	—
`embedded-pattern`	—	—	—	—

Table C.2: Which schedule IO demand each backend accepts. "`async`" is an asynchronous transfer; "`buffer>1`" is an in-flight depth greater than one, up to the stated maximum; the two `notify` columns are the wakeup disciplines. A dash means the backend rejects a schedule raising that demand, at compile time. Every backend additionally accepts the synchronous, unbuffered, barrier-or-blocking baseline, which is why a schedule using only that baseline is universally portable. `mpi-nonblocking` is a strict superset of `mpi-blocking`.

Two consequences of this table are worth drawing out, as they govern how the differential matrix is populated. First, a schedule that demands an

asynchronous, event-notified, double-buffered transfer is accepted only by the four asynchronous backends (`pthreads-async`, `mp-tcp-event`, `mp-uds-event`, and `mpi-nonblocking`); the matrix records such a schedule's *required* column — the set of backends the harness must hold to byte-identity — as exactly those four, and the synchronous backends correctly reject it rather than silently downgrading. Second, the polled discipline is offered by only one backend, `mp-tcp-poll`, so a schedule that specifically demands `notify=poll` isolates that backend's notification path. The matrix is what makes these required-column choices mechanical rather than ad hoc: the set of backends a cell requires is computed from the schedule's demands against these declarations, and the harness then holds exactly that set to byte-identity.

Appendix D

Reproducibility

Every result in this document is reproducible from a pinned environment, which is itself part of the contribution: a differential test across backends is only trustworthy if the toolchain that builds those backends is identical between them and over time (section 9.6). This appendix records how to obtain the environment and how to build and run the compiler, the example matrix, and this document. Commands are given as task-runner recipes; the recipe names are stable entry points, and a reader who prefers the underlying invocations can read them off the `Justfile` at the repository root.

D.1 The pinned environment

The entire build environment — the Rust compiler version, every crate dependency, the message-passing libraries, the embedded cross-compiler, and the instruction-set simulator — is described by a Nix flake [8], a content-addressed, declaratively pinned specification, so that the same inputs reproducibly materialise the same environment on any machine that evaluates it. Entering the development shell materialises exactly that environment:

```
nix develop
```

All compiler and matrix commands below are run inside that shell. The single pinned toolchain is also the sole source of truth for the minimum supported compiler version, so there is no second place for it to drift. Two heavier tiers have their own shells, entered on demand: a message-passing shell carrying a local launcher for the cluster tier, and a simulator closure for the embedded tier, both described in appendix D.3 below. Keeping these out of the default shell is deliberate, so that the common case — building the compiler and running the tier-1 matrix — needs only the light environment.

D.2 Building and testing the compiler

The compiler is a Rust workspace. Within the development shell:

```
just build          # build all crates in the workspace
just test           # unit tests, development profile (debug assertions on)
just test-release   # unit tests, release profile
just check          # fast type-check without code generation
just clippy         # lint; warnings are errors
```

The release-profile test run is not redundant with the development one: debug assertions are stripped in release, so any test that pins a debug-assertion bite would diverge between profiles, and running both makes that skew visible rather than hidden.

D.3 Running the example matrix

The tier-1 differential matrix — every required cell across the seven CPU backends — is a single recipe, and it is the gate that establishes the central byte-identity result:

```
just e2e            # full tier-1 differential matrix; a non-zero
                    # required-cell failure count fails the build
just ci             # the full hard gate: build, lint, both test
                    # profiles, the structural checks, and e2e
```

The harness emits one line per cell and a totals line; on success it reclaims its scratch directories, and on failure it retains them with a diagnostic so the offending cell can be inspected. The matrix can also be scoped to a milestone tier of required cells for a faster, narrower run. A single cell can be reproduced by hand by invoking the compiler driver to emit one project to a directory, then building and running that project against the example's fixed input and comparing the output bytes to the committed reference; because the compiler is deterministic, two such emissions of the same cell are byte-identical source.

The cluster and embedded tiers follow the compile-mandatory, run-best-effort discipline (section 9.5) and have their own recipes, each entering its own pinned shell:

```
just e2e-mpi              # the message-passing differential arm, run
                          # under a local mpiexec in the MPI shell;
                          # hard-fails if no launcher is present
just renode-multimcu-gate # the embedded co-simulation gate: emit the
                          # bare-metal binaries, cross-compile, co-
                          # simulate the wired microcontrollers, and
                          # byte-diff the captured output
```

Both diff against the same immutable reference files the tier-1 rig uses, so the cluster and embedded tiers are held to byte-exact value-correctness where their runtimes are present, not merely to compiling.

D.4 Building this document

The document is itself built from a pinned environment, separate from the compiler's, carrying a complete TEX distribution and the same task runner. From the document directory:

```
cd paper
nix develop --command just build   # lualatex + biber via latexmk
```

The build is reproducible in the same sense as the compiler's: the flake pins the TEX distribution, and a corresponding package output builds the PDF without a development shell at all, for a fully hermetic invocation. The figures are TikZ sources compiled inline with the document, so there is no separate figure-generation step to run first.

D.5 Reproducing the quantitative characteristics

The static figures of section 10.6 are collected with the same pinned toolchain. Per-crate code footprint (table 10.1) is a line count of each crate's `src/` tree:

```
tokei <crate>/src           # code lines, excluding comments and blanks
```

Generated-code size (table 10.2) and the analysis-net node counts (table 10.3) come from emitting one cell and inspecting the output. The driver emits a project with `--out` and the Petri net with `--emit-pn`; the generated lines are every emitted `.rs` except the verbatim `kernels.rs`, and the net nodes are the shape-bearing nodes of the emitted graph:

```
nucleus build --algo A.algo.nuc --sched S.sched.nuc \
              --backend B --out OUT --emit-pn NET.dot
find OUT -name '*.rs' ! -name kernels.rs | xargs cat | wc -l
grep -c 'shape=' NET.dot
```

Because emission is deterministic, these counts are exact and stable across runs. The compile wall-times of section 10.6 are medians of repeated `nucleus build` invocations and are machine-dependent; they are reported only to bound an order of magnitude, not as a controlled benchmark.

Bibliography

[1] Antmicro. *Renode: Open Source Simulation Framework for Embedded Systems*. https://renode.io/. Scriptable instruction-set simulator with multi-microcontroller co-simulation; accessed June 2026. 2010.

[2] Riyadh Baghdadi et al. "Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code". In: *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2019, pp. 193–205. DOI: 10.1109/CGO.2019.8661197.

[3] Michael Bauer et al. "Legion: Expressing Locality and Independence with Logical Regions". In: *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2012. DOI: 10.1109/SC.2012.71.

[4] Uday Bondhugula et al. "A Practical Automatic Polyhedral Parallelizer and Locality Optimizer". In: *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2008, pp. 101–113. DOI: 10.1145/1375581.1375595.

[5] Tianqi Chen et al. "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning". In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 578–594.

[6] Leonardo Dagum and Ramesh Menon. "OpenMP: An Industry-Standard API for Shared-Memory Programming". In: *IEEE Computational Science & Engineering* 5.1 (1998), pp. 46–55. DOI: 10.1109/99.660313.

[7] James Demmel and Hong Diep Nguyen. "Fast Reproducible Floating-Point Summation". In: *Proceedings of the 21st IEEE Symposium on Computer Arithmetic (ARITH)*. IEEE, 2013, pp. 163–172. DOI: 10.1109/ARITH.2013.9.

[8] Eelco Dolstra, Merijn de Jonge, and Eelco Visser. "Nix: A Safe and Policy-Free System for Software Deployment". In: *18th USENIX Large Installation System Administration Conference (LISA 04)*. USENIX Association, 2004, pp. 79–92. URL: https://edolstra.github.io/pubs/nspfssd-lisa2004-final.pdf.

[9] Paul Feautrier. “Some Efficient Solutions to the Affine Scheduling Problem. Part II. Multidimensional Time”. In: *International Journal of Parallel Programming* 21.6 (1992), pp. 389–420. DOI: 10.1007/BF01379404.

[10] Troels Henriksen et al. “Futhark: Purely Functional GPU-Programming with Nested Parallelism and In-Place Array Updates”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2017, pp. 556–571. DOI: 10.1145/3062341.3062354.

[11] Yuka Ikarashi et al. “Exocompilation for Productive Programming of Hardware Accelerators”. In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2022, pp. 703–718. DOI: 10.1145/3519939.3523446.

[12] Gilles Kahn. “The Semantics of a Simple Language for Parallel Programming”. In: *Information Processing 74: Proceedings of the IFIP Congress*. North-Holland, 1974, pp. 471–475.

[13] Edward A. Lee and David G. Messerschmitt. “Synchronous Data Flow”. In: *Proceedings of the IEEE* 75.9 (1987), pp. 1235–1245. DOI: 10.1109/PROC.1987.13876.

[14] Nicholas D. Matsakis and II Klock Felix S. “The Rust Language”. In: *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT)*. ACM, 2014, pp. 103–104. DOI: 10.1145/2663171.2663188.

[15] William M. McKeeman. “Differential Testing for Software”. In: *Digital Technical Journal* 10.1 (1998), pp. 100–107.

[16] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.0*. Message Passing Interface Forum. 2021. URL: https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf.

[17] Tadao Murata. “Petri Nets: Properties, Analysis and Applications”. In: *Proceedings of the IEEE* 77.4 (1989), pp. 541–580. DOI: 10.1109/5.24143.

[18] Mark Ruvald Pedersen. “A Domain-Specific Language and Compiler for Image Signal Processor Firmware”. Earlier prototype of the Nuc language (v1), targeting Hive ISP firmware through a single backend. Bibliographic details to be confirmed against the archived thesis record. MA thesis. Technical University of Denmark (DTU), 2013.

[19] Carl Adam Petri. “Kommunikation mit Automaten”. Schriften des Rheinisch-Westfälischen Instituts für Instrumentelle Mathematik an der Universität Bonn, Nr. 2. PhD thesis. Technische Hochschule Darmstadt, 1962.

[20] Jonathan Ragan-Kelley et al. “Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines”. In: *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2013, pp. 519–530. DOI: 10.1145/2491956.2462176.

[21] Joel H. Saltz, Ravi Mirchandaney, and Kay Crowley. “Run-Time Parallelization and Scheduling of Loops”. In: *IEEE Transactions on Computers* 40.5 (1991), pp. 603–612. DOI: 10.1109/12.88484.

[22] The Embassy Project. *Embassy: The Next-Generation Framework for Embedded Applications*. https://embassy.dev/. Async embedded Rust runtime; accessed June 2026. 2026.

[23] William Thies, Michal Karczmarek, and Saman Amarasinghe. “StreamIt: A Language for Streaming Applications”. In: *Compiler Construction, 11th International Conference (CC 2002)*. Vol. 2304. Lecture Notes in Computer Science. Springer, 2002, pp. 179–196. DOI: 10.1007/3-540-45937-5_14.

[24] Sven Verdoolaege. “isl: An Integer Set Library for the Polyhedral Model”. In: *Mathematical Software – ICMS 2010*. Vol. 6327. Lecture Notes in Computer Science. Springer, 2010, pp. 299–302. DOI: 10.1007/978-3-642-15582-6_49.

[25] Xuejun Yang et al. “Finding and Understanding Bugs in C Compilers”. In: *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2011, pp. 283–294. DOI: 10.1145/1993498.1993532.